

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Himoyaga ruxsat
“ATDT” kafedra mudiri
_____ Raximov N.O.
«_____» _____ 2026-yil

BITIRUV MALAKAVIY ISHI

**Mavzu: Real vaqtda xabar almashish imkonini beruvchi “Milliy
chat” dasturini ishlab chiqish**

Bitiruvchi (315-21 Dlo‘
guruh)

(imzo)

S.O. Sohinazarov

Ilmiy rahbar

(imzo)

Y.Sh. Yuldoshev

Taqrizchi

(imzo)

HFX bo‘yicha maslahatchi

(imzo)

S. Abdullayeva

Toshkent – 2026

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Fakultet: Dasturiy injiniring

Kafedra: Axborot texnologiyalarining dasturiy ta’minoti

Ta’lim yo‘nalishi: 60610600 – Dasturiy injiniring

TASDIQLAYMAN

“ATDT” kafedra mudiri

_____ Raximov N.O.

« ____ » _____ 2026-yil

**315-21 DIo‘ guruh talabasi Sohinazarov Sardorbek O‘rinboy o‘g‘lining
bitiruv malakaviy ishiga**

T O P S H I R I Q

1. BMI mavzusi: Real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqish.
2. BMI tasdiqlangan buyruq: « ____ » _____ 2026-yil № _____.
3. BMIni himoyaga topshirish muddati: _____.
4. BMIning boshlang‘ich ma’lumotlari: mavjud messengerlar (Telegram, WhatsApp, Signal) tahlili, real vaqtda xabar almashish texnologiyalari, .NET 8, ASP.NET Core, PostgreSQL, SignalR va Angular texnologiyalariga oid adabiyotlar hamda amaliyot materiallari.
5. Hisob-tushuntirish yozuvining mazmuni: sohaning tahlili va masalaning qo‘yilishi, tizim arxitekturasini va ma’lumotlar bazasini loyihalash, server va klient qismlarini ishlab chiqish, xavfsizlik choralari, foydalanish yo‘riqnomasi, hayot faoliyati xavfsizligi, xulosa.
6. Grafik materiallar ro‘yxati: tizim arxitekturasini, ER-diagramma, UML (use-case, sequence) diagrammalari, dastur interfeysi skrinshotlari va PowerPoint taqdimoti.
7. Topshiriq berilgan sana: _____.

Rahbar:

(imzo)

Y.Sh. Yuldoshev

Topshiriqni oldim:

(imzo)

S.O. Sohinazarov

8. BMIning alohida bo‘limlari bo‘yicha maslahatchilar:

№	Bo‘lim nomi	Maslahatchi	Topshiriq berdi	Topshiriq oldi
1	Asosiy qism	Y.Sh. Yuldoshev		
2	Hayot faoliyati xavfsizligi	S. Abdullayeva		

9. BMIning bajarish bo‘yicha kalendar grafik:

№	Malakaviy ish bo‘limlari	Bajarish muddati	Rahbar imzosi
1	Kirish, masalaning qo‘yilishi		
2	Soha tahlili, mavjud messenjerlarni o‘rganish		
3	Tizimni loyihalash (arxitektura, ma’lumotlar bazasi, UML)		
4	Dasturning server va klient qismlarini ishlab chiqish		
5	Hayot faoliyati xavfsizligi		
6	Xulosa, adabiyotlar ro‘yxati, ilovalar		
7	Taqdimot slaydlarini tayyorlash		
8	Dastlabki himoya		

Bitiruv malakaviy ish rahbari:

(imzo)

Y.Sh. Yuldoshev

Bitiruvchi:

(imzo)

S.O. Sohinazarov

ANNOTATSIYA

Ushbu bitiruv malakaviy ishi real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqishga bag‘ishlangan. Ishda zamonaviy messenjerlar va real vaqtda aloqa texnologiyalari tahlil qilingan, tizimning arxitekturası, ma’lumotlar bazasi va UML modellari loyihalangan. Dasturning server qismi .NET 8 platformasida ASP.NET Core, Entity Framework Core, PostgreSQL va SignalR texnologiyalari asosida Clean Architecture tamoyillari bo‘yicha, klient qismi esa Angular freymvorkida ishlab chiqilgan. Tizim elektron pochta orqali bir martalik kod va Google hisobi yordamida autentifikatsiya, shaxsiy va guruh chatlari, real vaqtda xabar almashish, “yozmoqda” va o‘qilganlik belgilari, rasm, video va fayllar yuborish, hikoyalar (stories) hamda onlayn holatni kuzatish imkoniyatlarini taqdim etadi. Tizimning o‘ziga xos imkoniyatlari — xabarlarini lotin yoki kirill yozuvida ko‘rsatish va yozuvdan qat’i nazar qidiruv, e-pochta domeni orqali tashkilot a’zolicini tasdiqlash va yopiq tashkilot guruhları hamda Web Crypto API asosida uchdan-uchgacha shifrlangan maxfiy chatlar. Ilova Docker konteyneri ko‘rinishida bulutli serverga joylashtirilgan.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке национального мессенджера «Milliy chat», обеспечивающего обмен сообщениями в реальном времени. В работе проанализированы современные мессенджеры и технологии обмена данными в реальном времени, спроектированы архитектура системы, база данных и UML-модели. Серверная часть разработана на платформе .NET 8 с использованием ASP.NET Core, Entity Framework Core, PostgreSQL и SignalR в соответствии с принципами Clean Architecture, клиентская часть — на фреймворке Angular. Система поддерживает аутентификацию по одноразовому коду, отправляемому на электронную почту, и через аккаунт Google, личные и групповые чаты, обмен сообщениями в реальном времени, индикаторы набора текста и прочтения, отправку изображений, видео и файлов, истории (stories), а также

отслеживание статуса «в сети». Особенности системы — отображение сообщений на латинице или кириллице по выбору пользователя и поиск независимо от алфавита, подтверждение принадлежности к организации по домену электронной почты с закрытыми группами организаций, а также секретные чаты со сквозным шифрованием на основе Web Crypto API. Приложение развёрнуто в облаке в виде Docker-контейнера.

ANNOTATION

This graduation qualification work is devoted to the development of “Milliy chat”, a national messenger that allows for real-time messaging. The work analyses modern messengers and real-time communication technologies and designs the system architecture, the database and the UML models. The server side is built on .NET 8 with ASP.NET Core, Entity Framework Core, PostgreSQL and SignalR following Clean Architecture principles, and the client side is built with the Angular framework. The system provides sign-in with a one-time code sent by e-mail or with a Google account, private and group chats, real-time messaging, typing and read indicators, sending images, videos and files, stories, and online presence tracking. Its distinctive features are showing messages in the Latin or Cyrillic Uzbek script chosen by each user with script-independent search, organization membership verified by the e-mail domain with private organization groups, and end-to-end encrypted secret chats built on the Web Crypto API. The application is deployed to the cloud as a Docker container.

MUNDARIJA

KIRISH	8
I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI.....	11
1.1. Messenjerlarning rivojlanishi va hozirgi holati	11
1.2. Mavjud messenjerlar tahlili.....	12
1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi.....	14
1.4. Real vaqtda aloqa texnologiyalari.....	15
1.5. Masalaning qo'yilishi	16
1-bob bo'yicha xulosa	18
II-BOB. "MILLIY CHAT" TIZIMINI LOYIHALASH	19
2.1. Texnologiyalarni tanlash va asoslash.....	19
2.2. Tizim arxitekturasini.....	20
2.3. Ma'lumotlar bazasini loyihalash	23
2.4. Tizimning UML modellari.....	26
2.5. Xavfsizlikni loyihalash	31
2.6. Maxfiy chatlar kriptografik protokolini loyihalash.....	33
2-bob bo'yicha xulosa	37
III-BOB. "MILLIY CHAT" DASTURINI ISHLAB CHIQISH.....	38
3.1. Server qismini ishlab chiqish	38
3.2. Real vaqtda xabar almashishni amalga oshirish	40
3.3. Media fayllar, guruhlar va hikoyalar.....	42
3.4. Lotin–kirill transliteratsiyasi va yozuvdan qat'i nazar qidiruv	44
3.5. Tashkilot rejimini amalga oshirish.....	47
3.6. Maxfiy chatlarni amalga oshirish.....	48

3.7. Klient qismini ishlab chiqish	50
3.8. Joylashtirish va testlash	51
3.9. Foydalanish yo‘riqnomasi.....	53
3-bob bo‘yicha xulosa	55
IV-BOB. HAYOT FAOLIYATI XAVFSIZLIGI	56
4.1. Dasturchining ergonomik ish joyini tashkil etish	56
4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong‘inlar sodir bo‘lganda xodimlarning harakatlari	62
4-bob bo‘yicha xulosa	69
XULOSA	70
FOYDALANILGAN ADABIYOTLAR	73
ILOVALAR	77
1-ilova. SignalR habi (ChatHub)	77
2-ilova. Guruh chatlari servisi (GroupService)	79
3-ilova. Bir martalik kodlarni xeshlash (PBKDF2).....	86
4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)	86
5-ilova. Guruh chatlari uchun unit testlar (parcha).....	88
6-ilova. O‘zbek yozuvlari o‘rtasida transliteratsiya (UzbekTransliterator).....	89
7-ilova. Maxfiy chatlar kriptografiyasi: kalitlarni chiqarish, shifrlash va ratchet (secret-crypto.ts, parcha)	93

KIRISH

Bugungi kunda odamlar o'rtasidagi muloqotning katta qismi telefon qo'ng'iroqlari va elektron pochtdan messenjerlarga ko'chdi. Messenjerlarda o'quv guruhlari va ish jamoalari tashkil etiladi, yangiliklar tarqatiladi, davlat va tijorat xizmatlari ko'rsatiladi. O'zbekistonda aholining asosiy qismi xorijiy kompaniyalarga tegishli Telegram va WhatsApp ilovalaridan foydalanadi. Natijada fuqarolarning yozishmalari va shaxsga doir ma'lumotlari mamlakatdan tashqaridagi, milliy nazoratga bo'ysunmaydigan serverlarda saqlanadi.

Mavzuning dolzarbligi.

“Raqamli O'zbekiston – 2030” strategiyasi (PF-6079-son Farmon, 2020-yil 5-oktabr) mahalliy dasturiy mahsulotlarni rivojlantirish va axborot xavfsizligini ta'minlashni ustuvor vazifa sifatida belgilaydi. “Shaxsga doir ma'lumotlar to'g'risida”gi Qonunga 2021-yilda kiritilgan o'zgartishlarga ko'ra fuqarolarning shaxsga doir ma'lumotlari respublika hududidagi texnik vositalarda saqlanishi lozim. Bu talablar ma'lumotlarni o'z serverida saqlaydigan milliy messenjer yaratishni dolzarb qiladi.

Texnik jihatdan ham masala murakkab: xabar, “yozmoqda” belgisi va onlayn holat real vaqtda, sahifani yangilamasdan yetkazilishi, katta yozishmalar tarixi tez yuklanishi, autentifikatsiya esa ishonchli bo'lishi kerak. Bularni hal etish qatlamli arxitektura, real vaqt protokollari va ma'lumotlar bazasini optimallashtirishni amalda qo'llashni talab qiladi.

Bitiruv malakaviy ishining maqsadi

Ishning maqsadi — foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan, shaxsiy va guruh yozishmalarida real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini loyihalash va ishlab chiqishdir.

Qo'yilgan maqsadga erishish uchun quyidagi vazifalar belgilandi:

- mavjud messenjerlarni va real vaqtda ma'lumot almashish texnologiyalarini tahlil qilib, tizim uchun eng maqbul yechimni tanlash;

- funksional va nofunktional talablarni aniqlash, Clean Architecture asosida tizim arxitekturasini, ma'lumotlar bazasini va UML modellarini loyihalash;
- bir martalik kod, Google hisobi, JWT va refresh token asosida xavfsiz autentifikatsiyani ishlab chiqish;
- .NET 8 va SignalR asosida shaxsiy va guruh chatlari, media fayllar, hikoyalar va onlayn holatni amalga oshirish, Angular'da moslashuvchan interfeys yaratish;
- lotin–kirill transliteratsiyasi, e-pochta domeni orqali tashkilot rejimi va uchdan-uchgacha shifrlangan maxfiy chatlarni ishlab chiqish;
- dasturni Docker konteynerida bulutli serverga joylashtirish va sinovdan o'tkazish;
- dasturchi ish joyining ergonomikasi va favqulodda vaziyatlarda evakuatsiya tadbirlarini ishlab chiqish.

Tadqiqot obyekti va predmeti.

Tadqiqot obyekti — foydalanuvchilar o'rtasida matnli va multimedia xabarlar almashishni ta'minlaydigan onlayn muloqot tizimlari. Tadqiqot predmeti — real vaqtda ishlaydigan messenjerning arxitekturasini, ma'lumotlar modeli va xabarlarni yetkazish mexanizmlarini .NET va Angular texnologiyalari yordamida amalga oshirish usullari.

Tadqiqot usullari.

Ishda qiyosiy tahlil, obyektga yo'naltirilgan loyihalash, UML yordamida modellashtirish, relatsion ma'lumotlar bazasini normallashtirish hamda iterativ ishlab chiqish va avtomatlashtirilgan testlash usullaridan foydalanildi.

Ishning ilmiy-amaliy yangiligi.

- o'zbek tilining lotin va kirill yozuvlari o'rtasida kontekstga bog'liq qoidalarni hisobga oluvchi transliteratsiya algoritmi ishlab chiqildi. Uning asosida har bir foydalanuvchi xabarlarni o'zi tanlagan yozuvda ko'radi, qidiruv esa yozuvga bog'liq bo'lmagan kalit bo'yicha ishlaydi;

- tashkilotni oldindan ro'yxatga olmasdan, tasdiqlangan e-pochta domeni orqali a'zolikni tasdiqlash va yopiq tashkilot guruhlarini avtomatik shakllantirish usuli taklif etildi;
- faqat brauzerning standart kriptografik primitivlari (X25519, HKDF, HMAC, AES-256-GCM) asosida oldinga maxfiylikka ega maxfiy chat protokoli ishlab chiqildi; server xabar mazmunini bilmasligi ma'lumotlar bazasi darajasida ko'rsatib berildi.

Ishning amaliy ahamiyati.

“Milliy chat” to'liq ishlaydigan veb-ilova bo'lib, uni tashkilotlar, ta'lim muassasalari va davlat idoralari o'z serverlarida ichki muloqot vositasi sifatida qo'llashi mumkin. Xodimlar korporativ pochta bilan kirishi bilanoq tashkilotning yopiq guruhiga qo'shiladi, maxfiy chatlar esa nozik yozishmalarni serverdan ham himoya qiladi.

Bitiruv malakaviy ishining tuzilishi.

Ish kirish, to'rt bob, xulosa, foydalanilgan adabiyotlar ro'yxati va ilovalardan iborat. Birinchi bobda mavjud messenjerlar va real vaqtda aloqa texnologiyalari tahlil qilinib, masala qo'yilgan. Ikkinchi bobda tizim arxitekturasini, ma'lumotlar bazasi, UML modellari, xavfsizlik va maxfiy chat protokoli loyihalangan. Uchinchi bobda dasturni ishlab chiqish, joylashtirish, sinash va foydalanish yo'riqnomasi keltirilgan. To'rtinchi bob hayot faoliyati xavfsizligiga bag'ishlangan. Ish 46 ta rasm, 19 ta jadval, 10 ta kod listingi va 7 ta ilovani o'z ichiga oladi.

I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI

1.1. Messenjerlarning rivojlanishi va hozirgi holati

Messenjer (ingl. instant messenger) — foydalanuvchilar o‘rtasida matnli, ovoqli, video va boshqa xabarlarini Internet orqali deyarli bir zumda yetkazadigan dasturiy ta‘minot. Elektron pochtdan farqli ravishda messenjerda muloqot dialog ko‘rinishida kechadi: xabar darhol yetkaziladi, jo‘natuvchi esa suhbatdosh onlayn ekanini, xabarni o‘qiganini yoki javob yozayotganini ko‘radi.

Birinchi ommaviy chat tizimi 1988-yilda yaratilgan IRC protokoli edi. 1996-yilda chiqqan ICQ kontaktlar ro‘yxati va onlayn holat tushunchalarini ommalashtirdi, 1999-yilda boshlangan Jabber loyihasi esa IETF standarti XMPP’ga asos bo‘ldi. Smartfonlar paydo bo‘lgach, 2009-yilda WhatsApp foydalanuvchini telefon raqami orqali identifikatsiya qilishni joriy etdi va messenjerni SMS’ning bepul muqobiliga aylantirdi. 2013-yilda ishga tushirilgan Telegram yozishmalarni serverda saqlaydigan “bulutli” yondashuvni, 2014-yilda chiqqan Signal esa keyinchalik WhatsApp’da ham qo‘llangan uchdan-uchgacha shifrlash protokolini taklif etdi.

Hozirgi messenjerlar quyidagi yo‘nalishlarda rivojlanmoqda:

- **xavfsizlik** — uchdan-uchgacha shifrlash, faol sessiyalarni boshqarish, o‘z-o‘zidan o‘chadigan xabarlar;
- **ko‘p qurilmalilik** — bitta hisobdan telefon, kompyuter va brauzerda bir vaqtda foydalanish;
- **ommaviy kommunikatsiya** — katta guruhlar, kanallar va 24 soatlik hikoyalar (stories);
- **platformaga aylanish** — botlar, mini-ilovalar va to‘lov tizimlari (WeChat kabi “super-ilova”lar).

WhatsApp foydalanuvchilari soni 2 milliarddan, Telegramning oylik faol foydalanuvchilari 1 milliarddan oshgan. Shu bilan birga, ayrim davlatlarda mahalliy messenjerlar ustunlik qiladi: Janubiy Koreyada KakaoTalk, Yaponiyada LINE,

Vyetnamda Zalo, Xitoyda WeChat. Bu misollar mahalliy ehtiyojlarni yaxshi hisobga olgan milliy messenjer xalqaro raqobatchilar bilan muvaffaqiyatli raqobatlasha olishini ko'rsatadi.

1.2. Mavjud messenjerlar tahlili

Tizimga qo'yiladigan talablarni aniqlash uchun O'zbekistonda keng qo'llaniladigan Telegram va WhatsApp, maxfiylik bo'yicha etalon hisoblangan Signal hamda WeChat tahlil qilindi.

Telegram.

Telegram o'zining MTProto 2.0 protokolidan foydalanadi. Oddiy chatlarda xabarlar mijoz va server o'rtasida shifrlanib, serverda saqlanadi, shu tufayli yozishmalar har qanday qurilmadan ochiladi. Uchdan-uchgacha shifrlash faqat alohida "maxfiy chat"larda qo'llanadi. 200 000 a'zoli guruhlar, kanallar, botlar va 2 GB gacha fayllar qo'llab-quvvatlanadi. Kuchli tomonlari — tezlik, bulutli sinxronizatsiya va boy funktsionallik; kamchiliklari — server kodining yopiqligi va serverlarning xorijda joylashgani.

WhatsApp.

WhatsApp Meta kompaniyasiga tegishli. 2016-yildan barcha yozishmalar Signal Protocol asosida uchdan-uchgacha shifrlanadi va serverda faqat yetkazilgunga qadar turadi. Guruhlar 1024 a'zogacha bo'ladi. Kamchiliklari — kodi yopiq, metama'lumotlar (kim, kim bilan, qachon yozishgani) esa kompaniya ixtiyorida qoladi.

Signal va WeChat.

Signal'ning klient va server kodlari to'liq ochiq, barcha yozishmalar sukut bo'yicha shifrlanadi, metama'lumotlar minimal saqlanadi. Biroq u funksional jihatdan kambag'alroq: kanallar, botlar va katta guruhlar yo'q. Tencent'ga tegishli WeChat esa messenjer, to'lov tizimi va mini-ilovalarni birlashtirgan "super-ilova" bo'lib, uchdan-uchgacha shifrlashni qo'llamaydi. Messenjerlarning taqqoslanishi 1.1-jadvalda keltirilgan.

Messenjerlarning qiyosiy tahlili

Mezon	Telegram	WhatsApp	Signal	WeChat	Milliy chat
Egasi (davlat)	Telegram (BAA)	Meta (AQSh)	Signal Foundation (AQSh)	Tencent (Xitoy)	Mahalliy ishlab chiquvchi
Ro'yxatdan o'tish	Telefon raqami	Telefon raqami	Telefon raqami	Telefon raqami	E-pochta kodi, Google
E2E shifrlash	Faqat maxfiy chatlarda	Sukut bo'yicha	Sukut bo'yicha	Yo'q	Maxfiy chatlarda (X25519, AES-GCM)
Yozishmalar saqlanishi	Bulutda (serverda)	Qurilmada	Qurilmada	Serverda	O'z serverida (bulutli)
Guruh hajmi	200 000	1024	1000	500	200
Kanallar, botlar	Bor	Kanallar bor	Yo'q	Bor	Kelajakda (sxemasi tayyor)
Hikoyalar	Bor	Bor (Status)	Bor	Bor	Bor
Server kodi	Yopiq	Yopiq	Ochiq	Yopiq	Mahalliy nazoratda
Ma'lumotlar joylashuvi	Xorijda	Xorijda	Xorijda	Xitoyda	Istalgan serverda, jumladan O'zbekistonda
Lotin/kirill yozuvida ko'rsatish	Yo'q	Yo'q	Yo'q	Yo'q	Bor (qidiruv ham)
Tashkilot tomonidan tasdiqlash	Yo'q	Yo'q	Yo'q	Yo'q	E-pochta domeni orqali

Mavjud messenjerlar funksionallik va qulaylik bo'yicha yuqori darajada, ammo barchasi xorijiy kompaniyalar tomonidan boshqariladi va ma'lumotlarni mamlakatdan tashqarida saqlaydi. Funksionallik va qulaylik muvozanati eng yaxshi bo'lgani uchun foydalanuvchi tajribasi bo'yicha namuna sifatida Telegram tanlandi. Ro'yxatdan o'tish esa pullik SMS-shlyuz o'rniga elektron pochta va Google hisobi orqali amalga oshiriladi.

1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi

Messenjer foydalanuvchi haqidagi eng shaxsiy ma'lumotlarni — yozishmalar, fayllar, kontaktlar, kim bilan qachon muloqot qilingani, IP-manzillar va qurilmalarni saqlaydi. Xorijiy messenjerlardan foydalanishda quyidagi xavflar mavjud:

- **yurisdiksiya xavfi** — ma'lumotlar boshqa davlat qonunlariga bo'ysunadi va ularni o'sha davlat organlari talab qilishi mumkin;
- **metama'lumotlar xavfi** — xabar mazmuni shifrlangan bo'lsa ham, muloqot grafigi xizmat egasida qoladi;
- **xizmatning uzilishi** — shartlar o'zgarishi, hisob bloklanishi yoki mintaqaviy cheklov foydalanuvchini aloqadan mahrum qiladi;
- **nazorat qilib bo'lmazlik** — yopiq server kodi ma'lumotlar qanday qayta ishlanishini tekshirishga imkon bermaydi.

“Shaxsga doir ma'lumotlar to'g'risida”gi O'RQ-547-son Qonun (2019) shaxsga doir ma'lumotlarni yig'ish, saqlash va himoya qilish tartibini belgilaydi. 2021-yilgi qo'shimchalarga ko'ra fuqarolarning shaxsga doir ma'lumotlarini Internet orqali qayta ishlovchi operatorlar ularni respublika hududidagi ma'lumotlar bazalarida saqlashi shart. Milliy messenjer bu talabga tabiiy ravishda javob beradi: uning serverlari mamlakat ichidagi data-markazda joylashadi, kodi mahalliy mutaxassislar nazoratida bo'ladi.

Milliy messenjer mahalliy ehtiyojlarni ham hisobga oladi. O'zbekistonda lotin yozuvi rasmiy bo'lsa-da, kirill yozuvi hanuz keng qo'llanadi: katta avlod kirillda, yoshlar lotinda yozadi. Xorijiy messenjerlarda bu yozuvlar o'rtasida bog'lanish yo'q — lotincha qidiruv kirillcha xabarni topmaydi. Shuningdek, ta'lim muassasalari va tashkilotlar uchun a'zoning haqiqatan shu tashkilotga tegishli ekanini tasdiqlash muhim. Telefon raqamiga asoslangan messenjerlar buni ta'minlay olmaydi, tashkilotning e-pochta domeni esa bunday tasdiq uchun tabiiy vositadir.

1.4. Real vaqtda aloqa texnologiyalari

Messenjering asosiy texnik vazifasi — yangi xabarni qabul qiluvchiga imkon qadar tez yetkazish. HTTP protokoli “so‘rov–javob” modeliga asoslangan va server o‘z tashabbusi bilan mijozga ma’lumot yubora olmaydi. Shu sababli serverdan mijozga ma’lumot “itarish” (push) uchun bir necha usul ishlab chiqilgan.

Short polling usulida mijoz har bir necha soniyada “yangi xabar bormi?” deb so‘raydi: so‘rovlarning aksariyati bo‘sh qaytadi, xabar esa interval qadar kechikadi. **Long polling**’da server yangi ma’lumot paydo bo‘lguncha javobni ushlab turadi; kechikish kamayadi, lekin har bir xabar uchun yangi HTTP so‘rovi kerak. **Server-Sent Events** (SSE) bitta HTTP ulanish orqali hodisalarni uzluksiz yuboradi, ammo faqat serverdan mijozga yo‘nalishda ishlaydi.

WebSocket (RFC 6455) ulanishi oddiy HTTP so‘rovi bilan boshlanib, “101 Switching Protocols” javobidan so‘ng ikki tomonlama (full-duplex) kanalga aylanadi. Shundan keyin mijoz ham, server ham istalgan vaqtda kichik freymmlar yuboradi. Minimal kechikish va qo‘shimcha trafik tufayli WebSocket zamonaviy veb-messenjrlarning asosiy transporti hisoblanadi.

ASP.NET Core SignalR — Microsoft kutubxonasi bo‘lib, transport darajasini abstraksiya qiladi: avval WebSocket orqali ulanadi, u ishlamasa avtomatik ravishda SSE yoki long polling’ga o‘tadi. Dasturchi “hab” (Hub) abstraksiyasi bilan ishlaydi: mijoz server metodlarini, server esa mijoz funksiyalarini masofadan chaqiradi. Ulanishlarni guruhlash, ma’lum foydalanuvchiga xabar yuborish, JWT autentifikatsiyasi va qayta ulanish tayyor holda beriladi. Texnologiyalar 1.2-jadvalda taqqoslangan.

1.2-jadval

Real vaqtda aloqa texnologiyalarining taqqoslanishi

Texnologiya	Yo‘nalish	Kechikish	Server yuklamasi	Brauzerda qo‘llab-quvvatlash
Short polling	Mijoz → server	Yuqori (interval)	Yuqori	Barcha brauzerlar
Long polling	Mijoz → server	O‘rtacha	O‘rtacha	Barcha brauzerlar
Server-Sent Events	Server → mijoz	Past	Past	Zamonaviy brauzerlar

Texnologiya	Yo‘nalish	Kechikish	Server yuklamasi	Brauzerda qo‘llab-quvvatlash
WebSocket	Ikki tomonlama	Juda past	Past	Zamonaviy brauzerlar
SignalR	Ikki tomonlama (WebSocket, SSE yoki long polling)	Juda past	Past	Barcha brauzerlar (fallback hisobiga)

Tahlil asosida “Milliy chat”da real vaqt aloqasi uchun SignalR tanlandi: u WebSocket samaradorligini saqlagan holda cheklangan tarmoqlarda ham ishlaydi, .NET server bilan tabiiy integratsiyalashadi, brauzer uchun esa rasmiy @microsoft/signalr kutubxonasi mavjud.

1.5. Masalaning qo‘yilishi

Tahlil natijalariga ko‘ra, Telegram namunasi asosida veb-brauzerda ishlaydigan, ma’lumotlarni o‘z serverida saqlaydigan va real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqish talab etiladi. Tizimda uch turdagi aktor bor: tizimga hali kirmagan **mehmon**, ro‘yxatdan o‘tgan **foydalanuvchi** va **guruh administratori (egasi)**.

Funksional talablar. Tizim quyidagi imkoniyatlarni ta’minlashi kerak:

1. elektron pochtaga yuboriladigan 6 xonali bir martalik kod yoki Google hisobi orqali ro‘yxatdan o‘tish va kirish;
2. profilni (ism, username, bio, rasm) tahrirlash;
3. faol sessiyalar (qurilmalar) ro‘yxatini ko‘rish va ularni yakunlash;
4. foydalanuvchilarni ism yoki username bo‘yicha qidirish;
5. shaxsiy chat yaratish, chatlar ro‘yxatini oxirgi xabar va o‘qilmaganlar soni bilan ko‘rish;
6. guruh yaratish, a’zolar va administratorlarni boshqarish, guruh ma’lumotlarini o‘zgartirish;
7. xabar yuborish, javob berish (reply), tahrirlash, o‘chirish va chat tarixini tozalash;
8. rasm, video va fayllar yuborish, chatdagi media fayllarni ko‘rish;

9. chat ichida xabarlarni matn bo'yicha qidirish;
10. yangi xabarlar, "yozmoqda" holati, o'qilganlik belgilari va onlayn holatni real vaqtda ko'rsatish;
11. 24 soatlik hikoyalar (stories) joylash va ko'rish;
12. yorug' va qorong'i mavzular, mobil qurilmalarga moslashuvchan interfeys;
13. xabarlarni tanlangan yozuvda (asl holida, lotin yoki kirill) ko'rsatish va yozuvdan qat'i nazar qidirish;
14. e-pochta domeni orqali tashkilot a'zolarini tasdiqlash, yopiq tashkilot guruhlari va taklif havolalari;
15. uchdan-uchgacha shifrlangan maxfiy chat: kalitni tekshirish, o'z-o'zini o'chirish taymeri, shifrlangan fayllar.

Nofunksional talablar:

- **unumdorlik** — ro'yxatlarni kursorli sahifalash, real vaqt xabarlarini 1 soniyadan kam kechikish bilan yetkazish;
- **xavfsizlik** — HTTPS, qisqa muddatli access token, xavfsiz cookie'dagi refresh token, kod va tokenlarni xesh ko'rinishida saqlash, fayllarni antivirus orqali tekshirish;
- **ishonchlilik** — xatolarni markazlashgan qayta ishlash, ulanish uzilganda avtomatik qayta ulanish;
- **kengaytiriluvchanlik** — holatsiz API, yangi funktsiyani boshqa qismlarga ta'sir qilmasdan qo'shish;
- **qo'llab-quvvatlanuvchanlik** — Clean Architecture qatlamlari, yagona kod uslubi, versiyalar nazorati;
- **ko'chiriluvchanlik** — Docker konteynerida istalgan serverga joylashtirish;
- **qulaylik** — o'zbek tilidagi, Telegram foydalanuvchilariga tanish interfeys.

1-bob bo'yicha xulosa

Birinchi bobda messenjerlarning rivojlanishi ko'rib chiqildi va mavjud messenjerlar funktsionallik, xavfsizlik hamda ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi ma'lumotlarni mamlakatdan tashqarida saqlashi aniqlandi, bu esa lokalizatsiya talablari nuqtai nazaridan milliy messenjer yaratish zaruriyatini asoslaydi. Real vaqt aloqasi uchun WebSocket va zaxira transportlarni avtomatik tanlaydigan SignalR tanlandi. Bob yakunida 15 ta funktsional va 7 guruh nofunktsional talab aniqlanib, masala qo'yildi.

II-BOB. “MILLIY CHAT” TIZIMINI LOYIHALASH

2.1. Texnologiyalarni tanlash va asoslash

Texnologiyalarni tanlashda quyidagi mezonlar qo'yildi: real vaqtda ikki tomonlama aloqa, yuqori unumdorlik, tayyor va ishonchli xavfsizlik vositalari, ekotizimning yetukligi, ochiq kodli litsenziya va konteynerlarda joylashtirish qulayligi.

Server qismi uchun **.NET 8** (LTS) platformasi va **ASP.NET Core** freymvorki tanlandi. Ular kross-platformali, Linux konteynerlarida ishlaydi va mustaqil sinovlarda eng tezkor veb-freymvorklar qatorida turadi. Muqobil sifatida Node.js va Spring Boot ko'rib chiqildi, ammo hal qiluvchi omil **SignalR** kutubxonasining platforma tarkibiga kirishi bo'ldi: WebSocket va zaxira transportlar, hablar, guruhlar va JWT autentifikatsiyasi qo'shimcha kutubxonalarsiz ishlaydi. Ma'lumotlar bazasi sifatida ACID tranzaksiyalar, sxemalar va murakkab indeksni qo'llab-quvvatlaydigan **PostgreSQL** tanlandi, u bilan **Entity Framework Core 8** ORM orqali ishlanadi.

Klient qismi **Angular 22** freymvorkida yozildi: u marshrutlash, formalar, HTTP klient va DI'ni bitta ekotizimda beradi, TypeScript'ning qat'iy tiplanishi esa server qismidagi C# bilan uslubiy uyg'unlikni ta'minlaydi. Server ilovasi **Docker** konteyneriga yig'ilib, **Render** bulutli platformasida ishga tushirilgan. Tanlangan texnologiyalar 2.1-jadvalda keltirilgan.

2.1-jadval

Tizimda qo'llanilgan texnologiyalar

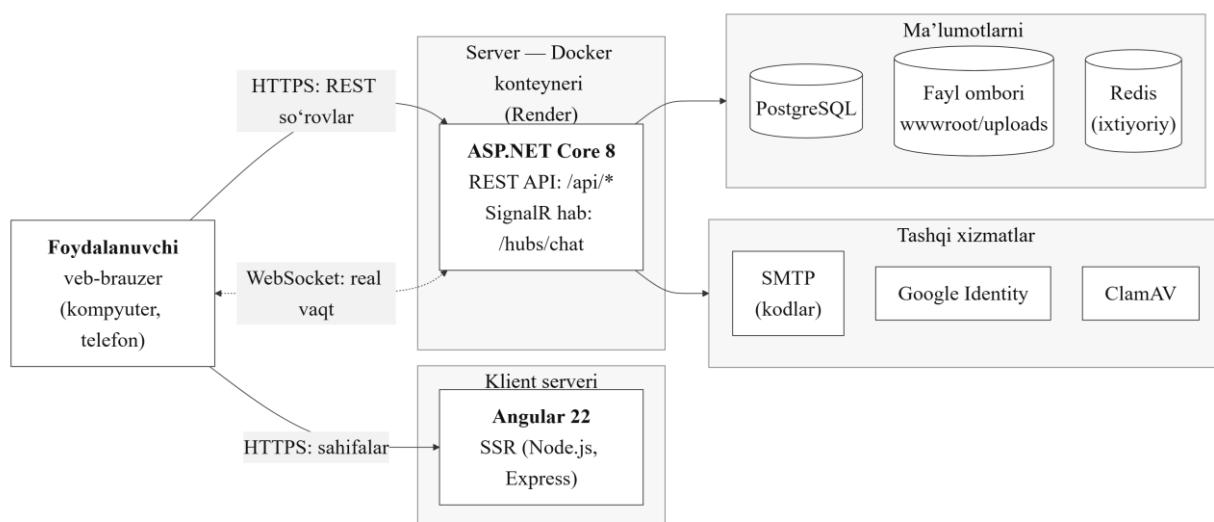
Qatlam	Texnologiya	Versiya	Vazifasi
Server	.NET, ASP.NET Core	8.0 (LTS)	Web API, middleware, DI, konfiguratsiya
Server	ASP.NET Core SignalR	8.0	Real vaqtda ikki tomonlama aloqa (hab)
Server	Entity Framework Core, Npgsql	8.0.8	ORM, LINQ so'rovlari, migratsiyalar
Server	FluentValidation	11.11	Kiruvchi ma'lumotlarni validatsiya qilish

Qatlam	Texnologiya	Versiya	Vazifasi
Server	System.IdentityModel.Tokens.Jwt	8.3.1	JWT access tokenlarni yaratish va tekshirish
Server	Google.Apis.Auth	1.68	Google ID tokenini tekshirish
Server	SixLabors.ImageSharp	3.1.12	Rasmlarni tekshirish va thumbnail yaratish
Server	StackExchange.Redis	2.8.16	Onlayn holatni taqsimlangan saqlash
Ma'lumotlar	PostgreSQL	16+	Relatsion ma'lumotlar bazasi
Klient	Angular, TypeScript, RxJS	22, 6.0, 7.8	SPA freymvork, tiplashtirilgan til, reaktiv oqimlar
Klient	@microsoft/signalr	10.0	Brauzerdagi SignalR klienti
Klient	Web Crypto API, IndexedDB	brauzer	Maxfiy chatlarda shifrlash va kalitlarni qurilmada saqlash
Infratuzilma	Docker, Render, Git/GitHub	—	Konteynerlash, joylashtirish, versiyalar nazorati

2.2. Tizim arxitekturası

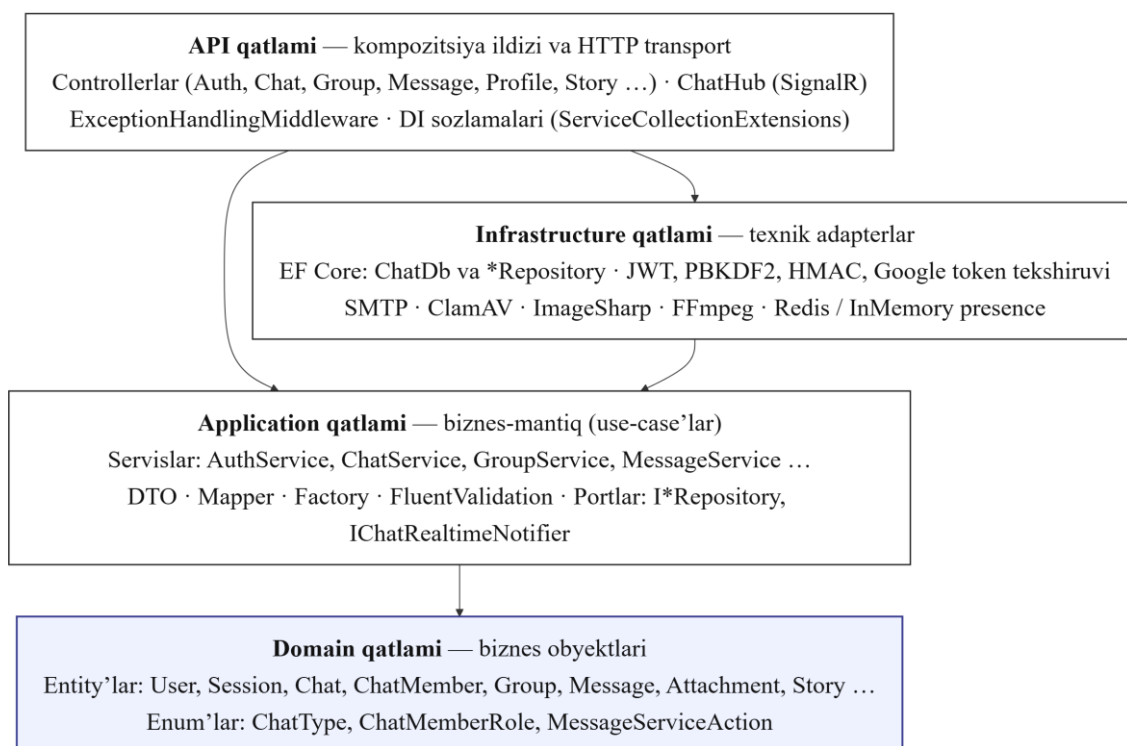
“Milliy chat” klient-server arxitekturası asosida qurilgan (2.1-rasm). Brauzerdagi Angular ilovasi server bilan ikki kanal orqali aloqa qiladi: holatni o‘zgartiruvchi barcha amallar (xabar yuborish, profilni tahrirlash, guruh yaratish) uchun HTTPS ustidagi **REST API**, serverdan mijozga hodisalarni (yangi xabar, o‘qilganlik, “yozmoqda”, onlayn holat) tezkor yetkazish uchun esa WebSocket ustidagi **SignalR** ulanishi. Tashqi xizmatlardan SMTP server bir martalik kodlarni yetkazadi, Google Identity kirish uchun ID token beradi, ClamAV esa yuklanayotgan fayllarni tekshiradi.

Server holatsiz (stateless) loyihalangan: sessiyalar ma'lumotlar bazasida, autentifikatsiya har bir so‘rovdagi JWT tokenda saqlanadi. Shu sababli bir necha server nusxasini ishga tushirish mumkin, bunda faqat onlayn holat va SignalR ulanishlarini Redis’da umumlashtirish talab etiladi.



2.1-rasm. Tizimning umumiy arxitekturası va joylashtirish sxemasi

Server kodi Robert Martin taklif etgan **Clean Architecture** tamoyillari asosida to'rtta loyihaga ajratilgan (2.2-rasm). Asosiy qoida — bog'liqliklar faqat tashqi qatlamlardan ichki qatlamlarga yo'naladi, shuning uchun biznes-qoidalar ma'lumotlar bazasi yoki freymvork almashtirilganda o'zgarmaydi va alohida sinalishi mumkin.



2.2-rasm. Server qismining Clean Architecture qatlamlari

- **Domain** — biznes obyektlari (User, Chat, ChatMember, Message, Story, Organization, SecretChat va boshqalar), sanab o‘tiladigan turlar va o‘zbek yozuvlari transliteratori. Hech bir loyihaga bog‘liq emas;
- **Application** — foydalanish holatlarini amalga oshiruvchi servislar, DTO‘lar, validatorlar hamda tashqi dunyo bilan aloqa uchun interfeyslar (IMessageRepository, IChatRealtimeNotifier, IEmailSender, IFileStorage). Kod funksiyalar (Chats, Groups, Messages, Organizations, SecretChats...) bo‘yicha guruhlangan;
- **Infrastructure** — interfeyslarning texnik amalga oshirilishi: EF Core konteksti va repozitoriylar, JWT, PBKDF2 va HMAC xeshlovchilari, SMTP, ClamAV, ImageSharp va FFmpeg ishlov beruvchilari, fon ishlari;
- **API** — REST controllerlar, SignalR habi (ChatHub), middleware‘lar va bog‘liqliklarni ro‘yxatga olish.

Application qatlami SignalR haqida hech narsa bilmaydi: xabar saqlangach, MessageService faqat IChatRealtimeNotifier interfeysini chaqiradi, uning SignalR‘dagi amalga oshirilishi esa API qatlamida joylashgan (Dependency Inversion tamoyili). Real vaqt xabarnomasi tranzaksiya muvaffaqiyatli yakunlangandan keyingina yuboriladi, shuning uchun mijoz bazaga yozilmagan xabar haqida bildirishnoma olmaydi.

Angular ilovasi ham qatlamlarga ajratilgan. **core** papkasida autentifikatsiya servisi, tokenni qo‘shuvchi va muddati o‘tganda yangilovchi HTTP interceptor, guard va mavzu servisi joylashgan. **features/chat** modulida REST so‘rovlari va holat signallarini boshqaruvchi ChatApiService hamda SignalR ulanishini boshqaruvchi ChatRealtimeService bor. Maxfiy chatlarning kriptografik qismi Angular‘ga bog‘liq bo‘lmagan alohida modulda, **shared** papkasida esa transliterator va dizayn tizimi komponentlari joylashgan. Barcha ranglar CSS o‘zgaruvchilari orqali berilgani uchun yorug‘ va qorong‘i mavzular faqat o‘zgaruvchilar qiymatini almashtirish bilan o‘tadi.

2.3. Ma'lumotlar bazasini loyihalash

Ma'lumotlar bazasi "kod birinchi" (code-first) yondashuvda loyihalandi: jadvallar Domain'dagi entity sinflari va EF Core konfiguratsiyalari asosida yaratiladi, sxemaning har bir o'zgarishi migratsiya sifatida saqlanadi. 36 ta jadval vazifasiga ko'ra PostgreSQL'ning o'nta sxemasiga ajratilgan (2.2-jadval). Ularning bir qismi (kanallar, qo'ng'iroqlar, stikerlar, botlar, so'rovnomalar) kelajakdagi funksiyalar uchun oldindan tayyorlangan.

2.2-jadval

Ma'lumotlar bazasi sxemalari va jadvallari

Sxema	Jadvallar	Vazifasi
identity	users, contacts, blocked_users	Foydalanuvchilar va ular o'rtasidagi aloqalar
security	sessions, email_verification_codes, two_factor_auth	Sessiyalar, bir martalik kodlar, qo'shimcha himoya
chat	chats, chat_members, groups, channels, channel_subscribers, secret_chats	Chatlar, a'zolik va rollar, guruh va kanallar, maxfiy chatlar
messaging	messages, attachments, message_views, reactions, polls, poll_options, poll_votes, secret_messages, secret_files	Xabarlar, biriktirmalar, o'qilganlik, reaksiya va so'rovnomalar, yetkazilmagan shifrlangan xabar va fayllar
organizations	organizations, organization_members	Tashkilotlar (e-pochta domenlari) va tasdiqlangan a'zolik
storage	files, photos, stickers, sticker_sets	Fayllar metama'lumotlari, rasmlar, stikerlar
story	stories, story_views	Hikoyalar va ularni ko'rishlar
personal	folders, folder_chats, saved_messages, subscriptions	Shaxsiy papkalar, saqlangan xabarlar, obunalar
call	calls, call_participants	Ovozli va video qo'ng'iroqlar (kelajak uchun)
bot	bots	Botlar platformasi (kelajak uchun)

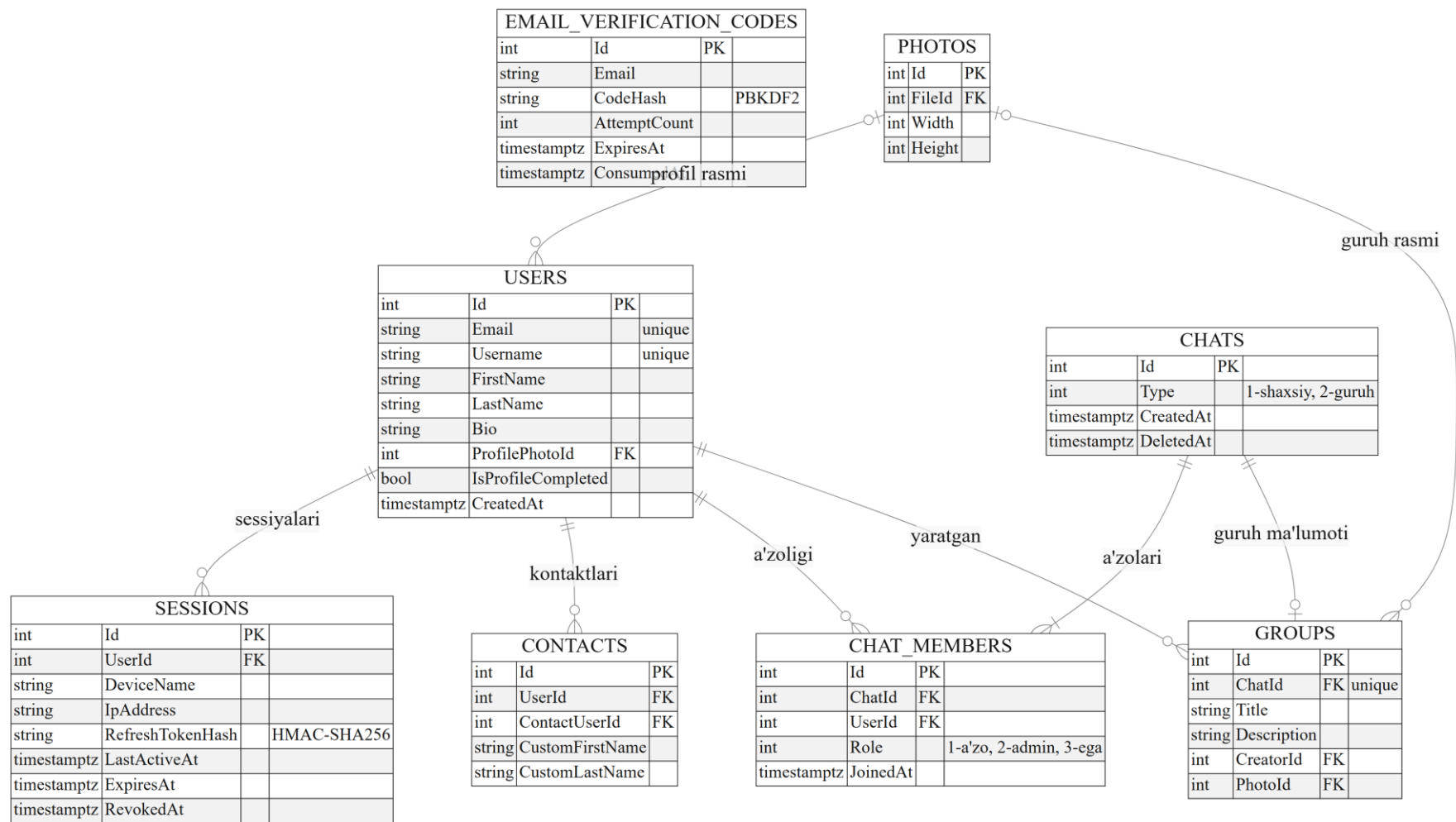
Modelning markazida **chats** jadvali turadi (2.3-rasm): shaxsiy chat ham, guruh ham bitta chats yozuvi bilan ifodalanadi va Type maydoni bilan farqlanadi. Ishtirokchilar **chat_members** jadvalida rollari (a'zo, administrator, ega) bilan saqlanadi, guruhga xos ma'lumotlar esa chats bilan "birga-bir" bog'langan **groups** jadvaliga chiqarilgan. Shu tufayli xabarlar, biriktirmalar, qidiruv va real vaqt

mexanizmlari shaxsiy chatlar va guruhlar uchun bir xil ishlaydi. **messages** jadvali o'ziga o'zi ReplyToMessageId orqali bog'langan (javob xabarlar), ServiceAction maydoni esa guruhdagi xizmat xabarlarini ("a'zo qo'shildi", "nom o'zgartirildi") farqlaydi.

Loyihalash qarorlari.

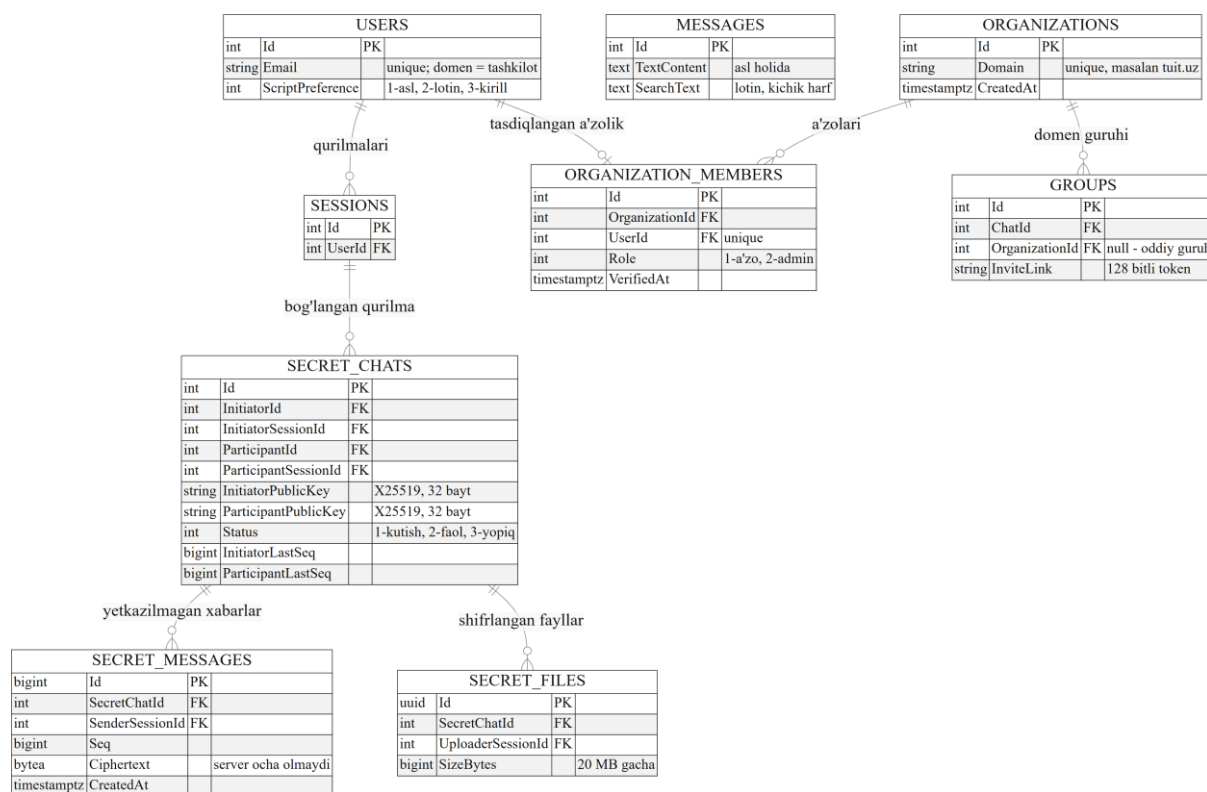
- **Vaqt** — barcha vaqtlar UTC'da, "timestamp with time zone" turida saqlanadi; mahalliy vaqtga faqat brauzerda o'giriladi;
- **Yumshoq o'chirish** — o'chirilgan chat va xabarlar DeletedAt maydoni bilan belgilanadi, EF Core global filtrlari ularni so'rovlardan avtomatik chiqaradi;
- **O'qilganlik** — message_views jadvali guruhda har bir a'zo uchun o'qilmaganlar sonini alohida hisoblash imkonini beradi;
- **Fayllar** — diskda saqlanadi, bazada faqat metama'lumotlari yoziladi; bitta fayl biriktirma, profil rasmi yoki hikoya sifatida qayta ishlatiladi;
- **Indeksrlar** — users(Email, Username), sessions(RefreshTokenHash), organizations(Domain) kabi unique va tez-tez bajariladigan so'rovlar uchun tarkibiy indekslar.

Chatlar ro'yxati va xabarlar tarixi uchun **kursorli (keyset) sahifalash** tanlandi. OFFSET/LIMIT usulida yangi xabar qo'shilganda sahifalar siljiydi va katta OFFSET'da so'rov sekinlashadi. Kursorli usulda mijoz "beforeId" va "limit" yuboradi, server esa birlamchi kalit indeksi bo'yicha limit+1 ta yozuvni o'qib, items, nextCursor va hasMore maydonlarini qaytaradi. So'rov tezligi tarix hajmiga bog'liq emas.



2.3-rasm. ER-diagramma: foydalanuvchilar, sessiyalar, chatlar va guruhlar

Tizimning o'ziga xos imkoniyatlari uchun qo'shilgan jadval va ustunlar 2.4-rasmda ko'rsatilgan. **organizations** jadvalida har bir tashkilot bitta e-pochta domeni bilan ifodalanadi, **organization_members** esa a'zolik va rolni saqlaydi; groups jadvalidagi OrganizationId ustuni guruhni tashkilotning yopiq guruhiga aylantiradi. **secret_chats** jadvalida shifrlash kalitining o'zi emas, faqat ikkala tomonning **ochiq** kalitlari va chat bog'langan sessiyalar saqlanadi, **secret_messages** va **secret_files** esa faqat hali yetkazilmagan shifrlangan bloblarni vaqtincha saqlaydi. Translitteratsiya uchun users jadvaliga ScriptPreference, messages jadvaliga esa matnning lotinga o'girilgan, kichik harfli nusxasi — SearchText ustuni qo'shildi.

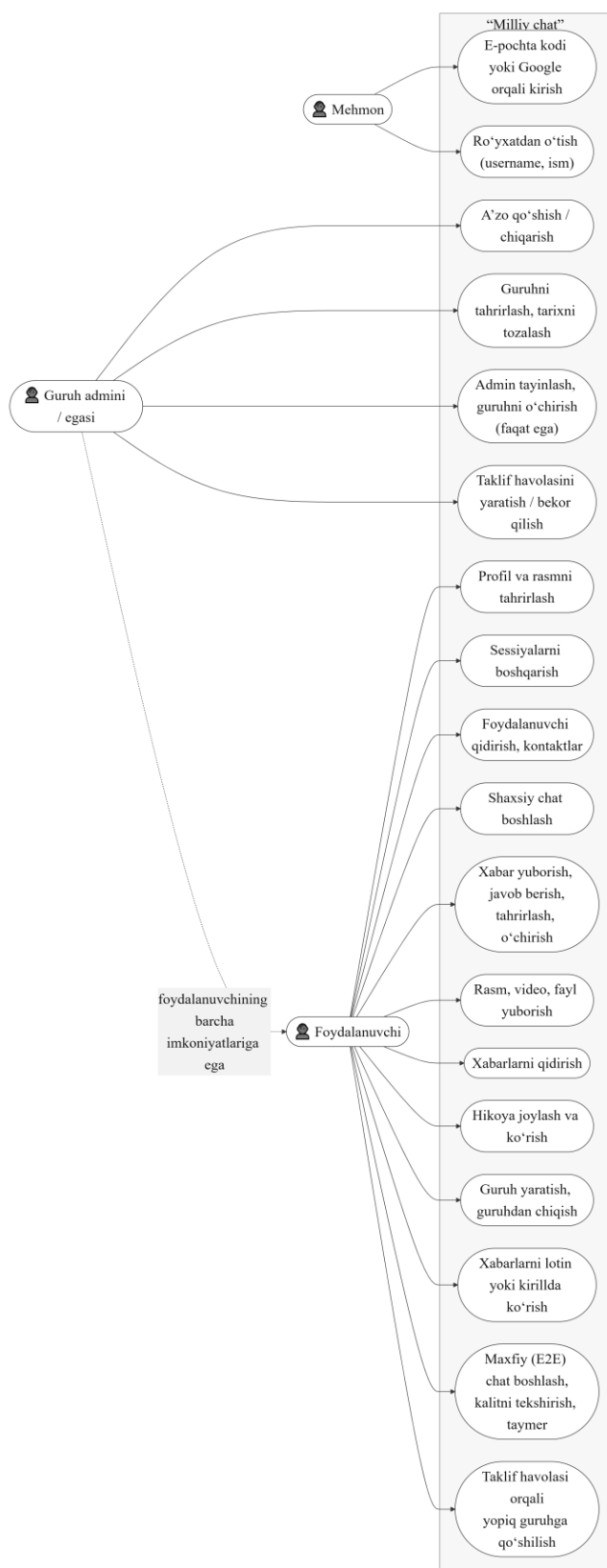


2.4-rasm. ER-diagramma: tashkilotlar, maxfiy chatlar va translitteratsiya uchun qo'shilgan jadval va ustunlar

2.4. Tizimning UML modellari

Tizimda uchta aktor mavjud. **Mehmon** faqat ro'yxatdan o'tishi yoki kirishi mumkin. **Foydalanuvchi** profilini boshqaradi, yozishadi, fayl va hikoyalar almashadi, guruh va maxfiy chat ochadi, xabarlarni o'zi tanlagan yozuvda ko'radi. **Guruh administratori yoki egasi** qo'shimcha ravishda guruh a'zolari, sozlamalari

va taklif havolasini boshqaradi (2.5-rasm). Guruhdagi ruxsatlar 2.3-jadvalda keltirilgan; ular server servislarida tekshiriladi, interfeys esa ruxsat berilmagan tugmalarni ko'rsatmaydi.



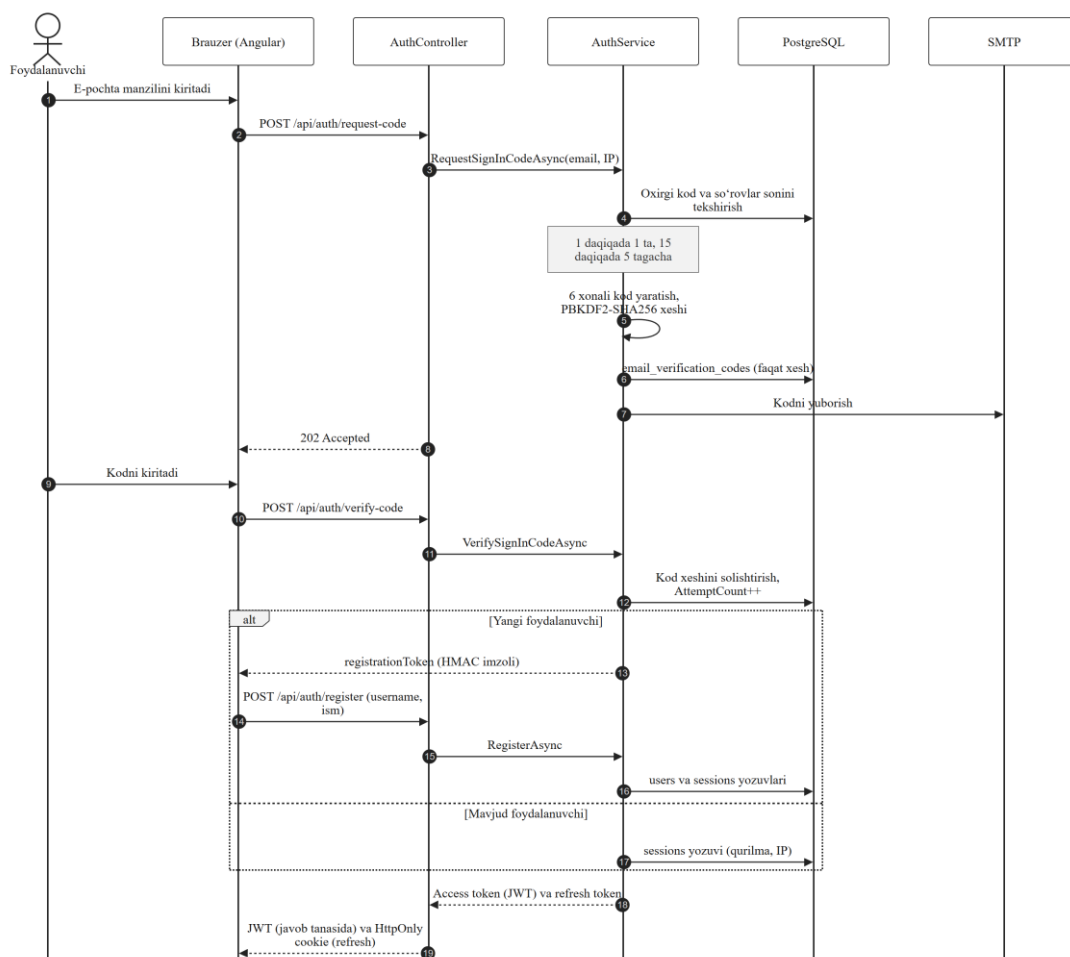
2.5-rasm. Foydalanish holatlari (use-case) diagrammasi

Guruh chatidagi rollar va ruxsatlar

Amal	A'zo	Administrator	Ega
Xabar yozish, o'qish, fayl yuborish	✓	✓	✓
A'zo qo'shish	—	✓	✓
A'zoni guruhdan chiqarish	—	Faqat oddiy a'zoni	✓ (hammani)
Administrator tayinlash yoki olib tashlash	—	—	✓
Guruh nomi, tavsifi va rasmini o'zgartirish	—	✓	✓
Taklif havolasini boshqarish	—	✓	✓
Guruhni butunlay o'chirish	—	—	✓
Guruhdan chiqish	✓	✓	✓ (egalik boshqa a'zoga o'tadi)

Elektron pochta orqali kirish jarayoni.

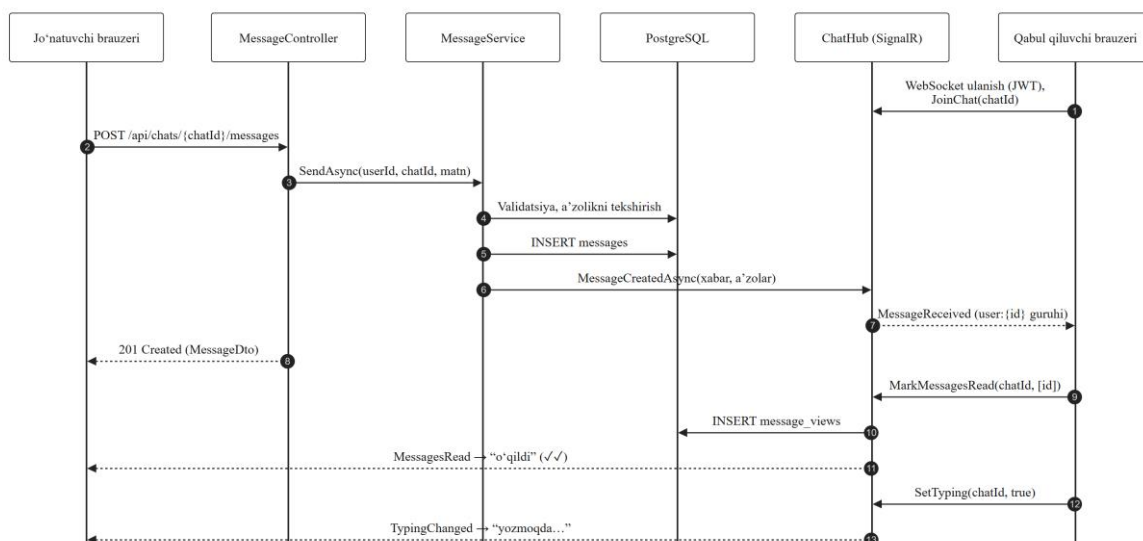
Kirish parolsiz: foydalanuvchi e-pochta manzilini kiritadi va unga yuborilgan 6 xonali kodni tasdiqlaydi (2.6-rasm). Server so'rovlar chastotasini cheklaydi (manzilga daqiqada bitta kod, 15 daqiqada manzil yoki IP'dan beshtadan ortiq emas), kodni kriptografik generator bilan yaratadi va bazaga faqat PBKDF2 xeshini yozadi. Kod 10 daqiqa amal qiladi, beshtagacha urinish beriladi. Yangi foydalanuvchi uchun server imzolangan vaqtinchalik token qaytaradi va ro'yxatdan o'tish yakunlanadi; mavjud foydalanuvchi uchun darhol sessiya yaratiladi.



2.6-rasm. Elektron pochtaga yuborilgan kod orqali kirish ketma-ketlik diagrammasi

Real vaqtda xabar yuborish jarayoni.

Brauzer tizimga kimgach, JWT token bilan SignalR habiga ulanadi va server ulanishni “user:{id}” guruhiga qo‘shadi — shu tufayli foydalanuvchining barcha qurilmalari hodisalarni bir vaqtda oladi (2.7-rasm). Xabarning o‘zi REST so‘rovi orqali yuboriladi: bu saqlash va validatsiyani oddiy HTTP semantikasi bilan bajarish imkonini beradi. Xabar bazaga yozilgach, chat a‘zolariga MessageReceived hodisasi yuboriladi. Qabul qiluvchi chatni ochib turgan bo‘lsa, MarkMessagesRead metodini chaqiradi va jo‘natuvchida belgi ikki belgiga (✓✓) almashadi. Guruh yaratish va a‘zo qo‘shish ham xizmat xabarlari orqali xuddi shu mexanizm bilan barcha a‘zolarga bir zumda yetkaziladi.

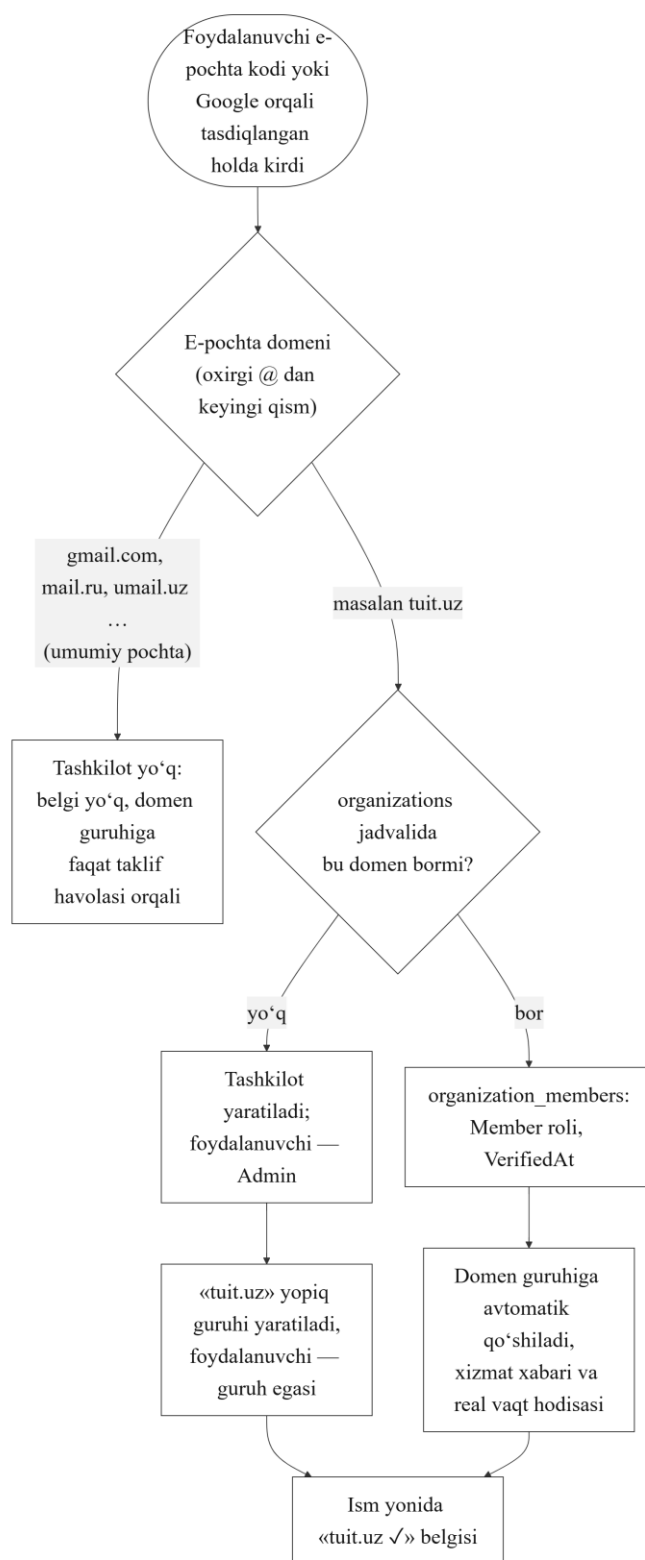


2.7-rasm. Real vaqtda xabar yuborish va o‘qilganlik belgisi ketma-ketlik diagrammasi

Tashkilotga avtomatik qo‘shilish jarayoni.

Tashkilot rejimi oldindan ro‘yxatni talab qilmaydi: **tashkilot — bu e-pochta domenining o‘zi**. Foydalanuvchi tasdiqlangan pochta bilan kirganda server “@” belgisidan keyingi domenni ajratib, normallashtiradi va faqat to‘liq tengligi bo‘yicha solishtiradi: “student.tuit.uz” alohida tashkilot, “evil-tuit.uz” esa hech qachon “tuit.uz” bilan mos kelmaydi. Umumiy pochta xizmatlari (gmail.com, mail.ru, umail.uz va 40 ga yaqin boshqa domen) tashkilot hisoblanmaydi.

Domendan birinchi kirgan foydalanuvchi tashkilot admini va uning yopiq guruhi egasi bo‘ladi, keyingilar esa guruhga avtomatik qo‘shiladi (2.8-rasm). Domen guruhi qidiruvda ko‘rinmaydi; tashqaridan unga faqat admin qo‘shganda yoki 128 bitli tasodifiy tokenli taklif havolasi orqali kirish mumkin. Bunday guruhga butun fakultet kirishi mumkinligi uchun a‘zolar chegarasi 200 o‘rniga 5000 qilib belgilangan.



2.8-rasm. Foydalanuvchini tashkilotga va uning domen guruhiga qo‘shish algoritmi

2.5. Xavfsizlikni loyihalash

Tizim parollardan foydalanmaydi, shuning uchun parollar bazasining sizib chiqishi yoki zaif parollar muammosi yo‘q. Bir martalik kodlar PBKDF2 (HMAC-

SHA256, 210 000 iteratsiya, tasodifiy tuz) bilan xeshlanadi va vaqtga bog‘liq bo‘lmagan usulda solishtiriladi. Google orqali kirishda ID tokenning imzosi, muddati va auditoriyasi tekshiriladi.

Kirishdan so‘ng ikki token beriladi. **Access token** — 15 daqiqa amal qiladigan, foydalanuvchi va sessiya identifikatorini saqlovchi JWT (HS256); u API so‘rovlarida va SignalR ulanishida yuboriladi. **Refresh token** — 30 kunlik tasodifiy qator bo‘lib, JavaScript o‘qiy olmaydigan HttpOnly, SameSite=Strict cookie‘da saqlanadi, bazaga esa faqat HMAC xeshi yoziladi. Access token muddati tugaganda klient interceptori uni avtomatik yangilaydi, refresh token esa har safar almashtiriladi (rotatsiya), shuning uchun o‘g‘irlangan eski token ishlamaydi.

Har bir kirish alohida sessiya yaratadi: foydalanuvchi barcha faol qurilmalarini ko‘radi va begona qurilmani chiqarib yuborishi mumkin. Avtorizatsiyada tokenning o‘zi yetarli emas — har bir chat bilan bog‘liq REST va hab chaqiruvida server foydalanuvchining shu chat a‘zosi ekanini bazadan tekshiradi. Kiruvchi ma’lumotlar FluentValidation bilan tekshiriladi, kutilmagan xatolar esa mijozga ichki tafsilotlarsiz qaytariladi.

Yuklanadigan rasm kengaytmasi bo‘yicha emas, tarkibi bo‘yicha tekshiriladi va qayta kodlanadi, ya’ni yashirin metama’lumot va zararli qo‘shimchalar olib tashlanadi. Fayllar tasodifiy nom bilan saqlanadi, faqat chat a‘zolariga beriladi va ClamAV bilan tekshiriladi. Klient ilovasi qat’iy **Content Security Policy** bilan himoyalangan: skriptlar faqat ilovaning o‘z manzilidan yuklanadi. Bu XSS’ga qarshi, ayniqsa maxfiy chat kalitlarini himoya qilish uchun muhim. Asosiy tahdidlar va himoya choralari 2.4-jadvalda umumlashtirilgan.

2.4-jadval

Asosiy tahdidlar va himoya choralari

Tahdid	Himoya choralari
Kodni tanlab topish (brute-force)	Kod 10 daqiqa amal qiladi, 5 tagacha urinish; kod so‘rash chastotasi e-pochta va IP bo‘yicha cheklangan
Ma’lumotlar bazasining sizib chiqishi	Kodlar PBKDF2, refresh tokenlar HMAC-SHA256 xeshi ko‘rinishida saqlanadi; parollar yo‘q
Tokenning o‘g‘irlanishi	Access token 15 daqiqa; refresh token HttpOnly cookie‘da va har safar rotatsiya qilinadi; sessiyani masofadan yakunlash mumkin

Tahdid	Himoya choralari
CSRF (so‘rovni soxtalashtirish)	Refresh cookie SameSite=Strict va faqat /api/auth yo‘li uchun; boshqa so‘rovlar Authorization sarlavhasini talab qiladi
XSS (skript kiritish)	Angular shablonlari ma’lumotlarni avtomatik ekranlaydi; qat’iy CSP; refresh token JavaScript’ga ko‘rinmaydi
Begona chatga kirish	Har bir REST va hab chaqiruvida a’zolik tekshiruvi; guruhda rolga asoslangan ruxsatlar
Zararli yoki soxta fayl	Hajm cheklovlari, tarkib bo‘yicha format tekshiruvi, rasmni qayta kodlash, ClamAV skaneri
Tinglab olish (MITM)	Barcha trafik HTTPS/WSS orqali; ishlab chiqarish muhitida cookie faqat Secure rejimida
Server yoki ma’lumotlar bazasi buzilishi	Maxfiy chatlarda uchdan-uchgacha shifrlash: serverda faqat ochiq kalitlar va vaqtincha shifrlangan bloblar
Xabarni takrorlash (replay)	Maxfiy chatda har xabar tartib raqami bilan: takrori rad etiladi; AES-GCM AAD xabarni chat, yo‘nalish va raqamga bog‘laydi
Yopiq guruhga ruxsatsiz kirish	Guruh qidiruvda ko‘rinmaydi; 128 bitli taklif tokeni; havolani yangilash yoki bekor qilish mumkin

2.6. Maxfiy chatlar kriptografik protokolini loyihalash

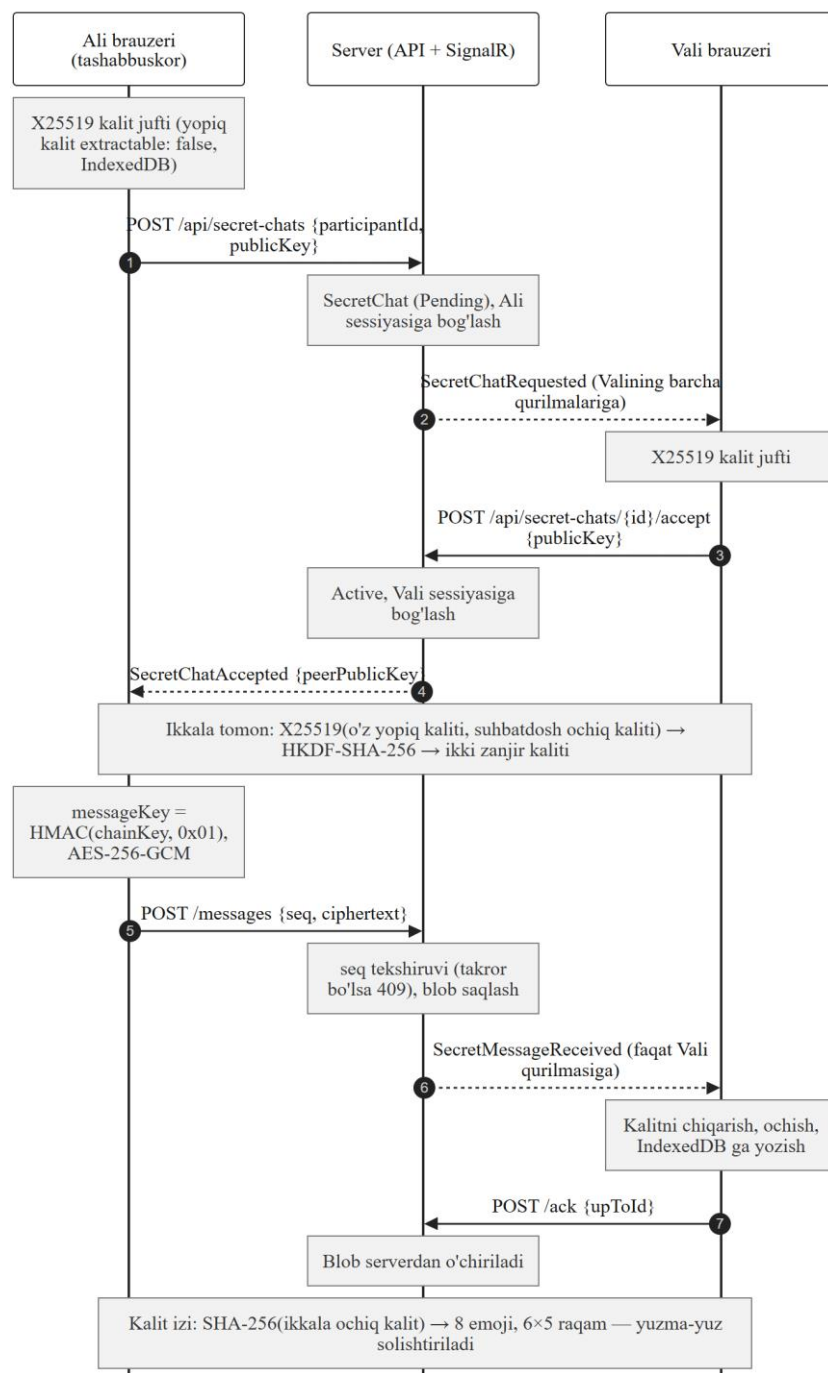
Oddiy chatlarda xabarlar serverga TLS orqali yetib keladi, lekin serverda ochiq saqlanadi — bu barcha qurilmalarda sinxronizatsiya, qidiruv va transliteratsiyani ta’minlaydi. Ammo ayrim yozishmalar server buzilganda yoki baza sizib chiqqanda ham o‘qilmasligi kerak. Buning uchun uchdan-uchgacha (E2E) shifrlangan **maxfiy chatlar** loyihalandi. Protokolda server “halol, lekin qiziquvchan” deb qaraladi: u xabarlarni yetkazadi, lekin faqat kim kimga, qachon va qancha hajmda yozganini biladi.

Signal va WhatsApp’dagi ko‘p qurilmali model har bir qurilma kalitlarini boshqarishni talab qiladi, shuning uchun Telegram’dagi kabi **qurilmaga bog‘langan model** tanlandi: maxfiy chat ikki aniq qurilma o‘rtasida ochiladi. Kriptografiyani noldan yozish xavfli bo‘lgani uchun protokol faqat brauzerning standart **Web Crypto API** primitivlaridan quriladi (2.5-jadval), barcha yopiq kalitlar esa “extractable: false” rejimida yaratiladi — skript ular bilan shifrlay oladi, lekin kalit baytlarini o‘qiy olmaydi.

Maxfiy chatlarda qo'llanilgan kriptografik primitivlar

Primitiv	Standart	Vazifasi
X25519	RFC 7748	Diffi-Xellman kalit kelishuvi: har bir qurilma har bir chat uchun yangi kalit jufti yaratadi, umumiy sir tarmoq orqali uzatilmaydi
HKDF-SHA-256	RFC 5869	Umumiy sirdan ikki yo'nalish uchun ikkita zanjir kalitini chiqarish
HMAC-SHA-256	RFC 2104	Zanjir kalitidan har bir xabar kaliti va keyingi zanjir kalitini hosil qilish (ratchet)
AES-256-GCM	NIST SP 800-38D	Xabar va fayllarni shifrlash va butunligini tekshirish
SHA-256	FIPS 180-4	Kalit izi (fingerprint): ikkala ochiq kalitdan emoji va raqamlar ko'rinishida

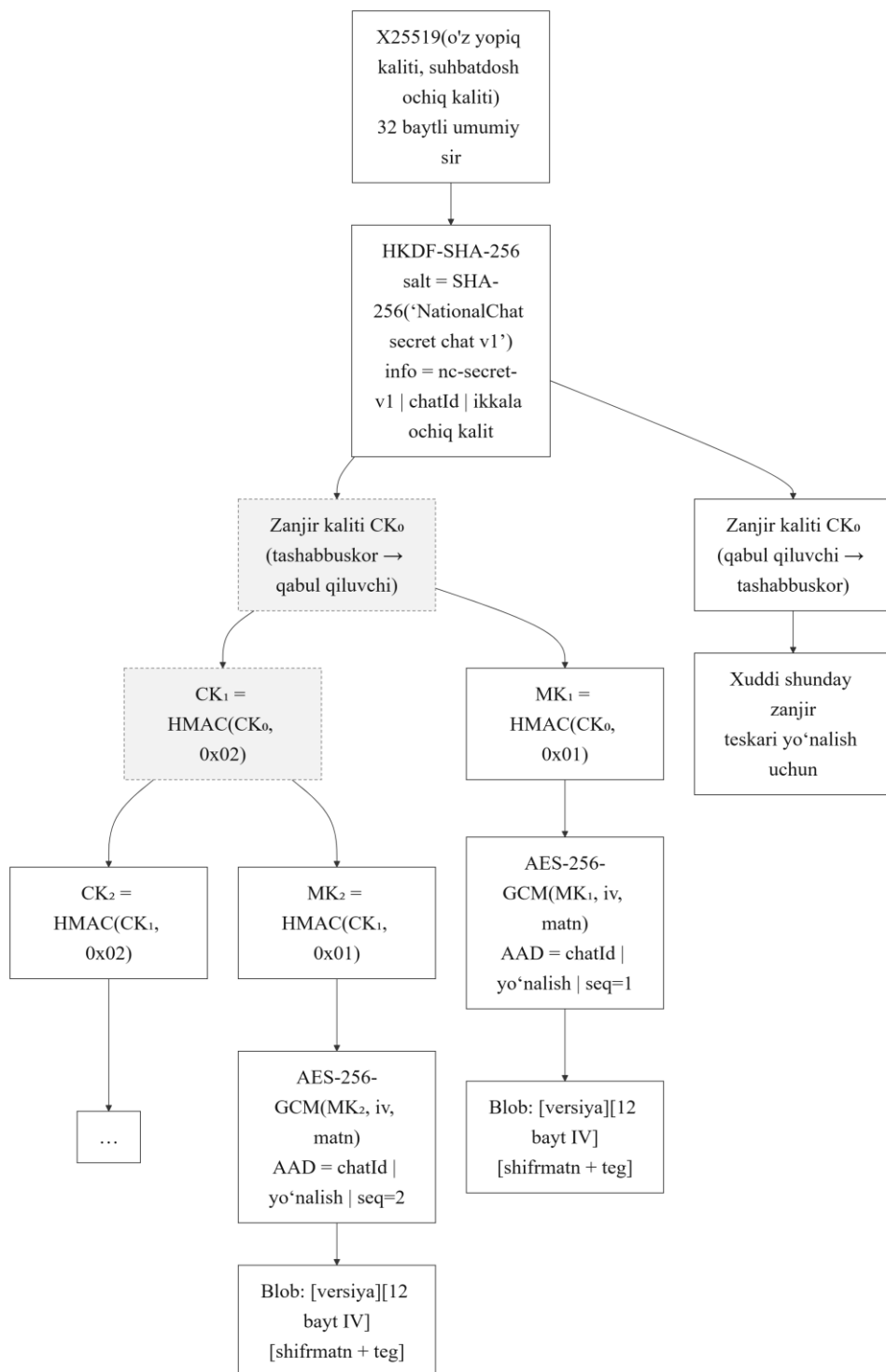
Kalit almashish. Tashabbuskorning brauzeri X25519 kalit juftini yaratib, serverga faqat ochiq kalitni yuboradi (2.9-rasm). Suhbatdosh taklifni qaysi qurilmada qabul qilsa, chat o'sha qurilmaga bog'lanadi va u ham o'z ochiq kalitini qaytaradi. Shundan so'ng ikkala tomon mustaqil ravishda bir xil umumiy sirni hisoblaydi. Serverdagi ikki ochiq kalitdan bu sirni topish diskret logarifm masalasini yechishni talab qiladi.



2.9-rasm. Maxfiy chatni ochish va shifrlangan xabar yetkazish ketma-ketlik diagrammasi

Xabar kalitlari zanjiri. Umumiy sir HKDF yordamida har bir yo‘nalish uchun alohida zanjir kalitiga kengaytiriladi (2.10-rasm). Har bir xabarda zanjir bir qadam suriladi: $MK = HMAC(CK, 0x01)$, $CK' = HMAC(CK, 0x02)$, eski kalit esa darhol unutiladi. Bu **oldinga maxfiylikni** ta’minlaydi: joriy kalit o‘g‘irlansa ham, avvalgi xabarlarni ochib bo‘lmaydi. Xabar AES-256-GCM bilan shifrlanadi, AAD

sifatida esa “chatId | yo‘nalish | tartib raqami” ishlatiladi, shuning uchun xabarni boshqa chatga ko‘chirish yoki o‘rnini almashtirish teg tekshiruvida aniqlanadi.



2.10-rasm. Maxfiy chatda kalitlarni chiqarish va xabar kalitlari zanjiri

Yetkazish. Server faqat saqlab-uzatish vazifasini bajaradi: shifrlangan blob qabul qiluvchi qurilmaga yuboriladi, u tasdiq (ack) qaytargach server blobni o‘chiradi, yetkazilmaganlari 7 kundan keyin tozalanadi. Har bir xabarning o‘sovchi

tartib raqami takroriy yuborish (replay) hujumidan himoyalaydi, tartibsiz kelgan xabarlar uchun esa 200 tagacha o'tkazib yuborilgan kalit vaqtincha saqlanadi. O'z-o'zini o'chirish taymeri va tarixni tozalash buyrug'i ham shifrlangan xabar sifatida yuboriladi.

Fayllar va kalit izi. Fayl brauzerda alohida tasodifiy AES kalit bilan shifrlanib yuklanadi, kalit va fayl nomi esa E2E xabar ichida yuboriladi. Kalit izi ikkala ochiq kalitning SHA-256 xeshidan 8 ta emoji va raqamlar ko'rinishida hisoblanadi: server o'rtada turib kalitlarni almashtirsa (MITM), ikki tomondagi belgilar farq qiladi.

Cheklovlar. Protokol Signal'dagi to'liq Double Ratchet emas: suhbat davomida yangi Diffi-Hellman almashinuvi bajarilmagani uchun buzilgandan keyin tiklanish xususiyati yo'q. Metama'lumotlar serverga ko'rinadi, maxfiy chat esa faqat bitta qurilmada ishlaydi. Veb-ilovada shifrlash kodini ham server yuborgani sababli bu xavf qat'iy CSP bilan kamaytirilgan, to'liq bartaraf etish esa alohida mobil ilova talab qiladi.

2-bob bo'yicha xulosa

Ikkinchi bobda "Milliy chat" tizimi loyihalandi. .NET 8, ASP.NET Core, SignalR, EF Core, PostgreSQL va Angular texnologiyalari tanlandi, server kodi Clean Architecture asosida to'rt qatlamga ajratildi. O'nta sxemadagi 36 ta jadvaldan iborat ma'lumotlar bazasi loyihalandi, katta ro'yxatlar uchun kursorli sahifalash tanlandi. UML diagrammalari yordamida kirish, real vaqtda xabar almashish va tashkilotga qo'shilish jarayonlari modellashtirildi. Xavfsizlik parolsiz autentifikatsiya, xeshlangan kod va tokenlar, rotatsiya qilinadigan refresh token, a'zolikka asoslangan avtorizatsiya va qat'iy CSP asosida qurildi. Maxfiy chatlar uchun Web Crypto API primitivlari asosida oldinga maxfiylikka ega E2E shifrlash protokoli ishlab chiqildi va uning cheklovlari aniqlandi.

III-BOB. “MILLIY CHAT” DASTURINI ISHLAB CHIQISH

Ushbu bobda loyihalangan tizimning dasturiy amalga oshirilishi yoritiladi. Server qismi 236 ta C# faylidan (migratsiyalarsiz 9 mingga yaqin qator), klient qismi esa 30 ta Angular komponenti va 13 mingdan ortiq qatordan iborat. Keltirilgan kod parchalari loyihaning haqiqiy manba kodidan olingan, qisqartirilgan joylar “...” bilan belgilangan.

3.1. Server qismini ishlab chiqish

Server yechimi to‘rtta loyihadan (Domain, Application, Infrastructure, API) va unit testlar loyihasidan iborat. Kirish nuqtasi — Program.cs fayli faqat so‘rovlarni qayta ishlash konveyerini sozlaydi: eng birinchi xatolarni ushlovchi middleware, so‘ng xavfsizlik sarlavhalari, autentifikatsiya, avtorizatsiya, controllerlar va SignalR habi. Servislarni ro‘yxatdan o‘tkazish esa mavzuli kengaytma metodlariga ajratilgan.

Server 11 ta controller orqali 73 ta HTTP endpointni taqdim etadi (3.1-jadval). Endpointlar RESTful tamoyil asosida nomlangan: resurslar ko‘plik shaklida, amal esa HTTP metodi orqali ifodalanadi, masalan “POST /api/chats/{chatId}/messages”.

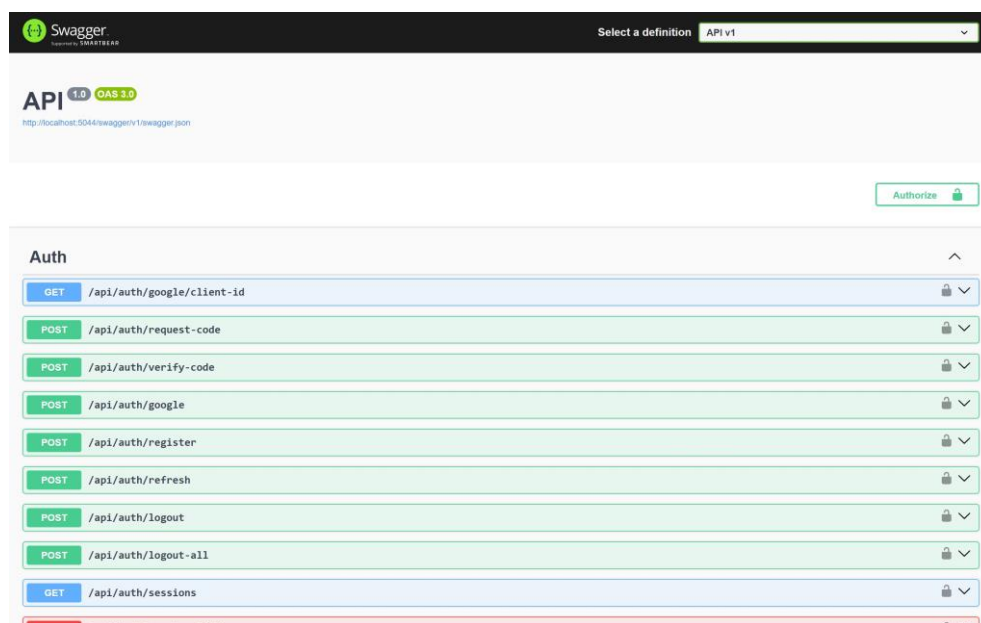
3.1-jadval

Server API controllerlari va endpointlari

Controller	Asosiy yo‘l	Soni	Vazifasi
AuthController	/api/auth	11	Kod so‘rash va tasdiqlash, Google, ro‘yxatdan o‘tish, tokenni yangilash, sessiyalar
ProfileController	/api/users/me	6	Profilni olish va tahrirlash, profil rasmi, yozuv sozlamasi
UserController, ContactController	/api/users, /api/contacts	4	Foydalanuvchilarni qidirish, kontaktlar
ChatController	/api/chats	3	Chatlar ro‘yxati, shaxsiy chat, chatni o‘chirish
GroupController	/api/groups	12	Guruh yaratish, tahrirlash, a‘zolar va rollar, taklif havolalari
MessageController	/api/chats/{id}/messages	7	Xabarlar tarixi, yuborish, tahrirlash, o‘chirish, qidiruv

Controller	Asosiy yo‘l	Soni	Vazifasi
MessageAttachmentController	/api/chats/..., /api/media/...	10	Rasm, video, fayl yuklash va berish, ulashilgan media
StoryController	/api/stories	8	Hikoyalar lentasi, yaratish, ko‘rishlar
OrganizationController	/api/organizations	1	Foydalanuvchining tashkiloti va roli
SecretChatController	/api/secret-chats	11	Maxfiy chat so‘rovi, qabul qilish, shifrlangan xabar va fayllar, ack
Jami		73	

API OpenAPI (Swagger) orqali avtomatik hujjatlashtiriladi: barcha endpointlarni JWT token bilan to‘g‘ridan-to‘g‘ri sinab ko‘rish mumkin (3.1-rasm). Barcha javoblar yagona formatda (succeeded, message, data) qaytadi, xato xabarlarini o‘zbek tilida yozilgan. Kiruvchi DTO‘lar FluentValidation validatorlari bilan tekshiriladi, kutilmagan istisnolar esa logga TraceId bilan yoziladi va mijozga ichki tafsilotlarsiz qaytariladi.



3.1-rasm. Swagger UI: server API hujjatlari

Kursorli sahifalash barcha repozitoriylar uchun umumiy IQueryable kengaytma metodi sifatida yozilgan (3.1-listing). Metod “Id < beforeId” shartini ifodalar daraxti orqali dinamik quradi va limitdan bittaga ko‘p yozuv o‘qiydi: qo‘shimcha yozuv borligi hasMore qiymatini beradi, alohida COUNT so‘roviga

hojat qolmaydi. Natija to‘g‘ridan-to‘g‘ri DTO‘ga proyeksiya qilinadi, shuning uchun bazadan faqat kerakli ustunlar o‘qiladi.

3.1-listing. Cursorli sahifalash (QueryableExtensions)

```
public static async Task<CursorPagedResponse<TResult>>
ToCursorPagedResponseAsync<TEntity, TResult>(
    this IQueryable<TEntity> query, CursorPaginationRequest pagination,
    Expression<Func<TEntity, int>> idSelector, Expression<Func<TEntity, TResult>>
resultSelector,
    Func<TResult, int> cursorSelector, CancellationToken cancellationToken = default)
{
    if (pagination.BeforeId is not null)
    {
        var beforeId = Expression.Lambda<Func<TEntity, bool>>(
            Expression.LessThan(idSelector.Body,
Expression.Constant(pagination.BeforeId.Value)),
            idSelector.Parameters);
        query = query.Where(beforeId);
    }

    var results = await query.OrderByDescending(idSelector)
        .Take(pagination.Limit + 1)
        .Select(resultSelector)
        .ToListAsync(cancellationToken);

    var hasMore = results.Count > pagination.Limit;
    IReadOnlyList<TResult> items = hasMore ? results.Take(pagination.Limit).ToArray()
: results;
    int? nextCursor = hasMore ? cursorSelector(items[^1]) : null;
    return new(items, nextCursor, hasMore);
}
```

3.2. Real vaqtda xabar almashishni amalga oshirish

Real vaqt aloqasi ChatHub SignalR habi orqali quriladi. Hab mijoz chaqira oladigan to‘rtta metodni taqdim etadi, server esa mijozlarga o‘n to‘rt turdagi hodisa yuboradi (3.2-jadval). Har bir ulanish ikki guruhga qo‘shiladi: foydalanuvchining barcha qurilmalari uchun “user:{id}” va aynan shu qurilma uchun “session:{sid}”; ikkinchisi maxfiy chat hodisalarini faqat bog‘langan qurilmaga yetkazadi.

3.2-jadval

SignalR habi metodlari va hodisalari

Nomi	Yo‘nalish	Vazifasi
JoinChat / LeaveChat	Mijoz → server	Ochilgan chatning “chat:{id}” guruhiga qo‘shilish yoki chiqish
SetTyping(chatId, isTyping)	Mijoz → server	“Yozmoqda” holatini chatdagi boshqalarga uzatish
MarkMessagesRead(chatId, ids)	Mijoz → server	Xabarlarni o‘qilgan deb belgilash (100 tagacha)

Nomi	Yoʻnalish	Vazifasi
MessageReceived / Updated / Deleted	Server → mijoz	Yangi, tahrirlangan yoki oʻchirilgan xabar
MessagesRead, TypingChanged	Server → mijoz	Oʻqilganlik (✓✓) va “yozmoqda” holati
ChatCleared / ChatDeleted, GroupUpdated	Server → mijoz	Chat tarixi tozalandi, chat oʻchirildi, guruh oʻzgardi
UserPresenceChanged, ProfilePhotoUpdated	Server → mijoz	Onlayn holat va profil rasmi oʻzgardi
SecretChatRequested / Accepted / Closed	Server → mijoz	Maxfiy chat taklifi, qabul qilinishi yoki yopilishi
SecretMessageReceived	Server → mijoz	Shifrlangan xabar (faqat bogʻlangan qurilmaga)

Hab metodlari ham REST kabi himoyalangan: har bir chaqiruvda foydalanuvchining chat aʼzosi ekani tekshiriladi (3.2-listing). Aks holda chat identifikatorini taxmin qilib, begona chatdagi hodisalarni tinglash mumkin boʻlardi.

3.2-listing. ChatHub: oʻqilganlik belgisini qayd etish va aʼzolikni tekshirish

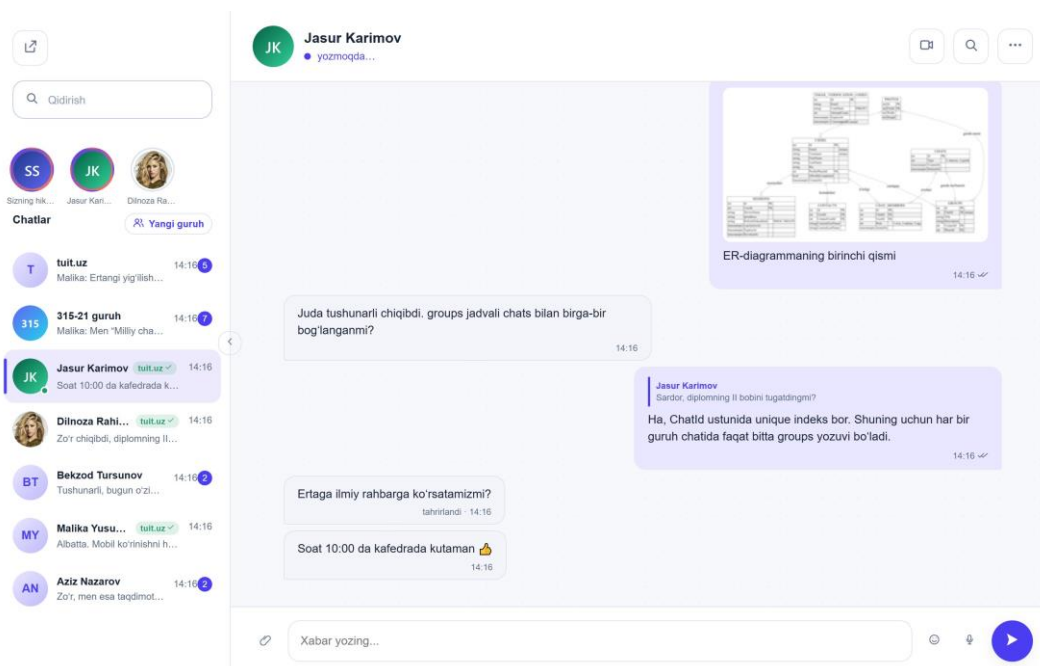
```
public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int> messageIds)
{
    await EnsureChatMembershipAsync(chatId);
    var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
    if (ids.Length == 0) return;

    var readerUserId = GetCurrentUser();
    await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
        timeProvider.UtcNow().UtcDateTime, Context.ConnectionAborted);
    await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids,
        await messageRepository.GetMemberUserIdsAsync(chatId,
Context.ConnectionAborted),
        Context.ConnectionAborted);
}

private async Task EnsureChatMembershipAsync(int chatId)
{
    if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUser(),
Context.ConnectionAborted))
        throw new HubException("Bu chatga kirish huquqi yo'q.");
}
```

Onlayn holat IPresenceTracker interfeysi orqali kuzatiladi: bitta serverda faol ulanishlar xotirada, bir necha serverda esa Redis’da saqlanadi. Oxirgi ulanish uzilganda server 8 soniya kutadi, shuning uchun sahifani yangilash yoki qisqa uzilishda suhbatdoshlarda “oflayn” holati koʻrinmaydi.

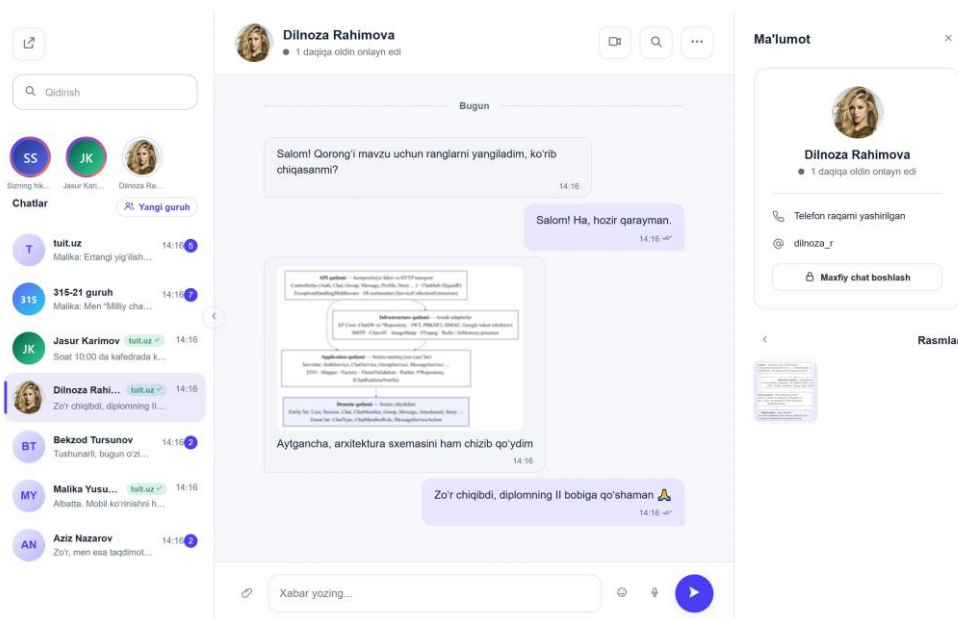
Klientdagi ChatRealtimeService ulanishni avtomatik qayta ulanish siyosati (0, 2, 5, 10 soniya) bilan yaratadi va qayta ulangach ochiq chatga yana qo‘shiladi. Kiruvchi hodisalar RxJS orqali ChatApiService’ga uzatiladi va u ilova holatini yangilaydi. 3.2-rasmda sarlavhada “yozmoqda...” holati, xabarlar yonida o‘qilganlik belgilari, ro‘yxatda esa tashkilot a‘zolari yonidagi “tuit.uz ✓” belgisi ko‘rinib turibdi.



3.2-rasm. Shaxsiy chat: real vaqtda “yozmoqda” holati va o‘qilganlik belgilari

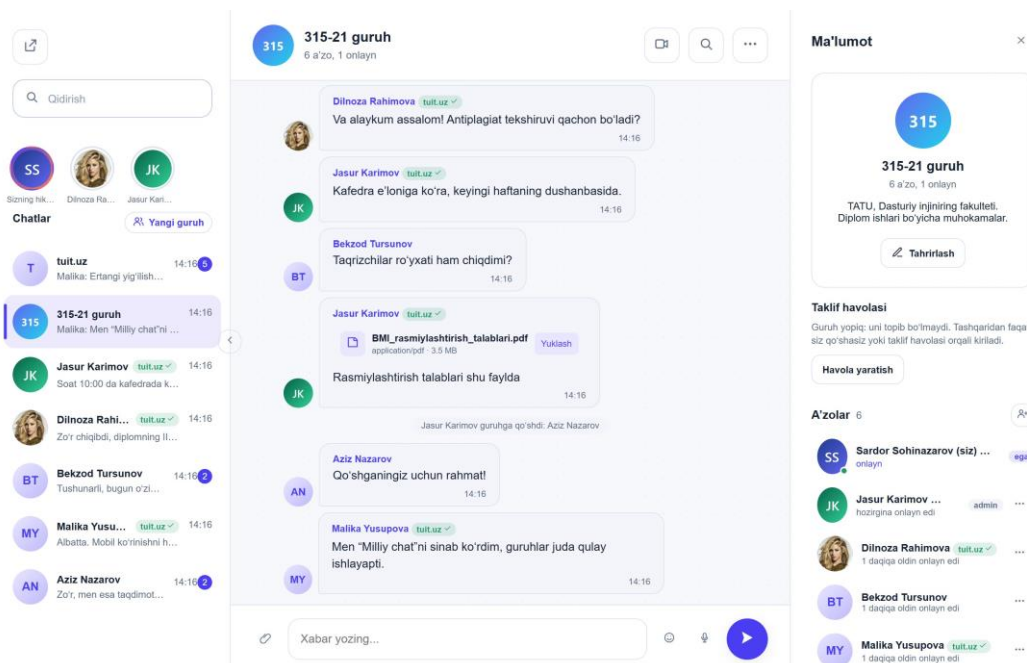
3.3. Media fayllar, guruhlar va hikoyalar

Rasm, video va fayllar bitta konveyerdan o‘tadi: hajm tekshiriladi (rasm 10 MB, fayl 50 MB, video 200 MB gacha), fayl ClamAV bilan skanerlanadi, rasm formati tarkibi bo‘yicha aniqlanib qayta kodlanadi va 256 pikseli kichik nusxasi yaratiladi, video uchun esa ffmpeg yordamida muqova olinadi. Fayl diskka GUID nom bilan yoziladi, bazaga metama’lumotlar va xabar qo‘shiladi, a‘zolarga esa MessageReceived hodisasi yuboriladi. Videolar HTTP Range so‘rovlari orqali oqim ko‘rinishida beriladi. Chatdagi barcha media ma’lumot panelida turlari bo‘yicha guruhlanadi (3.3-rasm).



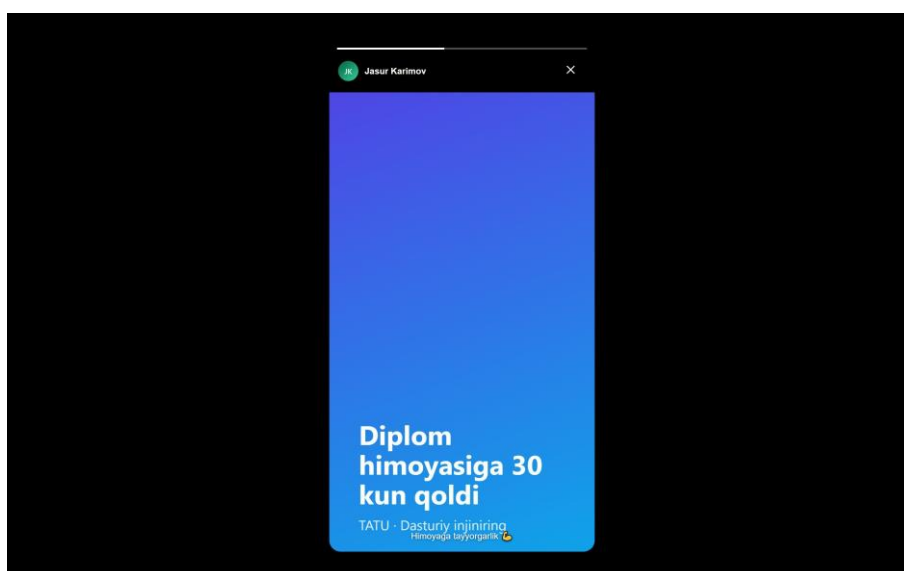
3.3-rasm. Chatdagi ulashilgan media paneli

Guruh chatlari GroupService servisinde amalga oshirilgan: har bir amaldan oldin foydalanuvchi roli aniqlanadi va 2.3-jadvaldagi ruxsatlar qo'llanadi, o'zgarishlar esa xizmat xabarlar bilan birga saqlanadi. Guruh egasi chiqib ketsa, egalik eng oldin tayinlangan administratorga, u bo'lmasa eng oldin qo'shilgan a'zoga o'tadi; guruhda hech kim qolmasa, u yumshoq o'chiriladi. Taklif havolasi uchun 16 baytli kriptografik tasodifiy token yaratiladi, havola yangilanganda eski token darhol ishlamay qoladi. Guruh chati va ma'lumot paneli 3.4-rasmda ko'rsatilgan.



3.4-rasm. Guruh chati va guruh haqidagi ma'lumot paneli

Hikoyalar StoryService servisida amalga oshirilgan: rasm yoki video (50 MB gacha) va izohdan iborat hikoya 24 soat ko'rinadi. Lentada foydalanuvchi va u bilan chati bor foydalanuvchilarning hikoyalari chiqadi, ko'rilmaganlari rangli halqa bilan ajratiladi, muallif esa kim ko'rganini biladi (3.5-rasm).



3.5-rasm. Hikoyani ko'rish oynasi

3.4. Lotin–kirill transliteratsiyasi va yozuvdan qat'i nazar qidiruv

Xabar bazada doimo **asl holda** saqlanadi, o'g'irish esa faqat ko'rsatish paytida klientda bajariladi. Shu sababli bitta xabarni bir foydalanuvchi lotinda, boshqasi

kirillda ko‘radi. Foydalanuvchi sozlamasi (ScriptPreference) serverda saqlanadi va barcha qurilmalarida amal qiladi, yozuv almashtirilganda esa xabarlar sahifani yangilamasdan qayta chiziladi.

Algoritm o‘zbek lotin alifbosi va imlo qoidalariga asoslanadi va Domain qatlamidagi UzbekTransliterator sinfida amalga oshirilgan; klientda uning TypeScript’dagi aynan nusxasi ishlaydi. Ikkalasi bitta umumiy test to‘plami (80 ta kirill–lotin juftligi) bo‘yicha tekshiriladi. Kirilldan lotinga o‘girish deyarli bir ma’noli, teskari yo‘nalishda esa kontekstga bog‘liq qoidalar talab etiladi (3.3-jadval).

3.3-jadval

Kontekstga bog‘liq transliteratsiya qoidalari

Holat	Qoida	Misol
Kirill “e”	So‘z boshida, unlidan yoki ъ, ь dan keyin “ye”, qolgan hollarda “e”	ер → yer; келди → keldi
Kirill “и”	Unlidan keyin “ts”, qolgan hollarda “s”	лицей → litsey; цирк → sirk
ŷ, ƒ va tutuq	ŷ → o‘, ƒ → g‘, ъ → ‘; “cx” tutuq bilan yoziladi	Ўзбек → O‘zbek; Исҳоқ → Is‘hoq
Lotin digraflari	sh → ш, ch → ч, o‘ → ŷ, g‘ → ƒ, yo → ё, yu → ю, ya → я	shahar → шахар; yo‘l → йўл
Lotin “e”	So‘z boshida yoki unlidan keyin “э”, qolgan hollarda “e”	eshik → эшик; кеча → кеча
Registr	Bosh harf saqlanadi; katta harfli so‘zda digraf ham katta	ШАҲАР → SHAHAR
Tutuq variantlari	‘, ‘, ‘, ‘, ‘, ‘ belgilari bir xil tushuniladi	o‘zbek = o‘zbek = o‘zbek
Himoyalangan qismlar	URL, e-pochta, @username, #teg va ‘kod’ o‘girilmaydi	https://tuit.uz o‘zgarmaydi

Kirilldan lotinga o‘girishning asosiy qismi 3.3-listingda keltirilgan: har bir harf uchun avval kontekstga bog‘liq qoidalar, keyin oddiy moslik lug‘ati tekshiriladi.

3.3-listing. UzbekTransliterator: kirill matnini lotinga o‘girish (C#)

```
private static string ToLatinSegment(string text)
{
    var builder = new StringBuilder(text.Length + 8);
    for (var i = 0; i < text.Length; i++)
    {
        var ch = text[i];
        var lower = char.ToLowerInvariant(ch);
        var previous = i > 0 ? char.ToLowerInvariant(text[i - 1]) : '\0';
```

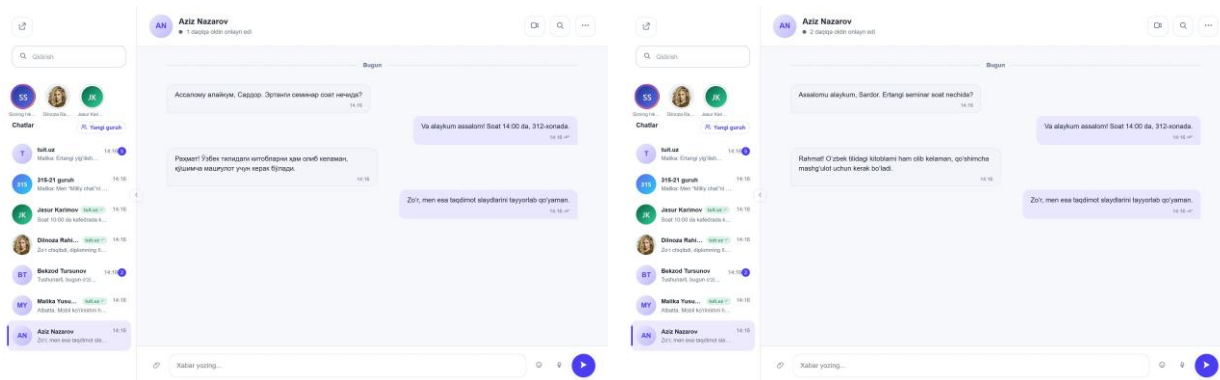
```

string latin;
if (lower == 'e')
{
    var soundsYe = !IsWordChar(text, i - 1) ||
CyrillicVowels.Contains(previous) || previous is 'Ѣ' or 'Ѧ';
    latin = soundsYe ? "ye" : "e";
}
else if (lower == 'ц')
    latin = CyrillicVowels.Contains(previous) ? "ts" : "s";
else if (lower == 'х' && previous == 'с')
    latin = TutuqMark + "h"; // "cx" is written with a tutuq so that it is
not read as "ш"
else if (!CyrillicToLatinMap.TryGetValue(lower, out latin!))
{
    builder.Append(ch);
    continue;
}

builder.Append(ch == lower ? latin : ApplyUpperCase(latin,
IsUpperCaseWord(text, i)));
}
return builder.ToString();
}

```

Natija 3.6-rasmda ko‘rsatilgan: “Asl holida” sozlamasida suhbatdosh kirillda, foydalanuvchi lotinda yozgan xabarlar aralash chiqadi, “Lotin” sozlamasida esa butun suhbat lotin yozuvida ko‘rinadi.



a) “Asl holida”

b) “Lotin” sozlamasi

3.6-rasm. Bitta suhbatning turli yozuv sozlamalarida ko‘rinishi

Qidiruv ham yozuvga bog‘liq emas: “rahmat” so‘rovi “Paxmat”ni ham topadi. Buning uchun xabar yaratilganda va tahrirlanganda uning **qidiruv kaliti** — lotinga o‘girilgan, kichik harfli, tutuq belgilari birxillashtirilgan nusxasi SearchText ustuniga yoziladi. Qidiruv so‘rovi ham xuddi shu funksiyadan o‘tkazilib, ILIKE bilan qidiriladi. Ustun qo‘shilishidan oldingi xabarlar uchun kalitlar SearchTextBackfill fon ishi tomonidan 500 tadan to‘ldiriladi. Bazadagi natija 3.7-rasmda ko‘rsatilgan.

```
PS> psql -d NationalChat -c "SELECT Id, TextContent, SearchText FROM messaging.messages WHERE ChatId = 2;"
 Id | TextContent | SearchText
-----+-----+-----
 5 | Ассалому алайкум, Сардор. Эртанги семи | assalomu alaykum, sardor. ertangi semi
 6 | Va alaykum assalom! Soat 14:00 da, 312 | va alaykum assalom! soat 14:00 da, 312
 7 | Раҳмат! Ўзбек тилидаги китобларни ҳам | rahmat! o'zbek tilidagi kitoblarni ham
 8 | Zo'r, men esa taqdimot slaydlarini tay | zo'r, men esa taqdimot slaydlarini tay
(4 rows)
```

3.7-rasm. messages jadvali: asl matn (TextContent) va qidiruv kaliti (SearchText)

3.5. Tashkilot rejimini amalga oshirish

OrganizationEmailMatcher sinfi e-pochtadan domenni ajratadi, normallashtiradi va umumiy pochta xizmatlari ro'yxati bilan solishtiradi; u tashqi bog'liqliksiz va 22 ta unit test bilan tekshirilgan. OrganizationMembershipService.EnsureMembershipAsync metodi foydalanuvchi tasdiqlangan pochta bilan kirgan har safar chaqiriladi (3.4-listing). Bir vaqtda ikki foydalanuvchi yangi domen bilan kirsam, unique indeks tufayli faqat bittasi tashkilotni yaratib admin bo'ladi, ikkinchisi a'zo sifatida qo'shiladi.

3.4-listing. OrganizationMembershipService: foydalanuvchini tashkilotga va domen guruhiga qo'shish

```
public async Task EnsureMembershipAsync(User user, CancellationToken cancellationToken
= default)
{
    // An e-mail never changes, so an existing membership is final.
    if (await repository.GetMembershipAsync(user.Id, cancellationToken) is not null)
        return;

    var domain = OrganizationEmailMatcher.OrganizationDomainOf(user.Email);
    if (domain is null) return;

    var now = timeProvider.GetUtcNow().UtcDateTime;
    var organization = await repository.FindByDomainAsync(domain, cancellationToken);
    var isFounder = false;
    if (organization is null)
    {
        var created = OrganizationFactory.Create(domain, now);
        isFounder = await repository.TryAddOrganizationAsync(created,
cancellationToken);
        organization = isFounder ? created : await
repository.FindByDomainAsync(domain, cancellationToken);
        if (organization is null) return;
    }

    // The first person from a domain becomes its admin.
    var role = isFounder ? OrganizationRole.Admin : OrganizationRole.Member;
    var member = OrganizationFactory.CreateMember(organization.Id, user.Id, role,
now);
    if (!await repository.TryAddMemberAsync(member, cancellationToken)) return;

    var groupChatIds = await
repository.GetOrganizationGroupChatIdsAsync(organization.Id, cancellationToken);
```



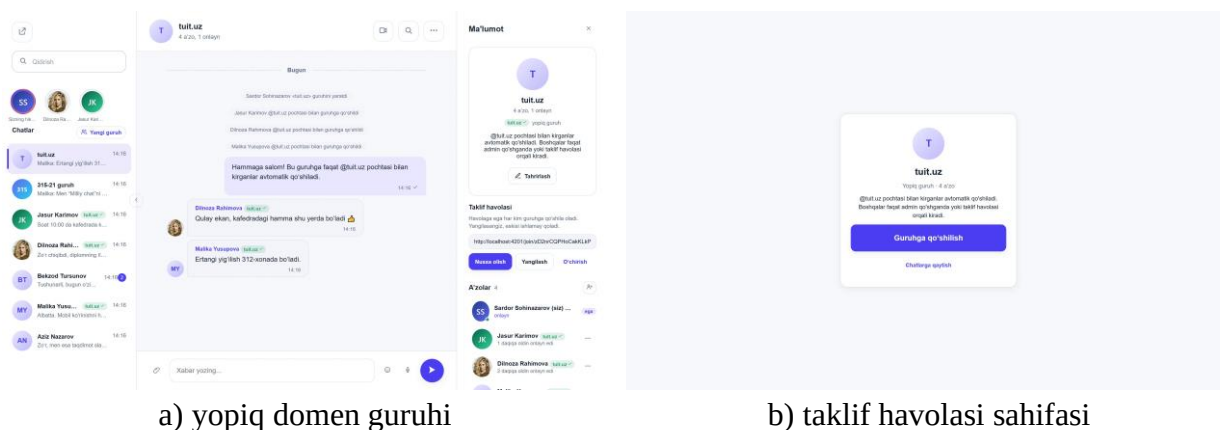
```

if (groupChatIds.Count == 0)
{
    await groupService.CreateOrganizationGroupAsync(user.Id, cancellationTokens);
    return;
}

foreach (var chatId in groupChatIds)
    await groupService.JoinViaOrganizationAsync(chatId, user.Id,
cancellationTokens);
}

```

Tashkilot a'zoliqi foydalanuvchi haqidagi barcha DTO'larga qo'shilgan va klientda ismlar yonida "tuit.uz ✓" ko'rinishida chiqadi. Domen guruhida har bir qo'shilish xizmat xabari bilan qayd etiladi, tashqaridagi foydalanuvchi esa guruhga taklif havolasi orqali kiradi (3.8-rasm).



3.8-rasm. Tashkilot rejimi

3.6. Maxfiy chatlarni amalga oshirish

Server tomonida maxfiy chatlar SecretChatService va SecretChatController orqali amalga oshirilgan. Barcha endpointlar joriy qurilma nomidan ishlaydi: qurilma access tokendagi sessiya identifikatori bo'yicha aniqlanadi va har bir so'rovda sessiyaning faolligi bazadan qayta tekshiriladi. Chat bog'lanmagan qurilmadan kelgan so'rov "topilmadi" javobini oladi.

Shifrlangan xabarni qabul qilish 3.5-listingda ko'rsatilgan. Server blobni ochmaydi, faqat formati va hajmini (32 KB gacha) tekshiradi. Tartib raqami oxirgisidan katta bo'lishi shart, bir vaqtda kelgan takroriy so'rovlar uchun esa unique indeks ikkinchi himoya chizig'ini beradi. Qabul qiluvchi xabarni olgach, ack so'rovi yuboradi va server uni o'chiradi; SecretChatCleanup fon ishi esa har soatda eskirgan so'rov, xabar va fayllarni tozalaydi.

3.5-listing. SecretChatService.SendAsync: shifrlangan xabarni qabul qilish (qisqartirilgan)

```
var chat = await repository.GetAsync(secretChatId, cancellationToken);
var side = chat is null ? SecretChatAccess.Side.None : SecretChatAccess.SideOf(chat,
userId, sessionId);
if (chat is null || side == SecretChatAccess.Side.None) return new(null,
NotFoundError);
if (chat.Status != SecretChatStatus.Active) return new(null, "Maxfiy chat faol
emas.");

var lastSeq = side == SecretChatAccess.Side.Initiator ? chat.InitiatorLastSeq :
chat.ParticipantLastSeq;
if (request.Seq <= lastSeq) return new(null, DuplicateSeqError, Duplicate: true);

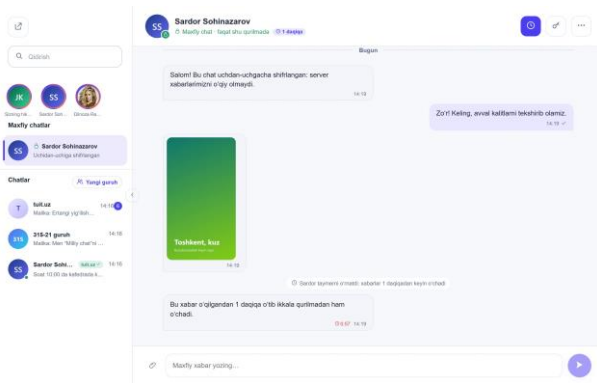
if (side == SecretChatAccess.Side.Initiator) chat.InitiatorLastSeq = request.Seq;
else chat.ParticipantLastSeq = request.Seq;

var ciphertext = SecretChatBase64.TryDecode(request.Ciphertext)!;
var message = SecretChatFactory.CreateMessage(chat.Id, sessionId, request.Seq,
ciphertext, Now());
if (!await repository.TryAddMessageAsync(message, cancellationToken))
    return new(null, DuplicateSeqError, Duplicate: true);

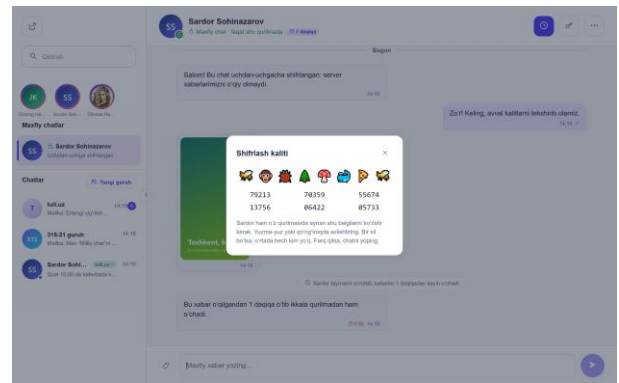
var recipientSessionId = side == SecretChatAccess.Side.Initiator
    ? chat.ParticipantSessionId!.Value : chat.InitiatorSessionId;
await realtimeNotifier.MessageReceivedAsync(SecretChatMapper.ToDto(message),
recipientSessionId, cancellationToken);
```

Klientdagi kriptografik kod secret-crypto.ts modulida joylashgan va Angular'ga bog'liq emas; uning asosiy qismi ilovalarda keltirilgan. Yangi zanjir holati xabar yuborilishidan oldin IndexedDB'ga xabar bilan bitta tranzaksiyada yoziladi, shuning uchun sahifa to'satdan yopilsa ham bitta xabar kaliti ikki marta ishlatilmaydi. Xabarlar qurilmadagi navbat (outbox) orqali qat'iy tartibda yuboriladi. Maxfiy chat oynasida qulf belgisi, taymer va kalit izini tekshirish tugmalari bor (3.9-rasm).

Server xabar mazmunini bilmasligi amalda ham tekshirildi: suhbatdosh oflayn bo'lgan paytda yuborilgan xabar bazada faqat 183 baytli ma'nosiz blob ko'rinishida turadi, yetkazilgan xabarlar esa serverdan butunlay o'chirilgan (3.10-rasm).



a) shifrlangan rasm va taymer



b) kalit izini tekshirish

3.9-rasm. Maxfiy chat

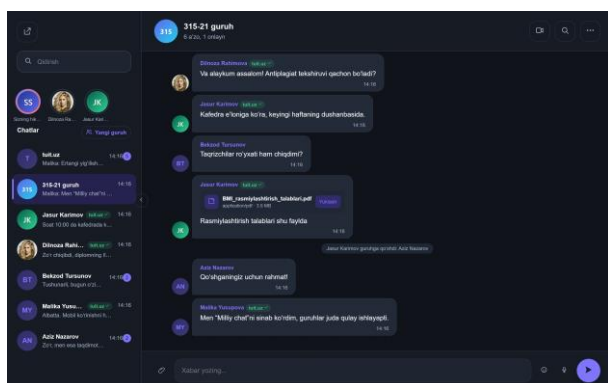
```
PS> psql -d NationalChat -c "SELECT Id, SecretChatId, Seq, length(Ciphertext), encode(Ciphertext, base64) FROM messaging.secret_messages;"
 Id | chat | Seq | bytes | ciphertext | created
-----+-----+-----+-----+-----+-----
  6 |   1 |   5 |  183 | AdBp3POJtrxqSydreRLgRuQr0qInOMLWPeLcchGuJmc... | 02:19:24
(1 row)
```

3.10-rasm. secret_messages jadvali: serverda faqat yetkazilmagan shifrlangan blob saqlanadi

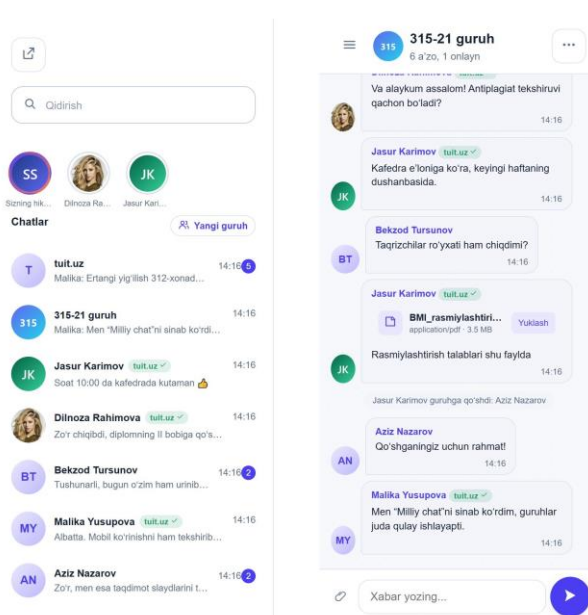
3.7. Klient qismini ishlab chiqish

Klient ilovasi Angular 22 ning mustaqil (standalone) komponentlaridan tashkil topgan va uchta marshrutga ega: /auth, /chat va /join/:token. /chat sahifasi authGuard, /auth esa guestGuard bilan himoyalangan, barcha sahifalar kerak bo‘lganda yuklanadi (lazy loading). Ilova holati Angular signals orqali boshqariladi: chatlar ro‘yxati, xabarlar va “yozmoqda” holati ChatApiService’da signal ko‘rinishida saqlanadi, komponentlar esa computed() yordamida “6 a‘zo, 2 onlayn” kabi hosila qiymatlar oladi. REST javoblari ham, SignalR hodisalari ham holatga bir xil yo‘l bilan qo‘llanadi.

authInterceptor har bir so‘rovga access tokenni qo‘shadi, 401 javobida esa tokenni yangilab so‘rovni takrorlaydi; bir vaqtda bir nechta so‘rov muvaffaqiyatsiz bo‘lsa ham bitta refresh so‘rovi ishlatiladi. Interfeys loyihaning dizayn tizimiga asoslangan: ranglar CSS o‘zgaruvchilari orqali berilgani uchun qorong‘i mavzu ularning muqobil qiymatlari bilan amalga oshiriladi. Interfeys moslashuvchan: katta ekranda uch ustunli joylashuv, telefonda esa bir vaqtda bitta ekran ko‘rsatiladi (3.11-rasm).



a) qorong'i mavzu



b) mobil qurilmada

3.11-rasm. Qorong'i mavzu va mobil ko'rinish

3.8. Joylashtirish va testlash

Server ilovasi ko'p bosqichli Dockerfile yordamida yig'iladi: birinchi bosqichda .NET SDK obrazida loyiha nashr qilinadi, ikkinchisida faqat natija yengil ASP.NET runtime obraziga ko'chiriladi, shuning uchun yakuniy obrazda kompilyator va manba kodi bo'lmaydi. Konteyner Render platformasida ishga tushiriladi va GitHub'dagi o'zgarishlardan so'ng avtomatik qayta yig'iladi. Maxfiy sozlamalar (bazaga ulanish, JWT va HMAC kalitlari, SMTP) kodda emas, muhit o'zgaruvchilarida saqlanadi.

Biznes-mantiq xUnit freymvorkida NationalChat.Tests loyihasi bilan tekshirildi. Tashqi bog'liqliklar NSubstitute yordamida soxtalashtirildi, vaqtga bog'liq mantiq esa qat'iy TimeProvider bilan sinaldi. Serverda 307 ta, klientda Vitest freymvorkida 229 ta test yozildi; klient testlari transliteratsiyaning server bilan bir xil ishlashini, shifrlash, kalit zanjiri va soxta xabarlarni rad etishni tekshiradi. Natija 3.12-rasmda, server testlari tarkibi 3.4-jadvalda keltirilgan.

```

PS> dotnet test tests\NationalChat.Tests --logger "console;verbosity=minimal"
Test run for C:\Users\Sardor\Desktop\diplom ish\NationalChat\tests\NationalChat.Tests\bin\Debug\net8.0\NationalChat.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.
Passed! - Failed: 0, Passed: 307, Skipped: 0, Total: 307, Duration: 1 s - NationalChat.Tests.dll (net8.0)
PS> cd ../NationalChatClient; npx ng test --watch=false
RUN v4.1.10 C:/Users/Sardor/Desktop/diplom ish/NationalChatClient
Test Files 12 passed (12)
Tests 229 passed (229)
Duration 29.43s (transform 3.35s, setup 7.72s, import 6.08s, tests 6.03s, environment 256.78s)

```

3.12-rasm. Server (dotnet test) va klient (ng test) testlarini bajarish natijasi

3.4-jadval

Server unit testlari tarkibi va natijalari

Test sinfi	Soni	Tekshiriladigan holatlar	Natija
UzbekTransliteratorTests, MessageSearchTextTests	167	Umumiy to‘plamdagi 80 ta juftlik ikkala yo‘nalishda, kontekst qoidalari, registr, himoyalangan qismlar, qidiruv kaliti	O‘tdi
Organization... (3 ta sinf)	44	Domenni ajratish, o‘xshash va umumiy domenlar, birinchi foydalanuvchi — admin, taklif havolalari	O‘tdi
SecretChatServiceTests, AuthServiceSecretChatTests	39	Qurilmaga bog‘lash, takroriy seq‘ni rad etish, boshqa qurilmadan yuborish, ack, logout‘da yopilish	O‘tdi
GroupServiceTests, ChatPermissionTests	19	Rollar va ruxsatlar, egalikni o‘tkazish, a‘zolar limiti	O‘tdi
Xeshlash, ClientOrigin, sahifalash, validatorlar	38	PBKDF2 va HMAC, ruxsat etilgan manzillar, kursorli sahifalash, validatsiya chegaralari	O‘tdi
Jami	307		307 / 307

Bundan tashqari, guruh chatlari uchun API darajasida 43 ta tekshiruvdan iborat avtomatik ssenariy yozildi va barchasi muvaffaqiyatli o‘tdi. Maxfiy chatlar ikki alohida brauzerda haqiqiy server bilan sinaldi, diplomdagi barcha skrinshotlar ham haqiqiy tizimdan avtomatik olindi. Qo‘lda o‘tkazilgan asosiy sinovlar 3.5-jadvalda keltirilgan.

3.5-jadval

Qo‘lda o‘tkazilgan asosiy sinov ssenariylari

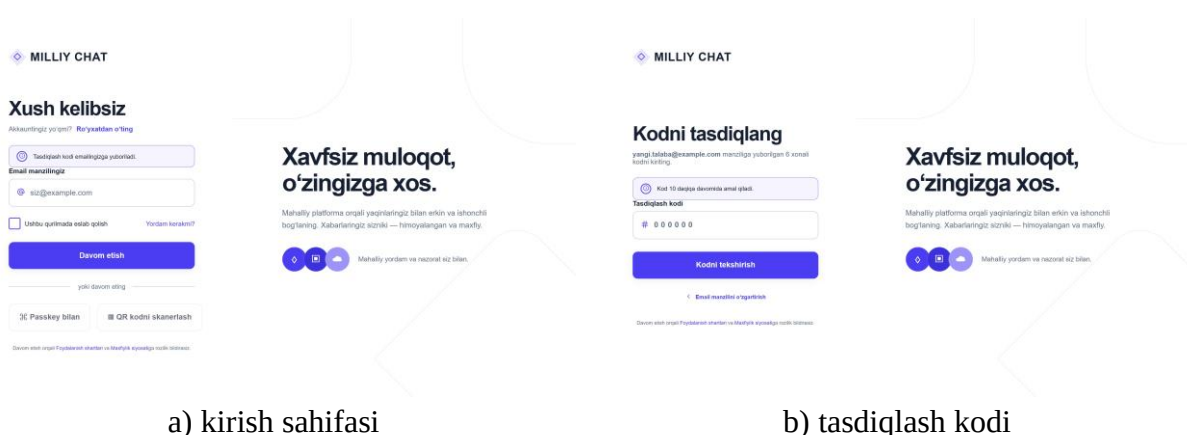
№	Ssenariy	Kutilgan natija	Natija
1	Yangi e-pochta bilan kirish va ro‘yxatdan o‘tish	Kod keladi, profil yaratiladi, chat sahifasi ochiladi	Mos
2	Noto‘g‘ri kodni 5 marta kiritish	Kod bekor qilinadi, keyin to‘g‘ri kod ham qabul qilinmaydi	Mos
3	Ikki brauzerda xabar almashish	Xabar sahifani yangilamasdan keladi, ✓✓ belgisi paydo bo‘ladi	Mos

№	Ssenariy	Kutilgan natija	Natija
4	Guruh yaratish va a'zo qo'shish	Guruh barcha a'zolarida darhol paydo bo'ladi, xizmat xabari chiqadi	Mos
5	Oddiy a'zo guruhni boshqarishga urinadi	Boshqaruv tugmalari ko'rinmaydi, API rad etadi	Mos
6	Boshqa qurilma sessiyasini yakunlash	Qurilma chiqadi, tokeni yangilanmaydi, undagi maxfiy chatlar yopiladi	Mos
7	Telefon o'lchamida (375 px) foydalanish	Gorizontal aylantirish yo'q, barcha amallar mavjud	Mos
8	Kirillcha xabarni "Lotin" sozlamasida ko'rish va qidirish	Xabar lotinda ko'rinadi, lotincha so'rov uni topadi	Mos
9	@tuit.uz va @gmail.com bilan ro'yxatdan o'tish	Birinchisi belgi oladi va domen guruhiga qo'shiladi, ikkinchisi yo'q	Mos
10	Ikki brauzerda maxfiy chat, rasm va taymer	Kalit izlari bir xil, xabar ochiladi, muddati o'tgani ikkala tomonda o'chadi	Mos

3.9. Foydalanish yo'riqnomasi

Tizimga kirish.

Kirish sahifasida e-pochta manzili kiritiladi yoki Google hisobi tanlanadi, so'ng pochtaga kelgan 6 xonali kod kiritiladi (3.13-rasm). Manzil birinchi marta ishlatilayotgan bo'lsa, ism, familiya va username kiritilib profil yaratiladi. Tashkilot pochta bilan kirilganda foydalanuvchi tashkilotning domen guruhiga avtomatik qo'shiladi.



a) kirish sahifasi

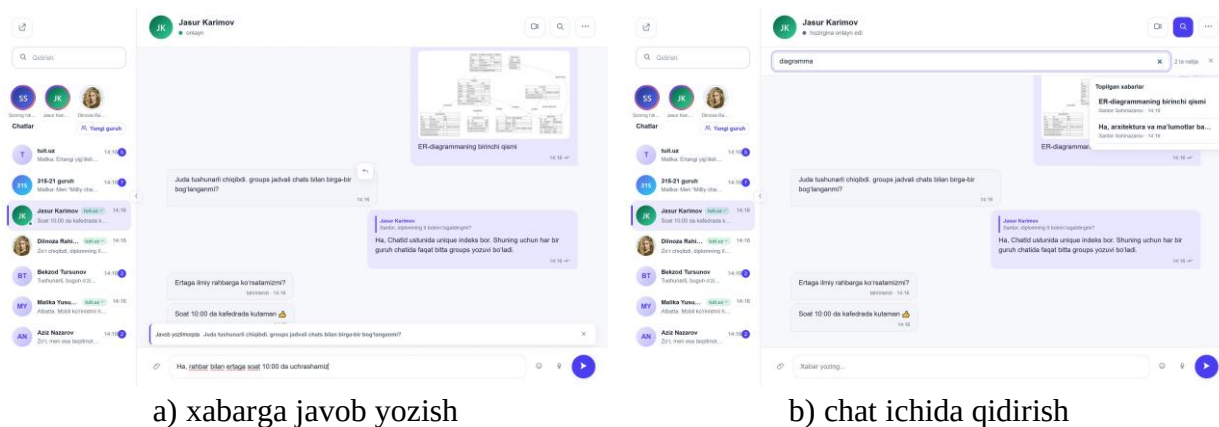
b) tasdiqlash kod

3.13-rasm. Tizimga kirish

Xabar almashish.

Chap paneldagi qidiruv maydoniga ism yoki username yozilib, suhbatdosh tanlanadi. Xabar pastdagi maydonga yoziladi va Enter bilan yuboriladi, qisqich

tugmasi orqali fayl biriktiriladi. Xabar ustiga olib borilganda javob berish, tahrirlash va o‘chirish amallari chiqadi. Sarlavhadagi qidiruv tugmasi chat ichida qidiradi, topilgan xabar bosilganda chat o‘sha joyga aylantiriladi (3.14-rasm).



3.14-rasm. Xabar almashish

Profil va yozuv sozlamasi.

Profil oynasida ism, username va profil rasmi tahrirlanadi, mavzu almashtiriladi, “Qurilmalar” bo‘limida esa begona qurilmani hisobdan chiqarish mumkin. “Yozuv” bo‘limida xabarlar qaysi yozuvda ko‘rsatilishi tanlanadi: “Asl holida”, “Lotin” yoki “Кирилл” (3.15-rasm).



3.15-rasm. Profil oynasi

Guruh va maxfiy chat.

Guruh yaratish uchun “Yangi guruh” tugmasi bosiladi, a’zolar tanlanadi, so‘ng nom, tavsif va rasm kiritiladi. Guruh sarlavhasi bosilganda ma’lumot paneli ochiladi (3.4-rasm): ega va administratorlar bu yerda a’zolari boshqaradi va taklif havolasini yaratadi.

Maxfiy chat shaxsiy chat menyusidagi “Maxfiy chat boshlash” bandi orqali ochiladi. Suhbatdosh taklifni qabul qilgach, kalit izini yuzma-yuz solishtirish, taymer oʻrnatish va shifrlangan fayl yuborish mumkin. Maxfiy chat faqat u ochilgan qurilmada ishlaydi.

3-bob boʻyicha xulosa

Uchinchi bobda “Milliy chat” tizimining dasturiy amalga oshirilishi koʻrib chiqildi. Server qismida 73 ta endpointli REST API, kursorli sahifalash, toʻrt metodli va oʻn toʻrt hodisali SignalR habi, onlayn holatni kuzatish, media fayllarni xavfsiz qayta ishlash, guruhlar va hikoyalar, klient qismida esa signals asosidagi holat boshqaruvi, tokenni avtomatik yangilash, qorongʻi mavzu va moslashuvchan interfeys yaratildi. Tizimning oʻziga xos uchta imkoniyati — lotin–kirill transliteratsiyasi va yozuvdan qatʼi nazar qidiruv, e-pochta domeni orqali tashkilot rejimi hamda E2E shifrlangan maxfiy chatlar amalga oshirildi. Tizim Docker konteynerida bulutga joylashtirildi va 307 ta server, 229 ta klient testi, 43 ta integratsion tekshiruv hamda qoʻlda oʻtkazilgan sinovlar bilan tasdiqlandi.

IV-BOB. HAYOT FAOLIYATI XAVFSIZLIGI

Hayot faoliyati xavfsizligi inson faoliyatidagi xavfli va zararli omillarni aniqlash, ularning ta'sirini kamaytirish hamda favqulodda vaziyatlarda odamlar hayoti va sog'lig'ini saqlashga qaratilgan chora-tadbirlar tizimidir. "Milliy chat" kabi dasturiy mahsulotni ishlab chiqish jamoasi ish vaqtining asosiy qismini kompyuter qarshisida o'tkazgani uchun bobda ikki masala ko'rib chiqiladi: dasturchining ergonomik ish joyini tashkil etish hamda IT-ofisda evakuatsiyani rejalashtirish va avariya yoki yong'inda xodimlarning harakatlari.

4.1. Dasturchining ergonomik ish joyini tashkil etish

Ergonomika (yunoncha *ergon* — ish va *nomos* — qonun) inson, mehnat vositalari va ish muhitining o'zaro moslashuvini o'rganadi: ergonomik ish joyida jihozlar insonning antropometrik va fiziologik xususiyatlariga moslashtiriladi. Dasturchi mehnati aqliy mehnat bo'lib, unda diqqatni uzoq jamlash va ko'rish organlariga yuqori yuklama ustunlik qiladi, jismoniy yuklama esa kam va asosan statik bo'ladi.

Dasturchining ish joyidagi asosiy xavfli va zararli omillar quyidagilar:

- **ko'rish analizatoriga yuklama** — ekranga uzoq tikilish ko'zning quruqlashishi va charchashiga ("kompyuter ko'rish sindromi") olib keladi;
- **statik yuklama va gipodinamiya** — noto'g'ri o'tirish bo'yin, yelka va bel og'riqlarini keltirib chiqaradi;
- **takroriy harakatlar** — klaviatura va sichqoncha bilan uzoq ishlash bilak (tunnel) sindromiga sabab bo'lishi mumkin;
- **noqulay mikroiklim, yoritish va shovqin** — jihozlarning issiqligi, quruq havo, ekrandagi yaltirash, ventilyator va suhbatlar shovqini;
- **elektr toki urish xavfi** — shikastlangan kabellar va yerga ulanmagan jihozlar;
- **psixosotsional zo'riqish** — muddatlar bosimi, tungi navbatchiliklar va tizim avariylarini bartaraf etish.

Ish joyiga qo'yiladigan talablar Mehnat kodeksi, "Mehnatni muhofaza qilish to'g'risida"gi Qonun, sanitariya qoidalari hamda ISO 9241-5, GOST 12.2.032-78, GOST 12.1.005-88 va KMK 2.01.05-98 standartlari bilan belgilanadi.

Ish xonasiga qo'yiladigan talablar.

Bitta kompyuterli ish joyiga kamida 6 m² maydon va 20 m³ hajm to'g'ri kelishi kerak. Monitorlar derazaga nisbatan yon tomondan, yorug'lik chapdan tushadigan qilib joylashtiriladi: deraza ekran orqasida bo'lsa ko'z doimo moslashishga majbur bo'ladi, xodim orqasida bo'lsa ekranda yaltirash hosil bo'ladi. Monitorlar orasidagi masofa oldi-orqa bo'yicha kamida 2 m, yonma-yon kamida 1,2 m bo'ladi. Devor va shift matli materiallar bilan pardozlanadi, xona har kuni nam usulda tozalanadi va shamollatiladi.

Ish joyi jihozlarining ergonomik parametrlari.

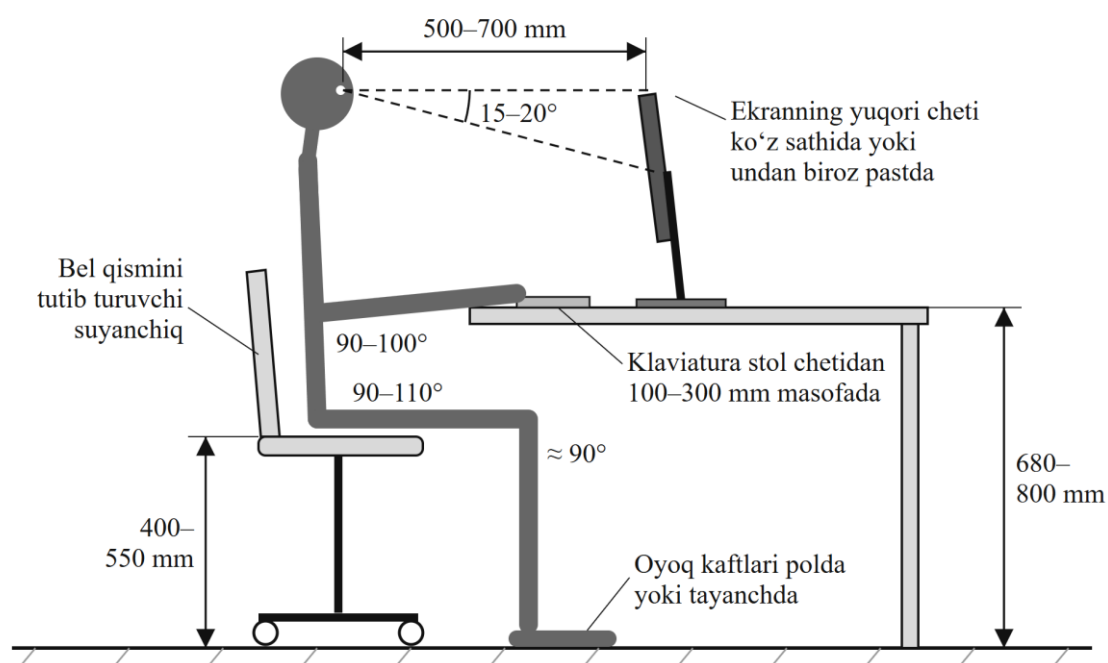
Dasturchining ish joyi ish stoli, sozlanadigan stul, monitor, klaviatura, sichqoncha va zarur bo'lsa oyoq tayanchidan iborat. Ularning tavsiya etiladigan o'lchamlari 4.1-jadvalda, xodimning to'g'ri gavda holati esa 4.1-rasmda keltirilgan.

4.1-jadval

Ish joyi elementlarining ergonomik parametrlari

Element	Parametr	Tavsiya etiladigan qiymat
Ish stoli	Ish yuzasining balandligi	680–800 mm (sozlanmaydigan stol uchun 725 mm)
	Ish yuzasining o'lchamlari	eni 800–1400 mm, chuqurligi 800–1000 mm
	Oyoqlar uchun bo'shliq	balandligi ≥ 600 mm, eni ≥ 500 mm, chuqurligi tizza sathida ≥ 450 mm, oyoq sathida ≥ 650 mm
Stul	O'rindiqlar balandligi	400–550 mm, sozlanadigan
	O'rindiqlar eni va chuqurligi	kamida 400 mm, old qirrasi yumaloqlangan
	Suyanchiq	balandligi 300 ± 20 mm, eni ≥ 380 mm, qiyalik burchagi $0...30^\circ$ oralig'ida sozlanadi
	Qo'l tayanchlari	uzunligi ≥ 250 mm, eni 50–70 mm, balandligi sozlanadi
Monitor	Ko'zdan ekrangacha masofa	500–700 mm

Element	Parametr	Tavsiya etiladigan qiymat
	Joylashuv balandligi	ekranning yuqori cheti ko‘z sathida yoki undan pastroq, nigoh ekran markaziga 15–20° pastga yo‘naladi
Klaviatura	Stol chetidan masofa	100–300 mm, bilak uchun tayanch joy qoldiriladi
Oyoq tayanchi	O‘lchamlari	eni ≥ 300 mm, chuqurligi ≥ 400 mm, qiyaligi 0...20°



4.1-rasm. Kompyuterda ishlovchi xodimning to‘g‘ri gavda holati va ish joyining asosiy o‘lchamlari

To‘g‘ri gavda holatining asosiy qoidasi — “to‘g‘ri burchaklar”: tirsaklar 90–100°, son-chanoq bo‘g‘imlari 90–110° va tizzalar taxminan 90° bukiladi, oyoqlar polga to‘liq tiraladi, bel suyanchiqqa tayanadi, bilaklar klaviatura ustida to‘g‘ri tutiladi. Ikki monitor bilan ishlaganda asosiysi ro‘parada, ikkinchisi yonida bir xil masofada joylashtiriladi, noutbuk ekrani esa tagliklar bilan ko‘z sathiga ko‘tariladi. Dasturiy interfeys ham ergonomik bo‘lishi kerak (ISO 9241-110): “Milliy chat”dagi yetarli kontrast, qorong‘i mavzu va moslashuvchan maket foydalanuvchilarning ko‘z charchog‘ini kamaytiradi.

Mikroiqlim, yoritish va shovqin.

Dasturchi mehnati energiya sarfi bo'yicha Ia toifadagi yengil ishlarga kiradi (energiya sarfi 139 W gacha). Bunday ishlar uchun ish zonası mikroiqlimining maqbul ko'rsatkichlari, yoritilganlik va shovqin me'yorlari 4.2-jadvalda keltirilgan.

4.2-jadval

Dasturchi ish joyi uchun mikroiqlim, yoritilganlik va shovqin me'yorlari

Ko'rsatkich	Yilning sovuq davri	Yilning iliq davri
Havo harorati, °C	22–24	23–25
Nisbiy namlik, %	40–60	40–60
Havo harakatining tezligi, m/s	≤ 0,1	≤ 0,1
Stol yuzasining yoritilganligi, lk	300–500	300–500
Ekran yuzasining yoritilganligi, lk	≤ 300	≤ 300
Yoritilganlik pulsatsiyasi koeffitsiyenti, %	≤ 5	≤ 5
Shovqin darajasi, dBA	≤ 50	≤ 50

Umumiy yoritish uchun miltillamaydigan, 4000–5000 K rang haroratli diffuzorli LED yoritgichlar monitorlar bilan parallel qatorlarda joylashtiriladi. Shovqinni kamaytirish uchun printerlar alohida joyga chiqariladi, ochiq ofisda tovush yutuvchi panellar o'rnatiladi, server uskunalari esa alohida xonada joylashtiriladi.

Ish xonasining sun'iy yoritilishini hisoblash.

Misol sifatida dasturchilar bo'limi uchun o'lchamlari $A \times B \times H = 6 \times 6 \times 3$ m bo'lgan xonani ko'rib chiqamiz. Xonaning maydoni $S = 36 \text{ m}^2$, hajmi $V = 108 \text{ m}^3$. Bitta ish joyiga kamida 6 m^2 maydon va 20 m^3 hajm to'g'ri kelishi kerakligidan xonaga joylashtirish mumkin bo'lgan ish joylari soni:

$$n_{\max} = \min(S / 6; V / 20) = \min(36 / 6; 108 / 20) = \min(6; 5,4) = 5 \quad (4.1)$$

Demak, xonada 5 ta ish joyi tashkil etiladi. Umumiy sun'iy yoritish yorug'lik oqimidan foydalanish koeffitsiyenti usuli bilan hisoblanadi. Avval xona indeksi aniqlanadi:

$$i = A \cdot B / (h \cdot (A + B)), \quad (4.2)$$

bu yerda h — yoritgichlarning ish yuzasidan balandligi. Yoritgichlar shiftga oʻrnatiladi, ish yuzasi poldan 0,8 m balandlikda joylashgan: $h = 3,0 - 0,8 = 2,2$ m. U holda $i = 36 / (2,2 \cdot 12) \approx 1,36$. Shift, devor va polning yorugʻlik qaytarish koʻeffitsiyentlari mos ravishda 70, 50 va 30 % boʻlganda LED panellar uchun bu indeksga yorugʻlik oqimidan foydalanish koʻeffitsiyenti $\eta \approx 0,54$ toʻgʻri keladi. Kerakli yoritgichlar soni quyidagi formula bilan topiladi:

$$N = E_n \cdot S \cdot K_z \cdot z / (\Phi \cdot \eta), \quad (4.3)$$

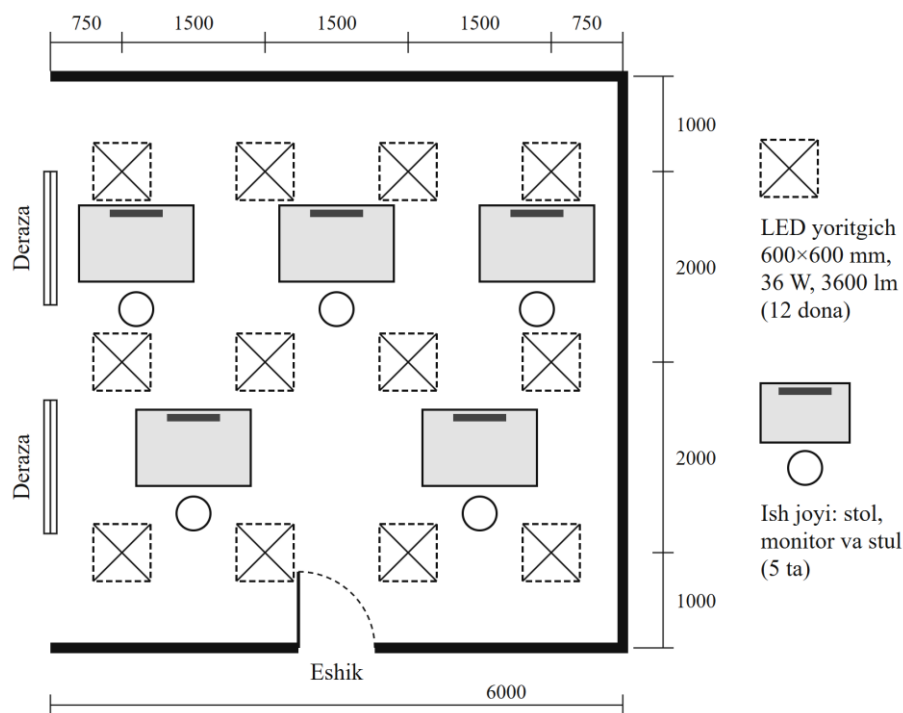
bu yerda $E_n = 400$ lk — meʼyoriy yoritilganlik; $K_z = 1,4$ — zaxira koʻeffitsiyenti (yoritgichlarning ifloslanishi va eskirishini hisobga oladi); $z = 1,1$ — yoritilganlikning notekislik koʻeffitsiyenti; $\Phi = 3600$ lm — bitta yoritgichning yorugʻlik oqimi (600×600 mm oʻlchamli, 36 W quvvatli LED panel). Hisoblash natijasi:

$$N = 400 \cdot 36 \cdot 1,4 \cdot 1,1 / (3600 \cdot 0,54) = 22\,176 / 1944 \approx 11,4 \quad (4.4)$$

Yoritgichlar soni butun songa yaxlitlanadi va simmetrik joylashuv uchun $N = 12$ qabul qilinadi (4 ta ustun $\times$ 3 ta qator). Haqiqiy oʻrtacha yoritilganlik quyidagiga teng:

$$E_h = N \cdot \Phi \cdot \eta / (S \cdot K_z \cdot z) = 12 \cdot 3600 \cdot 0,54 / (36 \cdot 1,4 \cdot 1,1) \approx 421 \text{ lk} \quad (4.5)$$

Haqiqiy yoritilganlik meʼyordan 5 % ga yuqori. Bu ruxsat etilgan $-10 \dots +20$ % oraligʻida. Yoritish tizimining umumiy quvvati $12 \cdot 36 = 432$ W, solishtirma quvvati 12 W/m². Yoritgichlar va ish joylarining joylashuvi 4.2-rasmda koʻrsatilgan.



4.2-rasm. Dasturchilar xonasida yoritgichlar va ish joylarining joylashuv sxemasi

Xonani shamollatish uchun zarur havo miqdori bitta xodimga to‘g‘ri keladigan hajmga bog‘liq. Bu hajm 20–40 m³ bo‘lganda (bizda $108 / 5 = 21,6$ m³) har bir xodimga soatiga kamida 30 m³ toza havo berilishi kerak:

$$L = n \cdot L_1 = 5 \cdot 30 = 150 \text{ m}^3/\text{soat}; \quad K = L / V = 150 / 108 \approx 1,4 \text{ soat}^{-1} \quad (4.6)$$

Demak, xonada havo soatiga kamida 1,4 marta almashishi kerak. Buni tabiiy shamollatish yetarli bo‘lmaganda kichik quvvatli so‘ruvchi-beruvchi ventilyatsiya yoki rekuperatorli konditsioner ta‘minlaydi.

Mehnat va dam olish rejimi, elektr xavfsizligi.

Kompyuterda uzluksiz ish 2 soatdan oshmasligi, har 45–60 daqiqada 5–10 daqiqalik tanaffus qilinishi tavsiya etiladi; tanaffusda ekrandan uzoqlashib, yengil mashqlar bajariladi. Ko‘z charchog‘ini kamaytirish uchun “20-20-20” qoidasi qo‘llanadi: har 20 daqiqada 20 soniya davomida 6 m uzoqlikdagi narsaga qaraladi. Ish beruvchi xodimlarni davriy tibbiy ko‘rik va yo‘riqnomadan o‘tkazadi, bu talablar masofadan ishlaganda ham amal qiladi.

Barcha jihozlar yerga ulash kontaktli rozetkalarga ulanadi, kabellar kanallarda yotqiziladi. Uzaytirgichlarni ketma-ket ulash, shikastlangan kabellardan foydalanish va jihoz korpusini ochish taqiqlanadi, server uskunalari esa UPS orqali ulanadi.

4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong'inlar sodir bo'lganda xodimlarning harakatlari

Favqulodda vaziyat — avariya, halokat, tabiiy ofat yoki boshqa hodisa natijasida odamlar hayoti, sog'lig'i va atrof-muhitga zarar yetishi mumkin bo'lgan holat. IT-ofis uchun eng ehtimoliy vaziyatlar yong'in, zilzila (Toshkent yuqori seysmik faollik zonasida joylashgan), elektr ta'minotidagi avariya, quvurlarning yorilishi va gaz sizishidir. Bunday vaziyatlarda xodimlar hayotini saqlashning asosiy usuli o'z vaqtida va tartibli evakuatsiyadir.

Asosiy talablar “Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to'g'risida”gi, “Yong'in xavfsizligi to'g'risida”gi qonunlar va GOST 12.1.004-91 standarti bilan belgilanadi.

IT-ofisda yong'in chiqish sabablari va o't o'chirish vositalari.

Ofis yong'inlarining aksariyati elektr tarmog'i bilan bog'liq: tarmoqning ortiqcha yuklanishi, eskirgan izolyatsiya tufayli qisqa tutashuv, noutbuk va powerbanklarning litiy-ion akkumulyatorlari qizib ketishi, UPS nosozligi va qarovsiz qoldirilgan maishiy jihozlar. Ofis materiallari yonganda ko'p zaharli tutun ajraladi va qurbonlarning aksariyati aynan tutundan jabrlanadi, shuning uchun tutunni erta aniqlash va tezkor evakuatsiya hal qiluvchi ahamiyatga ega.

Bino avtomatik yong'in signalizatsiyasi, xabardor qilish tizimi va birlamchi o't o'chirish vositalari bilan jihozlanadi, server xonasida esa avtomatik gazli o't o'chirish tizimi o'rnatiladi. Yong'in sinflari va o'chirish vositalari 4.3-jadvalda keltirilgan.

4.3-jadval

Yong'in sinflari va o't o'chirish vositalari

Sinf	Yonuvchi material	Qo'llaniladigan vositalar
A	Qattiq materiallar: qog'oz, yog'och, mato, plastmassa	suv, ko'pikli va kukunli (ABCE) o't o'chirgichlar, ichki yong'in krani

Sinf	Yonuvchi material	Qo'llaniladigan vositalar
B	Yonuvchi suyuqliklar: erituvchilar, bo'yoqlar, yoqilg'i	ko'pikli, kukunli va karbonat angidridli o't o'chirgichlar
C	Yonuvchi gazlar: tabiiy gaz, propan	kukunli o't o'chirgichlar; avvalo gaz berilishi to'xtatiladi
D	Metallar: magniy, natriy, alyuminiy kukuni	maxsus kukunli o't o'chirgichlar
E	Kuchlanish ostidagi elektr jihozlari	karbonat angidridli (OU) va kukunli o't o'chirgichlar; suv va ko'pik ishlatilmaydi

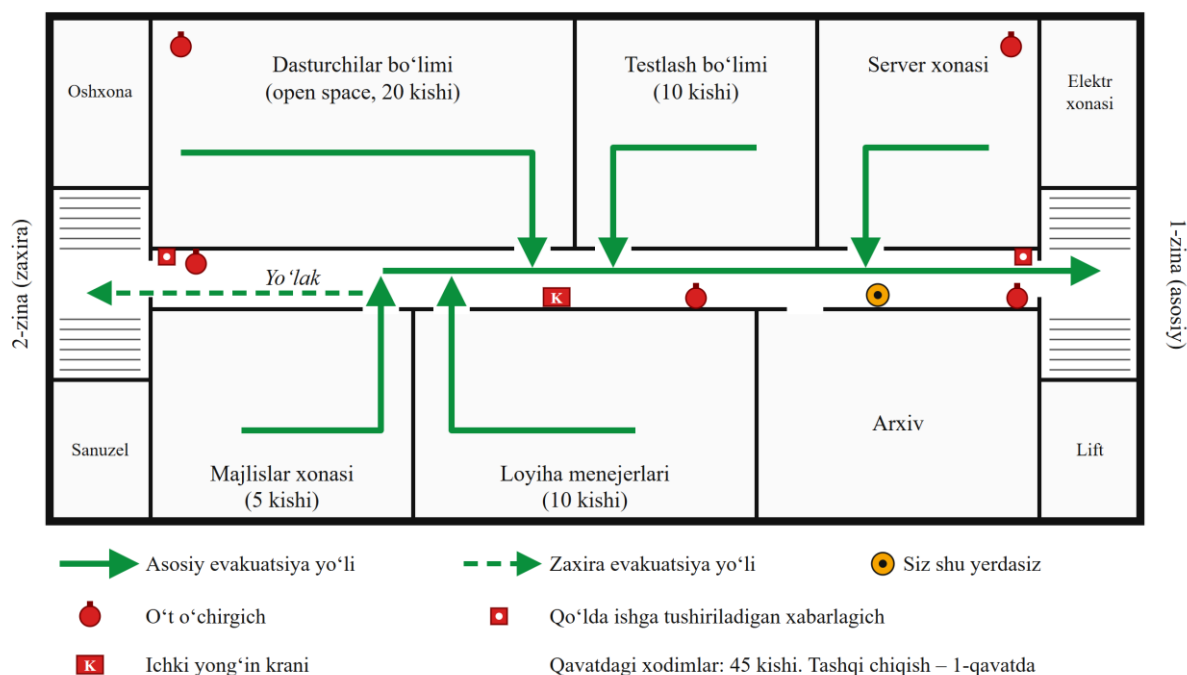
Kompyuter xonalarida elektr o'tkazmaydigan va qoldiq qoldirmaydigan karbonat angidridli o't o'chirgichlar (OU-3, OU-5) ishlatiladi; ularning karnayi –70 °C gacha sovigani uchun qo'l bilan ushlanmaydi. O't o'chirgichlar poldan 1,5 m dan baland bo'lmagan ko'rinadigan joyga, bir-biridan 20 m dan uzoq bo'lmagan masofada osiladi.

Evakuatsiya tadbirlarini rejalashtirish.

Evakuatsiya — odamlarni xavfli omillar ta'sir qila boshlashidan oldin binodan xavfsiz hududga uyushgan holda chiqarish jarayoni. Uni rejalashtirish texnik va tashkiliy tadbirlardan iborat. Texnik tadbirlar:

- har bir qavatda kamida ikkita, bir-biridan uzoqda joylashgan evakuatsiya chiqishi;
- yo'laklar kengligi kamida 1–1,2 m, eshiklar kamida 0,8 m; eshiklar chiqish tomonga va ichkaridan kalitsiz ochiladi; yo'l va zinlarni to'sib qo'yish taqiqlanadi;
- avariya yoritishi, ISO 7010 bo'yicha yashil "Favqulodda chiqish" belgilari, yong'in signalizatsiyasi va ovozli xabardor qilish tizimi;
- har bir qavatda fotolyuminessent materialdan tayyorlangan evakuatsiya reja-sxemalari.

Reja-sxemada qavat rejasi, "Siz shu yerdasiz" belgisi, asosiy (yashil yaxlit) va zaxira (yashil punktir) yo'llar, chiqishlar, o't o'chirgichlar va xabarlagichlar ko'rsatiladi. Namunaviy IT-ofis 3-qavatining reja-sxemasi 4.3-rasmda keltirilgan.



4.3-rasm. IT-kompaniya ofisi 3-qavatining evakuatsiya reja-sxemasi (namuna)

Tashkiliy tadbirlar:

- rahbar buyrug'i bilan yong'in xavfsizligi uchun mas'ul shaxs va har bir qavat bo'yicha evakuatsiya mas'ullari tayinlanadi;
- binodan tashqarida xodimlar yig'iladigan joy belgilanadi va ro'yxatlar tayyorlanadi;
- xodimlar kirish va davriy yo'riqnomadan o'tkaziladi, yarim yilda kamida bir marta amaliy mashg'ulot o'tkaziladi;
- o't o'chirish vositalari, signalizatsiya va avariya yoritishining sozligi muntazam tekshiriladi.

Evakuatsiya vaqtini hisoblash.

Evakuatsiya yo'llarining yetarliligini baholash uchun hisobiy evakuatsiya vaqti aniqlanadi. Hisoblash GOST 12.1.004-91 standartida keltirilgan odamlar oqimining harakat parametrlari asosida soddalashtirilgan usulda bajariladi. Evakuatsiya yo'li o'lchamlari va odamlar soni bir xil bo'lgan uchastkalarga ajratiladi. Har bir uchastkadagi odamlar oqimining zichligi quyidagicha aniqlanadi:

$$D_i = N_i \cdot f / (l_i \cdot \delta_i), \quad (4.7)$$

bu yerda N_i — uchastkadagi odamlar soni; $f = 0,1 \text{ m}^2$ — yozgi kiyimdagi katta yoshli odamning gorizontal proyeksiya maydoni; l_i va δ_i — uchastkaning uzunligi va kengligi, m. Zichlikka qarab 4.4-jadvaldan harakat tezligi v_i va oqim intensivligi q_i olinadi. Uchastkadan o'tish vaqti va umumiy hisobiy evakuatsiya vaqti:

$$t_i = l_i / v_i; \quad t_h = t_1 + t_2 + \dots + t_n \quad (4.8)$$

Eshiklar va yo'l torayadigan joylarda tirbandlik bor-yo'qligi oqim intensivligi orqali tekshiriladi:

$$q_e = q_i \cdot \delta_i / \delta_e \leq q_{\max}, \quad (4.9)$$

bu yerda δ_e — eshik kengligi; $q_{\max} = 19,6 \text{ m/min}$ — eshikdan o'tishdagi oqim intensivligining eng katta qiymati. Bu shart bajarilsa, eshik oldida odamlar to'planib qolmaydi.

4.4-jadval

Odamlar oqimining harakat parametrlari (GOST 12.1.004-91 bo'yicha, parcha)

Oqim zichligi $D, \text{ m}^2/\text{m}^2$	Gorizontal yo'l: $v, \text{ m/min}$	Gorizontal yo'l: $q, \text{ m/min}$	Eshik: $q, \text{ m/min}$	Zinadan pastga: $v, \text{ m/min}$	Zinadan pastga: $q, \text{ m/min}$
0,01	100	1,0	1,0	100	1,0
0,05	100	5,0	5,0	100	5,0
0,10	80	8,0	8,7	95	9,5
0,20	60	12,0	13,4	68	13,6
0,30	47	14,1	16,5	52	15,6
0,40	40	16,0	18,4	40	16,0
0,50	33	16,5	19,6	31	15,5

4.3-rasmdagi qavat uchun hisoblashni bajaramiz. Qavatda 45 nafar xodim ishlaydi. Eng og'ir holat ko'rib chiqiladi: 2-zina yong'in tufayli to'sib qo'yilgan va barcha xodimlar asosiy (1-) zina orqali chiqadi. Hisobiy yo'l dasturchilar bo'limining eng uzoq ish joyidan boshlanadi va to'rtta uchastkadan iborat:

1. dasturchilar bo'limi ichidagi o'tish joyi: $l_1 = 12 \text{ m}$, $\delta_1 = 1,5 \text{ m}$, $N_1 = 20$ kishi;
2. yo'lak: bo'lim eshigidan zinagacha $l_2 = 17 \text{ m}$, $\delta_2 = 2,0 \text{ m}$, $N_2 = 45$ kishi;

3. 3-qavatdan 1-qavatgacha zina marshlari va maydonchalari: $l_3 = 2 \cdot 10 = 20$ m, $\delta_3 = 1,35$ m, $N_3 = 45$ kishi;

4. 1-qavat vestibulidan tashqi chiqishgacha: $l_4 = 15$ m, $\delta_4 = 3,0$ m, $N_4 = 45$ kishi.

Masalan, birinchi uchastka uchun $D_1 = 20 \cdot 0,1 / (12 \cdot 1,5) \approx 0,11$. 4.4-jadvaldagi qiymatlar orasida chiziqli interpolyatsiya qilib $v_1 \approx 78$ m/min va $q_1 \approx 8,4$ m/min topiladi, $t_1 = 12 / 78 \approx 0,15$ min. Qolgan uchastkalar ham xuddi shunday hisoblanadi. Natijalar 4.5-jadvalda keltirilgan.

4.5-jadval

Evakuatsiya vaqtini hisoblash natijalari

Uchastka	l, m	δ , m	N, kishi	D, m ² /m ²	v, m/min	t, min
1. Bo‘lim ichidagi o‘tish joyi	12	1,5	20	0,11	78	0,15
2. Yo‘lak	17	2,0	45	0,13	74	0,23
3. Zina (pastga)	20	1,35	45	0,17	77	0,26
4. Vestibul	15	3,0	45	0,10	80	0,19
Jami hisobiy evakuatsiya vaqti t_h						0,83

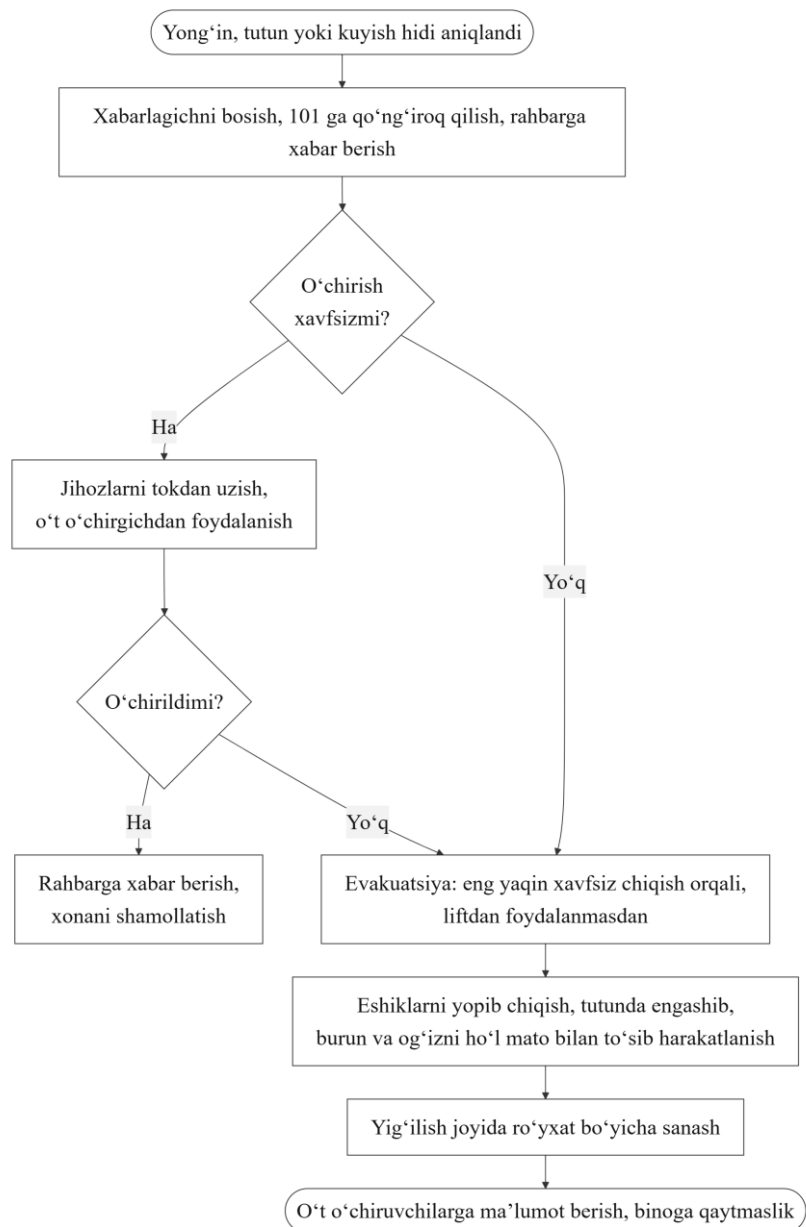
Eshiklarda tirbandlik yo‘qligini (4.9) formula bilan tekshiramiz. Bo‘lim eshigi ($\delta = 1,2$ m): $q = 8,4 \cdot 1,5 / 1,2 \approx 10,5$ m/min. Zinaga kirish eshigi ($\delta = 1,2$ m): $q = 9,3 \cdot 2,0 / 1,2 \approx 15,5$ m/min. Tashqi eshik ($\delta = 1,5$ m): $q = 8,0 \cdot 3,0 / 1,5 = 16,0$ m/min. Barcha qiymatlar $q_{\max} = 19,6$ m/min dan kichik, ya’ni odamlar oqimi to‘xtovsiz harakatlanadi.

Demak, hisobiy evakuatsiya vaqti $t_h \approx 0,83$ min (taxminan 50 soniya), evakuatsiya boshlanishidagi 1 daqiqalik kechikish bilan esa qavatdan to‘liq chiqish taxminan 2 daqiqa oladi. Bu vaqt yong‘in xavfli omillari evakuatsiya yo‘llarida kritik qiymatga yetish vaqtidan kichik bo‘lishi shart. Eng tor joy zina bo‘lgani uchun zinaga olib boradigan eshiklar o‘zi yopiladigan, lekin qulflanmagan bo‘lishi, zina maydonchalarida esa buyum saqlanmasligi kerak.

Yong‘in sodir bo‘lganda xodimlarning harakatlari.

Yong‘in chiqqanda vahimaga tushmasdan, oldindan o‘rgatilgan tartibda harakat qilish kerak (4.4-rasm):

1. Yong'in yoki tutunni sezgan xodim xabarlagichni bosadi va 101 (yoki 112) raqamiga manzil, qavat, nima yonayotgani va odamlar bor-yo'qligini aytadi, so'ng mas'ul shaxsga xabar beradi.
2. Yong'in endi boshlangan va bu xavfsiz bo'lsa, jihozlar tarmoqdan uziladi va birlamchi vositalar bilan o'chiriladi; kuchlanish ostidagi jihozga suv sepilmaydi.
3. Yong'inni 1–2 daqiqada o'chirib bo'lmasa, reja-sxemadagi eng yaqin chiqish orqali evakuatsiya boshlanadi; lift dan foydalanish taqiqlanadi.
4. Xonadan chiqqanda eshik yopiladi, lekin qulflanmaydi; derazalar ochilmaydi.
5. Tutunli yo'lakda engashib harakatlaniladi, burun va og'iz ho'l mato bilan to'siladi.
6. Yig'ilish joyida qavat mas'uli xodimlarni ro'yxat bo'yicha sanaydi va yetib kelmaganlar haqida o't o'chiruvchilarga xabar beradi; ruxsatsiz binoga qaytilmaydi.



4.4-rasm. Yong'in sodir bo'lganda xodimlarning harakatlar algoritmi

Mas'ul shaxs evakuatsiyani boshqaradi, ventilyatsiya va konditsionerlarni o'chirishni tashkil etadi (ular tutunni tarqatadi) va o't o'chirish bo'linmasini kutib oladi. Kiyimga olov tushsa, yugurmasdan yerga yotib dumalash yoki qalin mato bilan o'rab o'chirish kerak; kuygan joy 10–20 daqiqa salqin oqar suvda sovutiladi, zaharlangan odam toza havoga olib chiqiladi va 103 chaqiriladi.

Boshqa avariyalarda xodimlarning harakatlari.

- **Zilzila** — zina va liftga yugurilmaydi; mustahkam stol ostiga yoki kapital devor burchagiga yashirinib, derazalar va shkaflardan uzoqlashiladi; silkinish tugagach bino tark etiladi.

- **Server xonasida gazli o‘t o‘chirish** — ogohlantiruvchi signalda xona darhol tark etiladi, shamollatilmaguncha unga kirilmaydi.
- **Elektr toki urishi** — jabrlanuvchiga yalang qo‘l tegizilmaydi, avval tok manbai o‘chiriladi, so‘ng 103 chaqirilib, zarur bo‘lsa reanimatsiya boshlanadi.
- **Gaz sizishi** — olov yoqilmaydi va elektr kalitlariga tegilmaydi, derazalar ochilib, odamlar chiqariladi va 104 ga xabar beriladi.

Xodimlarni tezkor xabardor qilish uchun korporativ aloqa vositalaridan ham foydalanish mumkin. Masalan, “Milliy chat”dagi tashkilotning domen guruhi orqali evakuatsiya haqidagi xabar va yig‘ilish joyi bir necha soniyada barcha xodimlarning telefon va kompyuterlariga yetkaziladi, xodimlar esa xavfsiz joyga yetib kelganini javob bilan tasdiqlaydi. Bu vosita signalizatsiya va ovozli xabardor qilish tizimini to‘ldiradi va ayniqsa masofadan ishlayotgan xodimlar uchun foydali.

4-bob bo‘yicha xulosa

To‘rtinchi bobda kompyuter bilan ishlashdagi xavfli va zararli omillar aniqlanib, ish xonasi, jihozlar, mikroiklim, yoritish va mehnat rejimiga qo‘yiladigan ergonomik talablar keltirildi. 6×6 m o‘lchamli xona uchun ish joylari soni (5 ta), sun‘iy yoritish (12 ta 36 W li LED yoritgich, 421 lk) va havo almashinuvi (150 m³/soat) hisoblandi. Ikkinchi bo‘limda IT-ofisda yong‘in sabablari, o‘t o‘chirish vositalari va evakuatsiyani rejalashtirish tadbirlari ko‘rib chiqildi, namunaviy qavat uchun reja-sxema ishlab chiqildi. 45 kishilik qavat uchun hisobiy evakuatsiya vaqti 0,83 daqiqa ekani va eshiklarda tirbandlik bo‘lmasligi hisoblab ko‘rsatildi, yong‘in va boshqa avariyalarda xodimlarning harakatlar tartibi berildi.

XULOSA

Bitiruv malakaviy ishida foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan, Telegramga o'xshash "Milliy chat" veb-messenjeri loyihalandi va ishlab chiqildi. Ishning kirish qismida qo'yilgan vazifalar quyidagicha hal etildi.

- Telegram, WhatsApp, Signal, Viber va WeChat messenjerlari funksional imkoniyatlari, xavfsizlik modeli va ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi xorijiy yurisdiksiyada ekani va bu "Shaxsga doir ma'lumotlar to'g'risida"gi Qonunning lokalizatsiya talablari nuqtai nazaridan milliy yechim zarurligini asoslashi ko'rsatildi.
- Real vaqtda aloqa texnologiyalari (short va long polling, Server-Sent Events, WebSocket, SignalR) tahlil qilindi. WebSocket samaradorligini saqlagan holda zaxira transportlarga avtomatik o'tadigan SignalR tanlandi.
- Tizimning aktorlari, 15 ta funksional va 7 guruh nofunksional talablari aniqlandi.
- Server qismi Clean Architecture tamoyillari asosida Domain, Application, Infrastructure va API qatlamlariga ajratildi. PostgreSQL ma'lumotlar bazasi 10 ta sxemadagi 36 ta jadvaldan iborat qilib loyihalandi. Tizim ER, use-case va ketma-ketlik diagrammalari bilan modellashtirildi.
- Parolsiz autentifikatsiya tizimi ishlab chiqildi. U elektron pochtaga yuboriladigan va PBKDF2 bilan xeshlanadigan bir martalik kod, Google hisobi, 15 daqiqalik JWT access token, HMAC xeshi saqlanadigan va har safar almashtiriladigan refresh token hamda qurilmalar bo'yicha sessiyalarni boshqarishni o'z ichiga oladi.
- .NET 8 platformasida 73 ta endpointli REST API va SignalR habi amalga oshirildi. Ular shaxsiy va guruh chatlari, xabarlarni yuborish, tahrirlash, o'chirish, javob berish va qidirish, rasm, video va fayllar almashish, "yozmoqda" va o'qilganlik belgilari, onlayn holat hamda 24 soatlik hikoyalarni ta'minlaydi. Guruhlar uchun egasi, administrator va a'zo rollariga asoslangan ruxsatlar tizimi, xizmat xabarlari va taklif havolalari yaratildi.

- Angular 22 freymvorkida signals asosidagi holat boshqaruvi, o‘z dizayn tizimi, yorug‘ va qorong‘i mavzular va mobil qurilmalarga moslashuvchan o‘zbek tilidagi interfeys ishlab chiqildi. Ilova qat’iy Content Security Policy bilan himoyalangan.
- O‘zbek tilining lotin va kirill yozuvlari o‘rtasida kontekstga bog‘liq qoidalarni hisobga oluvchi transliteratsiya algoritmi ishlab chiqildi. Har bir foydalanuvchi xabarlarini asl holida, lotin yoki kirill yozuvida ko‘radi, chat ichidagi qidiruv esa yozuvga bog‘liq emas. Algoritmning server (C#) va klient (TypeScript) nusxalari 80 ta juftlikdan iborat yagona test to‘plami bilan tekshiriladi.
- Tashkilot rejimi amalga oshirildi: e-pochta domeni tashkilot sifatida qabul qilinadi, uning tasdiqlangan a‘zolari “tuit.uz ✓” kabi belgi oladi va tashkilotning yopiq guruhiga avtomatik qo‘shiladi. Umumiy pochta xizmatlari foydalanuvchilari bu guruhlarga faqat admin qo‘shganda yoki 128 bitli taklif havolasi orqali kira oladi.
- Web Crypto API asosida uchdan-uchgacha shifrlangan maxfiy chatlar yaratildi: X25519 kalit kelishuvi, HKDF, HMAC asosidagi kalitlar zanjiri (oldinga maxfiylik) va AES-256-GCM shifrlash, kalit izini tekshirish, o‘z-o‘zini o‘chirish taymeri va shifrlangan fayllar. Serverda faqat ochiq kalitlar va yetkazilguncha saqlanadigan shifrlangan bloblar turishi ma’lumotlar bazasi darajasida ko‘rsatib berildi.
- Server ilovasi Docker konteyneri ko‘rinishida bulutli platformaga joylashtirildi. Tizimning to‘g‘riligi serverdagi 307 ta va klientdagi 229 ta unit test, API darajasidagi 43 ta integratsion tekshiruv va qo‘lda o‘tkazilgan sinov ssenariylari bilan tasdiqlandi.
- Hayot faoliyati xavfsizligi bo‘limida dasturchining ish joyiga qo‘yiladigan ergonomik talablar tizimlashtirildi, dasturchilar xonasi uchun ish joylari soni, sun’iy yoritish va havo almashinuvi hisoblandi. IT-ofis uchun evakuatsiya reja-sxemasi ishlab chiqildi, hisobiy evakuatsiya vaqti

aniqlandi, yong'in va boshqa avariyalarda xodimlarning harakatlar tartibi belgilandi.

Ishning amaliy ahamiyati shundaki, "Milliy chat" to'liq ishlaydigan dasturiy mahsulot bo'lib, uni ta'lim muassasalari, tashkilotlar yoki davlat idoralari o'z serverlarida, jumladan respublika hududidagi data-markazlarda joylashtirib, ichki muloqot vositasi sifatida qo'llashi mumkin. Bunda foydalanuvchi ma'lumotlari ustidan to'liq nazorat mamlakat ichida qoladi. Tashkilot rejimi korporativ pochta orqali a'zolari avtomatik birlashtiradi, lotin–kirill transliteratsiyasi turli avlod foydalanuvchilarining bir-birini tushunishini osonlashtiradi, maxfiy chatlar esa eng nozik yozishmalarni serverning o'zidan ham himoya qiladi. Qatlamli arxitektura va oldindan loyihalangan ma'lumotlar bazasi sxemasi tizimni bosqichma-bosqich kengaytirish imkonini beradi.

Kelajakda tizimni quyidagi yo'nalishlarda rivojlantirish rejalashtirilgan: kanallar va botlar platformasi; WebRTC asosidagi ovozli va video qo'ng'iroqlar; OneID (id.egov.uz) tizimi orqali shaxsni davlat darajasida tasdiqlash; maxfiy chatlarni ko'p qurilmali modelga va to'liq Double Ratchet protokoliga o'tkazish; ikki bosqichli autentifikatsiya va foydalanuvchilarni bloklash; xabarlarga reaksiyalar, so'rovnomalar va stikerlar; push-bildirishnomalar; mobil ilovalar (Android, iOS); Redis va bir nechta server nusxasi yordamida gorizontol kengaytirish.

FOYDALANILGAN ADABIYOTLAR

Normativ-huquqiy hujjatlar

1. O‘zbekiston Respublikasining “Shaxsga doir ma’lumotlar to‘g‘risida”gi Qonuni. 2019-yil 2-iyul, O‘RQ-547-son.
2. O‘zbekiston Respublikasining “Axborotlashtirish to‘g‘risida”gi Qonuni. 2003-yil 11-dekabr, 560-II-son.
3. O‘zbekiston Respublikasining Mehnat kodeksi. 2022-yil 28-oktabr, O‘RQ-798-son.
4. O‘zbekiston Respublikasining “Mehnatni muhofaza qilish to‘g‘risida”gi Qonuni. 2016-yil 22-sentabr, O‘RQ-410-son.
5. O‘zbekiston Respublikasining “Yong‘in xavfsizligi to‘g‘risida”gi Qonuni. 2009-yil 30-sentabr, O‘RQ-226-son.
6. O‘zbekiston Respublikasining “Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to‘g‘risida”gi Qonuni. 1999-yil 20-avgust, 824-I-son.
7. O‘zbekiston Respublikasi Prezidentining “«Raqamli O‘zbekiston – 2030» strategiyasini tasdiqlash va uni samarali amalga oshirish chora-tadbirlari to‘g‘risida”gi Farmoni. 2020-yil 5-oktabr, PF-6079-son.
8. O‘zbekiston Respublikasining “Lotin yozuviga asoslangan o‘zbek alifbosini joriy etish to‘g‘risida”gi Qonuni. 1993-yil 2-sentabr, 931-XII-son (1995-yil 6-maydagi 71-I-son Qonun tahririda).
9. O‘zbekiston Respublikasi Vazirlar Mahkamasining “O‘zbek tilining asosiy imlo qoidalarini tasdiqlash to‘g‘risida”gi qarori. 1995-yil 24-avgust, 339-son.

Asosiy adabiyotlar

10. Martin R. C. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 p.
11. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. — Prentice Hall, 2008. — 464 p.

- 12.Kleppmann M. Designing Data-Intensive Applications. — O'Reilly Media, 2017. — 616 p.
- 13.Fowler M. Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — 560 p.
- 14.Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. — Addison-Wesley, 2003. — 560 p.
- 15.Lock A. ASP.NET Core in Action. 3rd ed. — Manning Publications, 2023.
- 16.Smith J. P. Entity Framework Core in Action. 2nd ed. — Manning Publications, 2021.
- 17.Price M. J. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. 8th ed. — Packt Publishing, 2023.
- 18.Obe R. O., Hsu L. S. PostgreSQL: Up and Running. 3rd ed. — O'Reilly Media, 2017.
- 19.Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. — Pearson, 2022.
- 20.Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed. — Addison-Wesley, 2003.
- 21.Yormatov G'.Yo., Isamuxamedov Yo.U. Hayot faoliyati xavfsizligi. — T.: Aloqachi, 2009.

Standartlar va texnik spetsifikatsiyalar

- 22.Fette I., Melnikov A. RFC 6455: The WebSocket Protocol. — IETF, 2011. — <https://www.rfc-editor.org/rfc/rfc6455>
- 23.Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). — IETF, 2015. — <https://www.rfc-editor.org/rfc/rfc7519>
- 24.Moriarty K., Kaliski B., Rusch A. RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1. — IETF, 2017.
- 25.Krawczyk H., Bellare M., Canetti R. RFC 2104: HMAC: Keyed-Hashing for Message Authentication. — IETF, 1997.

26. Langley A., Hamburg M., Turner S. RFC 7748: Elliptic Curves for Security (X25519, X448). — IRTF, 2016. — <https://www.rfc-editor.org/rfc/rfc7748>
27. Krawczyk H., Eronen P. RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). — IETF, 2010.
28. Dworkin M. NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. — NIST, 2007.
29. NIST FIPS 180-4: Secure Hash Standard (SHS). — NIST, 2015.
30. W3C. Web Cryptography API. W3C Recommendation. — <https://www.w3.org/TR/WebCryptoAPI/>
31. W3C. Content Security Policy Level 3. — <https://www.w3.org/TR/CSP3/>
32. Perrin T., Marlinspike M. The Double Ratchet Algorithm. — Signal, 2016. — <https://signal.org/docs/specifications/doubleratchet/>
33. OWASP Foundation. OWASP Top 10 — 2021. — <https://owasp.org/Top10/>
34. ISO 9241-5:1998. Ergonomic requirements for office work with visual display terminals (VDTs) — Part 5: Workstation layout and postural requirements.
35. ISO 7010:2019. Graphical symbols — Safety colours and safety signs — Registered safety signs.
36. GOST 12.2.032-78. SSBT. Rabocheye mesto pri vypolnenii rabot sidya. Obshchiye ergonomicheskiye trebovaniya.
37. GOST 12.1.005-88. SSBT. Obshchiye sanitarno-gigiyenicheskiye trebovaniya k vozdukhу rabochey zony.
38. GOST 12.1.004-91. SSBT. Pozharnaya bezopasnost. Obshchiye trebovaniya.
39. KMK 2.01.05-98. Tabiiy va sun'iy yoritish. — T.: O'zbekiston Respublikasi Davarxitektqurilish qo'mitasi, 1998.

Internet manbalari

40. Microsoft. ASP.NET Core SignalR documentation. —
<https://learn.microsoft.com/aspnet/core/signalr/>
41. Microsoft. Entity Framework Core documentation. —
<https://learn.microsoft.com/ef/core/>
42. Microsoft. ASP.NET Core fundamentals. —
<https://learn.microsoft.com/aspnet/core/>
43. Angular. Official documentation. — <https://angular.dev>
44. The PostgreSQL Global Development Group. PostgreSQL Documentation. — <https://www.postgresql.org/docs/>
45. Docker Inc. Docker documentation: Multi-stage builds. —
<https://docs.docker.com/build/building/multi-stage/>
46. Telegram. MTProto Mobile Protocol. — <https://core.telegram.org/mtproto>
47. Signal Messenger. Technical documentation. — <https://signal.org/docs/>
48. FluentValidation documentation. — <https://docs.fluentvalidation.net>
49. xUnit.net documentation. — <https://xunit.net>
50. MDN Web Docs. SubtleCrypto (Web Crypto API). —
<https://developer.mozilla.org/docs/Web/API/SubtleCrypto>
51. Vitest documentation. — <https://vitest.dev>

ILOVALAR

1-ilova. SignalR habi (ChatHub)

Fayl: NationalChat/src/API/Hubs/ChatHub.cs

```
using System.Security.Claims;
using Application.Features.Authentication;
using Application.Features.Messages;
using Application.Features.Presence;
using Application.Features.Profiles;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;

namespace API.Hubs;

[Authorize]
public sealed class ChatHub(
    IMessageRepository messageRepository,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IAuthRepository authRepository,
    IProfileRepository profileRepository,
    TimeProvider timeProvider) : Hub
{
    private static readonly TimeSpan OfflineGracePeriod = TimeSpan.FromSeconds(8);

    public override async Task OnConnectedAsync()
    {
        var userId = GetCurrentUserId();
        await Groups.AddToGroupAsync(Context.ConnectionId,
            ChatHubGroups.User(userId));
        await Groups.AddToGroupAsync(Context.ConnectionId,
            ChatHubGroups.Session(GetCurrentSessionId()));

        if (presenceTracker.AddConnection(userId, Context.ConnectionId))
        {
            var relatedUserIds = await
                profileRepository.GetRelatedUserIdsAsync(userId, Context.ConnectionAborted);
            await realtimeNotifier.UserPresenceChangedAsync(userId, true, null,
                relatedUserIds, Context.ConnectionAborted);
        }

        await base.OnConnectedAsync();
    }

    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = GetCurrentUserId();
        var sessionId = GetCurrentSessionId();
        await base.OnDisconnectedAsync(exception);

        if (!presenceTracker.RemoveConnection(userId, Context.ConnectionId))
        {
            return;
        }

        // Grace period absorbs quick reconnects (page refresh, brief network drop)
        // without flashing "offline".
        await Task.Delay(OfflineGracePeriod, CancellationToken.None);
        if (presenceTracker.IsOnline(userId))
        {
            return;
        }
    }
}
```

```

    }

    var lastSeenAt = timeProvider.GetUtcNow().UtcDateTime;
    var session = await authRepository.FindSessionAsync(userId, sessionId,
CancellationToken.None);
    if (session is not null)
    {
        session.LastActiveAt = lastSeenAt;
        await authRepository.SaveChangesAsync(CancellationToken.None);
    }

    var recipientUserIds = await profileRepository.GetRelatedUserIdsAsync(userId,
CancellationToken.None);
    await realtimeNotifier.UserPresenceChangedAsync(userId, false, lastSeenAt,
recipientUserIds, CancellationToken.None);
}

public async Task JoinChat(int chatId)
{
    await EnsureChatMembershipAsync(chatId);
    await Groups.AddToGroupAsync(Context.ConnectionId,
ChatHubGroups.Chat(chatId));
}

public Task LeaveChat(int chatId) =>
    Groups.RemoveFromGroupAsync(Context.ConnectionId, ChatHubGroups.Chat(chatId));

public async Task SetTyping(int chatId, bool isTyping)
{
    await EnsureChatMembershipAsync(chatId);
    await Clients.OthersInGroup(ChatHubGroups.Chat(chatId))
        .SendAsync("TypingChanged", new { chatId, userId = GetCurrentUserId(),
isTyping });
}

public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int>
messageIds)
{
    await EnsureChatMembershipAsync(chatId);
    var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
    if (ids.Length == 0) return;

    var readerUserId = GetCurrentUserId();
    await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
timeProvider.GetUtcNow().UtcDateTime, Context.ConnectionAborted);
    await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids, await
messageRepository.GetMemberUserIdsAsync(chatId, Context.ConnectionAborted),
Context.ConnectionAborted);
}

private async Task EnsureChatMembershipAsync(int chatId)
{
    if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUserId(),
Context.ConnectionAborted))
    {
        throw new HubException("Bu chatga kirish huquqi yo'q.");
    }
}

private int GetCurrentUserId() =>
    int.Parse(Context.User?.FindFirstValue(ClaimTypes.NameIdentifier)
        ?? throw new HubException("Foydalanuvchi aniqlanmadi.));

private int GetCurrentSessionId() =>
    int.Parse(Context.User?.FindFirstValue("sid"))

```

```

        ?? throw new HubException("Sessiya aniqlanmadi.");
    }

```

2-ilova. Guruh chatlari servisi (GroupService)

Fayl: NationalChat/src/Application/Features/Groups/Services/GroupService.cs

```

using Application.Features.Files;
using Application.Features.Files.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Responses;
using Application.Features.Groups.Factories;
using Application.Features.Groups.Mappers;
using Application.Features.Messages;
using Application.Features.Messages.Factories;
using Application.Features.Messages.Mappers;
using Application.Features.Presence;
using Domain.Entities;
using FluentValidation;

namespace Application.Features.Groups;

public sealed class GroupService(
    IGroupRepository repository,
    IFileService fileService,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IValidator<CreateGroupRequest> createValidator,
    IValidator<UpdateGroupRequest> updateValidator,
    IValidator<AddGroupMembersRequest> addMembersValidator,
    IValidator<UpdateGroupMemberRoleRequest> updateRoleValidator,
    TimeProvider timeProvider) : IGroupService
{
    private const string NotFoundError = "Guruh topilmadi.";
    private const string ForbiddenError = "Bu amal uchun ruxsat yo'q.";

    public async Task<GroupDto?> GetAsync(int currentUserId, int chatId,
        CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        return group is null || FindMember(group, currentUserId) is null ? null :
        ToDto(group, currentUserId);
    }

    public async Task<GroupResult> CreateAsync(int currentUserId, CreateGroupRequest
        request, CancellationToken cancellationToken = default)
    {
        var validation = await createValidator.ValidateAsync(request,
            cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var memberIds = request.MemberIds.Where(id => id !=
            currentUserId).Distinct().ToArray();
        if (memberIds.Length == 0) return Fail("Kamida bitta a'zo tanlanishi kerak.");

        var users = await repository.FindUsersAsync([currentUserId, .. memberIds],
            cancellationToken);
        var creator = users.FirstOrDefault(user => user.Id == currentUserId);
        var members = users.Where(user => user.Id != currentUserId).ToArray();
        if (creator is null) return Fail("Foydalanuvchi aniqlanmadi.");
        if (members.Length != memberIds.Length) return Fail("Ba'zi foydalanuvchilar
            topilmadi.");
    }

```

```

        var now = timeProvider.GetUtcNow().UtcDateTime;
        var group = GroupFactory.Create(request.Title.Trim(),
NormalizeDescription(request.Description), creator, members, now);
        await repository.AddGroupAsync(group, cancellationTokens);
        await repository.SaveChangesAsync(cancellationTokens);

        await PublishServiceMessageAsync(group.Chat.Messages.Single(), group,
cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateAsync(int currentUserId, int chatId,
UpdateGroupRequest request, CancellationTokens cancellationTokens = default)
    {
        var validation = await updateValidator.ValidateAsync(request,
cancellationTokens);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var title = request.Title.Trim();
        var titleChanged = title != group.Title;
        group.Title = title;
        group.Description = NormalizeDescription(request.Description);
        var serviceMessage = titleChanged
            ? AddServiceMessage(group, actor.User, MessageServiceAction.TitleChanged,
title)
            : null;
        await repository.SaveChangesAsync(cancellationTokens);

        if (serviceMessage is not null) await
PublishServiceMessageAsync(serviceMessage, group, cancellationTokens);
        await PublishGroupUpdatedAsync(group, cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdatePhotoAsync(int currentUserId, int chatId,
StoreImageRequest request, CancellationTokens cancellationTokens = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var storedResult = await fileService.StoreImageAsync(request,
cancellationTokens);
        if (storedResult.File is null) return Fail(storedResult.Error ?? "Rasm
yuklanmadi.");
        var stored = storedResult.File;

        var oldFile = group.Photo?.File;
        var file = new Domain.Entities.File { Name = stored.FileName, MimeType =
stored.MimeType, SizeBytes = stored.SizeBytes, StoragePath = stored.StoragePath };
        var photo = new Photo { File = file, Width = stored.Width, Height =
stored.Height };
        await repository.AddPhotoAsync(file, photo, cancellationTokens);
        group.Photo = photo;
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.PhotoChanged, null);
        await repository.SaveChangesAsync(cancellationTokens);

        if (oldFile is not null) await fileService.DeleteAsync(oldFile.StoragePath,
cancellationTokens);
    }

```



```

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> AddMembersAsync(int currentUserId, int chatId,
AddGroupMembersRequest request, CancellationToken cancellationToken = default)
    {
        var validation = await addMembersValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var existingIds = group.Chat.Members.Select(member =>
member.UserId).ToHashSet();
        var newIds = request.UserIds.Where(id =>
!existingIds.Contains(id)).Distinct().ToArray();
        if (newIds.Length == 0) return Fail("Tanlangan foydalanuvchilar allaqachon
guruhda.");
        var maxMembers = GroupLimits.MaxMembersFor(group.OrganizationId is not null);
        if (existingIds.Count + newIds.Length > maxMembers) return Fail($"Guruhda
{maxMembers} tadan ortiq a'zo bo'lishi mumkin emas.");

        var users = await repository.FindUsersAsync(newIds, cancellationToken);
        if (users.Count != newIds.Length) return Fail("Ba'zi foydalanuvchilar
topilmadi.");

        var now = timeProvider.GetUtcNow().UtcDateTime;
        foreach (var user in users)
        {
            group.Chat.Members.Add(GroupFactory.CreateMember(user,
ChatMemberRole.Member, now));
        }

        var names = string.Join(", ", users.Select(GroupMapper.DisplayName));
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MembersAdded, names);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> RemoveMemberAsync(int currentUserId, int chatId,
int userId, CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.UserId == currentUserId) return Fail("Guruhdan chiqish uchun
\"Guruhdan chiqish\" amalidan foydalaning.");
        if (!CanRemove(actor, target)) return Fail(ForbiddenError);

        RemoveMember(group, target);
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MemberRemoved, GroupMapper.DisplayName(target.User));

```

```

        await repository.SaveChangesAsync(cancellationToken);

        await realtimeNotifier.ChatDeletedAsync(chatId, [target.UserId],
cancellationToken);
        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateMemberRoleAsync(int currentUserId, int
chatId, int userId, UpdateGroupMemberRoleRequest request, CancellationTok
cancellationToken = default)
    {
        var validation = await updateRoleValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (actor.Role != ChatMemberRole.Creator) return Fail("Faqat guruh egasi
adminlarni tayinlay oladi.");

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.Role == ChatMemberRole.Creator) return Fail("Guruh egasining rolini
o'zgartirib bo'lmaydi.");

        target.Role = request.Role;
        await repository.SaveChangesAsync(cancellationToken);

        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupLeaveResult> LeaveAsync(int currentUserId, int chatId,
CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var leaver = group is null ? null : FindMember(group, currentUserId);
        if (group is null || leaver is null) return new(false, NotFoundError);

        var now = timeProvider.GetUtcNow().UtcDateTime;
        RemoveMember(group, leaver);

        if (group.Chat.Members.Count == 0)
        {
            group.Chat.DeletedAt = now;
            await repository.SaveChangesAsync(cancellationToken);
            await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
cancellationToken);
            return new(true, null);
        }

        if (leaver.Role == ChatMemberRole.Creator)
        {
            // Ownership passes to the longest-serving admin, or to the longest-
serving member if there are no admins.
            var successor = group.Chat.Members
                .OrderByDescending(member => member.Role == ChatMemberRole.Admin)
                .ThenBy(member => member.JoinedAt)
                .ThenBy(member => member.Id)
                .First();
            successor.Role = ChatMemberRole.Creator;
            group.CreatorId = successor.UserId;
        }
    }

```

```

        group.Creator = successor.User;
    }

    var serviceMessage = AddServiceMessage(group, leaver.User,
MessageServiceAction.MemberLeft, GroupMapper.DisplayName(leaver.User));
    await repository.SaveChangesAsync(cancellationToken);

    await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
cancellationToken);
    await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
    await PublishGroupUpdatedAsync(group, cancellationToken);
    return new(true, null);
}

public async Task<GroupResult> CreateInviteLinkAsync(int currentUserId, int
chatId, CancellationToken cancellationToken = default)
{
    var group = await repository.GetGroupAsync(chatId, cancellationToken);
    var actor = group is null ? null : FindMember(group, currentUserId);
    if (group is null || actor is null) return Fail(NotFoundError);
    if (!CanManageGroup(actor)) return Fail(ForbiddenError);

    // A new token also revokes the previous link.
    group.InviteLink = GroupInviteTokens.Create();
    await repository.SaveChangesAsync(cancellationToken);
    return new(ToDto(group, currentUserId), null);
}

public async Task<GroupResult> RevokeInviteLinkAsync(int currentUserId, int
chatId, CancellationToken cancellationToken = default)
{
    var group = await repository.GetGroupAsync(chatId, cancellationToken);
    var actor = group is null ? null : FindMember(group, currentUserId);
    if (group is null || actor is null) return Fail(NotFoundError);
    if (!CanManageGroup(actor)) return Fail(ForbiddenError);

    group.InviteLink = null;
    await repository.SaveChangesAsync(cancellationToken);
    return new(ToDto(group, currentUserId), null);
}

public async Task<GroupInvitePreviewDto?> GetInvitePreviewAsync(int currentUserId,
string token, CancellationToken cancellationToken = default)
{
    if (!GroupInviteTokens.IsWellFormed(token)) return null;
    var group = await repository.GetGroupByInviteTokenAsync(token,
cancellationToken);
    return group is null ? null : GroupMapper.ToInvitePreview(group,
FindMember(group, currentUserId) is not null);
}

public async Task<GroupResult> JoinByInviteAsync(int currentUserId, string token,
CancellationToken cancellationToken = default)
{
    const string invalidLinkError = "Taklif havolasi yaroqsiz yoki bekor
qilingan.";
    if (!GroupInviteTokens.IsWellFormed(token)) return Fail(invalidLinkError);
    var group = await repository.GetGroupByInviteTokenAsync(token,
cancellationToken);
    if (group is null) return Fail(invalidLinkError);
    if (FindMember(group, currentUserId) is not null) return new(ToDto(group,
currentUserId), null);

    var maxMembers = GroupLimits.MaxMembersFor(group.OrganizationId is not null);
    if (group.Chat.Members.Count >= maxMembers) return Fail($"Guruhda {maxMembers}
tadan ortiq a'zo bo'lishi mumkin emas.");
}

```

```

        var user = (await repository.FindUsersAsync([currentUserId],
cancellationToken)).FirstOrDefault();
        if (user is null) return Fail("Foydalanuvchi aniqlanmadi.");

        group.Chat.Members.Add(GroupFactory.CreateMember(user, ChatMemberRole.Member,
timeProvider.GetUtcNow().UtcDateTime));
        var serviceMessage = AddServiceMessage(group, user,
MessageServiceAction.MemberJoinedViaInvite, GroupMapper.DisplayName(user));
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<bool> CreateOrganizationGroupAsync(int userId, Cancellation token
cancellationToken = default)
    {
        var owner = (await repository.FindUsersAsync([userId],
cancellationToken)).FirstOrDefault();
        var organization = owner?.OrganizationMembership?.Organization;
        if (owner is null || organization is null) return false;

        var members = await repository.FindOrganizationUsersAsync(
            organization.Id, [userId], GroupLimits.MaxOrganizationMembers - 1,
cancellationToken);
        var group = GroupFactory.Create(
            organization.Domain,
            $"@{organization.Domain} pochta si bilan kirganlar avtomatik qo'shiladi.
Boshqalar faqat admin qo'shganda yoki taklif havolasi orqali kiradi.",
            owner, members, timeProvider.GetUtcNow().UtcDateTime, organization);
        await repository.AddGroupAsync(group, cancellationToken);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(group.Chat.Messages.Single(), group,
cancellationToken);
        return true;
    }

    public async Task<bool> JoinViaOrganizationAsync(int chatId, int userId,
Cancellation token cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        if (group?.Organization is null || FindMember(group, userId) is not null)
return false;
        if (group.Chat.Members.Count >= GroupLimits.MaxOrganizationMembers) return
false;

        var user = (await repository.FindUsersAsync([userId],
cancellationToken)).FirstOrDefault();
        if (user is null || !IsOrganizationMember(user, group.Organization.Id)) return
false;

        group.Chat.Members.Add(GroupFactory.CreateMember(user, ChatMemberRole.Member,
timeProvider.GetUtcNow().UtcDateTime));
        var serviceMessage = AddServiceMessage(group, user,
MessageServiceAction.MemberJoinedViaOrganization, group.Organization.Domain);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return true;
    }

```

```

private static bool IsOrganizationMember(User user, int organizationId) =>
    user.OrganizationMembership?.OrganizationId == organizationId;

private static bool CanManageGroup(ChatMember actor) =>
    actor.Role is ChatMemberRole.Admin or ChatMemberRole.Creator;

private static bool CanRemove(ChatMember actor, ChatMember target) => actor.Role
switch
{
    ChatMemberRole.Creator => true,
    ChatMemberRole.Admin => target.Role == ChatMemberRole.Member,
    _ => false
};

private static ChatMember? FindMember(Group group, int userId) =>
    group.Chat.Members.FirstOrDefault(member => member.UserId == userId);

private void RemoveMember(Group group, ChatMember member)
{
    group.Chat.Members.Remove(member);
    repository.RemoveMember(member);
}

private Message AddServiceMessage(Group group, User actor, MessageServiceAction
action, string? textContent)
{
    var message = MessageFactory.CreateService(group.ChatId, actor.Id, action,
textContent, timeProvider.GetUtcNow().UtcDateTime);
    message.Sender = actor;
    group.Chat.Messages.Add(message);
    return message;
}

private Task PublishServiceMessageAsync(Message message, Group group,
Cancellation token cancellationToken) =>
    realtimeNotifier.MessageCreatedAsync(MessageMapper.ToDto(message),
MemberIds(group), cancellationToken);

private Task PublishGroupUpdatedAsync(Group group, Cancellation token
cancellationToken) =>
    realtimeNotifier.GroupUpdatedAsync(group.ChatId, group.Title,
group.Description, group.PhotoId, group.Chat.Members.Count, MemberIds(group),
cancellationToken);

private static IReadOnlyCollection<int> MemberIds(Group group) =>
    group.Chat.Members.Select(member => member.UserId).ToArray();

private GroupDto ToDto(Group group, int currentUserId) =>
    GroupMapper.ToDto(group, currentUserId, presenceTracker.IsOnline);

private static string? NormalizeDescription(string? description) =>
    string.IsNullOrEmpty(description) ? null : description.Trim();

private static GroupResult Fail(string error) => new(null, error);
}

```

3-ilova. Bir martalik kodlarni xeshlash (PBKDF2)

Fayl:

NationalChat/src/Infrastructure/Security/Hashing/Pbkdf2OneTimeCodeHasher.cs

```
using System.Security.Cryptography;
using Application.Features.Authentication;

namespace Infrastructure.Security.Hashing;

public sealed class Pbkdf2OneTimeCodeHasher : IOneTimeCodeHasher
{
    private const int Iterations = 210_000;
    private const int SaltSize = 16;
    private const int HashSize = 32;

    public string Hash(string code)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var hash = Rfc2898DeriveBytes.Pbkdf2(code, salt, Iterations,
        HashAlgorithmName.SHA256, HashSize);
        return
        $"{Iterations}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }

    public bool Verify(string code, string storedHash)
    {
        var parts = storedHash.Split('.', StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length != 3 || !int.TryParse(parts[0], out var iterations))
        {
            return false;
        }

        try
        {
            var salt = Convert.FromBase64String(parts[1]);
            var expectedHash = Convert.FromBase64String(parts[2]);
            var actualHash = Rfc2898DeriveBytes.Pbkdf2(code, salt, iterations,
            HashAlgorithmName.SHA256, expectedHash.Length);
            return CryptographicOperations.FixedTimeEquals(actualHash, expectedHash);
        }
        catch (FormatException)
        {
            return false;
        }
    }
}
```

4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)

Fayl: NationalChatClient/src/app/features/chat/data-access/chat-realtime.service.ts

```
async connect(): Promise<void> {
    if (!isPlatformBrowser(this.platformId) || !this.auth.accessToken()) return;
    if (!this.connection) this.createConnection();
    if (this.connection!.state === HubConnectionState.Connected) return;
    if (this.connectPromise) return this.connectPromise;

    this.connectionState.set('connecting');
    this.connectPromise = this.connection!.start()
        .then(() => this.connectionState.set('connected'))
}
```

```

        .catch((error: unknown) => {
            this.connectionState.set('disconnected');
            throw error;
        })
        .finally(() => (this.connectPromise = undefined));
    return this.connectPromise;
}

async joinChat(chatId: number): Promise<void> {
    const previousChatId = this.joinedChatId;
    this.joinedChatId = chatId;
    await this.connect();
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    if (previousChatId && previousChatId !== chatId)
        await this.connection.invoke('LeaveChat', previousChatId);
    await this.connection.invoke('JoinChat', chatId);
}

async leaveChat(chatId: number): Promise<void> {
    if (this.joinedChatId === chatId) this.joinedChatId = null;
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    await this.connection.invoke('LeaveChat', chatId);
}

async setTyping(chatId: number, isTyping: boolean): Promise<void> {
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    await this.connection.send('SetTyping', chatId, isTyping);
}

async markMessagesRead(chatId: number, messageIds: number[]): Promise<void> {
    if (
        !messageIds.length ||
        !this.connection ||
        this.connection.state !== HubConnectionState.Connected
    )
        return;
    await this.connection.send('MarkMessagesRead', chatId, messageIds);
}

async disconnect(): Promise<void> {
    this.joinedChatId = null;
    if (this.connection && this.connection.state !== HubConnectionState.Disconnected)
        await this.connection.stop();
    this.connectionState.set('disconnected');
}

private createConnection(): void {
    this.connection = new HubConnectionBuilder()
        .withUrl(apiUrl('/hubs/chat'), { accessTokenFactory: () =>
this.auth.accessToken() ?? '' })
        .withAutomaticReconnect([0, 2_000, 5_000, 10_000])
        .configureLogging(LogLevel.Warning)
        .build();

    this.connection.on('MessageReceived', (message: Message) =>
        this.messageReceivedSubject.next(message),
    );
    this.connection.on('TypingChanged', (event: TypingChangedEvent) =>
        this.typingChangedSubject.next(event),
    );
    this.connection.on('MessagesRead', (event: MessagesReadEvent) =>
        this.messagesReadSubject.next(event),
    );
}

```

```

        this.connection.on('MessageUpdated', (message: Message) =>
this.messageUpdatedSubject.next(message));
        this.connection.on('MessageDeleted', (event: { chatId: number; messageId: number
}) => this.messageDeletedSubject.next(event));
        this.connection.on('ChatCleared', (event: { chatId: number }) =>
this.chatClearedSubject.next(event));
        this.connection.on('ChatDeleted', (event: { chatId: number }) =>
this.chatDeletedSubject.next(event));
        this.connection.on('ProfilePhotoUpdated', (event: ProfilePhotoUpdatedEvent) =>
this.profilePhotoUpdatedSubject.next(event));
        this.connection.on('UserPresenceChanged', (event: UserPresenceChangedEvent) =>
this.userPresenceChangedSubject.next(event));
        this.connection.on('GroupUpdated', (event: GroupUpdatedEvent) =>
this.groupUpdatedSubject.next(event));
        this.connection.on('SecretChatRequested', (chat: SecretChatDto) =>
this.secretChatRequestedSubject.next(chat));
        this.connection.on('SecretChatAccepted', (chat: SecretChatDto) =>
this.secretChatAcceptedSubject.next(chat));
        this.connection.on('SecretMessageReceived', (message: SecretMessageDto) =>
this.secretMessageReceivedSubject.next(message));
        this.connection.on('SecretChatClosed', (event: { secretChatId: number }) =>
this.secretChatClosedSubject.next(event));
        this.connection.onreconnecting(() => this.connectionState.set('reconnecting'));
        this.connection.onreconnected(async () => {
            this.connectionState.set('connected');
            if (this.joinedChatId) await this.connection?.invoke('JoinChat',
this.joinedChatId);
            this.reconnectedSubject.next();
        });
        this.connection.onclose(() => this.connectionState.set('disconnected'));

```

5-ilova. Guruh chatlari uchun unit testlar (parcha)

Fayl: NationalChat/tests/NationalChat.Tests/Groups/GroupServiceTests.cs

```

[Fact]
public async Task
Create_ValidRequest_CreatesGroupWithCreatorRoleAndServiceMessage()
{
    var result = await _service.CreateAsync(_sardor.Id, new
CreateGroupRequest("315-21 guruh", "TATU", [_jasur.Id, _dilnoza.Id]));

    Assert.Null(result.Error);
    Assert.NotNull(result.Group);
    Assert.Equal(ChatMemberRole.Creator, result.Group!.MyRole);
    Assert.Equal(3, result.Group.Members.Count);

    var chat = Assert.Single(_repository.Groups).Chat;
    Assert.Equal(ChatType.Group, chat.Type);
    var message = Assert.Single(chat.Messages);
    Assert.Equal(MessageServiceAction.GroupCreated, message.ServiceAction);

    await _notifier.Received(1).MessageCreatedAsync(
        Arg.Is<MessageDto>(m => m.ServiceAction ==
MessageServiceAction.GroupCreated),
        Arg.Is<IReadOnlyCollection<int>>(ids => ids.Count == 3),
        Arg.Any<CancellationToken>());
}

[Fact]
public async Task Create_UnknownMember_IsRejected()
{
    var result = await _service.CreateAsync(_sardor.Id, new
CreateGroupRequest("Guruh", null, [_jasur.Id, 999]));

```



```

        Assert.Null(result.Group);
        Assert.NotNull(result.Error);
        Assert.Empty(_repository.Groups);
    }

    [Fact]
    public async Task Create_EmptyTitle_FailsValidation()
    {
        var result = await _service.CreateAsync(_sardor.Id, new CreateGroupRequest("
", null, [_jasur.Id]));

        Assert.Null(result.Group);
        Assert.Empty(_repository.Groups);
    }

```

6-ilova. O‘zbek yozuvlari o‘rtasida transliteratsiya (UzbekTransliterator)

Fayl: NationalChat/src/Domain/Text/UzbekTransliterator.cs

```

using System.Text;
using System.Text.RegularExpressions;

namespace Domain.Text;

/// <summary>
/// Converts Uzbek text between the Cyrillic and the 1995 Latin alphabet.
/// The TypeScript client keeps an identical copy (uzbek-transliterator.ts); both are
/// tested
/// against the same transliteration-cases.json, so any rule change must be made in
/// both places.
/// URLs, e-mail addresses, @usernames, #tags and `code` spans are never converted.
/// </summary>
public static partial class UzbekTransliterator
{
    /// <summary>Letter part of o' and g' in the output of <see
    cref="ToLatin"/>.</summary>
    public const char OkinaMark = 'ʻ';

    /// <summary>Tutuq belgisi (ʻ) in the output of <see cref="ToLatin"/>.</summary>
    public const char TutuqMark = 'ʼ';

    private const string Apostrophes = "ʻʼ`'";
    private const string CyrillicVowels = "aeёиоуўэюяы";
    private const string LatinVowels = "aeiou";

    // Loanwords where Latin "ts" stands for Cyrillic "ц"; everywhere else "ts" is
    "тс".
    private static readonly string[] TsExceptionStems =
    [
        "tsiy", "tsion", "litsey", "patsient", "ofitsiant", "ofitser", "spetsial",
        "sotsial", "abzats", "kvarts"
    ];

    private static readonly Dictionary<char, string> CyrillicToLatinMap = new()
    {
        ['a'] = "a", ['б'] = "b", ['в'] = "v", ['г'] = "g", ['ё'] = "g'", ['д'] = "d",
        ['ë'] = "yo",
        ['ж'] = "j", ['з'] = "z", ['и'] = "i", ['й'] = "y", ['к'] = "k", ['қ'] = "q",
        ['л'] = "l",
        ['м'] = "m", ['н'] = "n", ['о'] = "o", ['п'] = "p", ['р'] = "r", ['с'] = "s",
        ['т'] = "t",

```

```

        ['y'] = "u", ['ŷ'] = "o'", ['ф'] = "f", ['x'] = "x", ['x'] = "h", ['u'] =
"ch", ['ш'] = "sh",
        ['ш'] = "sh", ['ъ'] = "'", ['ы'] = "i", ['ь'] = "", ['э'] = "e", ['ю'] = "yu",
['я'] = "ya"
    };

    private static readonly Dictionary<char, char> LatinToCyrillicMap = new()
    {
        ['a'] = 'a', ['b'] = 'б', ['c'] = 'c', ['d'] = 'д', ['f'] = 'ф', ['g'] = 'г',
['h'] = 'х',
        ['i'] = 'и', ['j'] = 'ж', ['k'] = 'к', ['l'] = 'л', ['m'] = 'м', ['n'] = 'н',
['o'] = 'o',
        ['p'] = 'п', ['q'] = 'к', ['r'] = 'р', ['s'] = 'с', ['t'] = 'т', ['u'] = 'у',
['v'] = 'в',
        ['w'] = 'в', ['x'] = 'x', ['y'] = 'й', ['z'] = 'з'
    };

    // Order matters: e-mail must win over @username, which would otherwise match its
    // domain part.
    [GeneratedRegex(
        @"`[^`]*`|(?:(https?:|www\.)\S+|[\p{L}\p{N}._%+-]+@[ \p{L}\p{N}-
]+(?:\.[ \p{L}\p{N}-]+)+|[@#][ \p{L}\p{N}_]+",
        RegexOptions.IgnoreCase)]
    private static partial Regex ProtectedSpanRegex();

    [GeneratedRegex(@"\s+")]
    private static partial Regex WhitespaceRegex();

    public static string ToLatin(string? text) => ConvertOutsideProtectedSpans(text,
    ToLatinSegment);

    public static string ToCyrillic(string? text) =>
    ConvertOutsideProtectedSpans(text, ToCyrillicSegment);

    /// <summary>
    /// Script-independent search key: Latin, lower case, one apostrophe form, single
    /// spaces.
    /// "Ўзбек", "o'zbek", "o'zbek" and "O'zbek" all become "o'zbek".
    /// </summary>
    public static string NormalizeForSearch(string? text)
    {
        if (string.IsNullOrEmpty(text)) return string.Empty;

        var latin = ToLatin(text).ToLowerInvariant();
        var builder = new StringBuilder(latin.Length);
        foreach (var ch in latin) builder.Append(IsApostrophe(ch) ? '\'' : ch);
        return WhitespaceRegex().Replace(builder.ToString(), " ").Trim();
    }

    public static bool IsApostrophe(char ch) => Apostrophes.Contains(ch);

    private static string ConvertOutsideProtectedSpans(string? text, Func<string,
    string> convert)
    {
        if (string.IsNullOrEmpty(text)) return text ?? string.Empty;

        var builder = new StringBuilder(text.Length + 8);
        var position = 0;
        foreach (Match match in ProtectedSpanRegex().Matches(text))
        {
            builder.Append(convert(text[position..match.Index]));
            builder.Append(match.Value);
            position = match.Index + match.Length;
        }
    }

```

```

        builder.Append(convert(text[position..]));
        return builder.ToString();
    }

    private static string ToLatinSegment(string text)
    {
        var builder = new StringBuilder(text.Length + 8);
        for (var i = 0; i < text.Length; i++)
        {
            var ch = text[i];
            var lower = char.ToLowerInvariant(ch);
            var previous = i > 0 ? char.ToLowerInvariant(text[i - 1]) : '\0';

            string latin;
            if (lower == 'e')
            {
                var soundsYe = !IsWordChar(text, i - 1) ||
CyrillicVowels.Contains(previous) || previous is 'ѣ' or 'ѥ';
                latin = soundsYe ? "ye" : "e";
            }
            else if (lower == 'ц')
            {
                latin = CyrillicVowels.Contains(previous) ? "ts" : "s";
            }
            else if (lower == 'х' && previous == 'с')
            {
                // "cx" (Исхок) is written with a tutuq in Latin so that it is not
read as "ш".
                latin = TutuqMark + "h";
            }
            else if (!CyrillicToLatinMap.TryGetValue(lower, out latin!))
            {
                builder.Append(ch);
                continue;
            }

            builder.Append(ch == lower ? latin : ApplyUpperCase(latin,
IsUpperCaseWord(text, i)));
        }

        return builder.ToString();
    }

    private static string ToCyrillicSegment(string text)
    {
        var lower = text.ToLowerInvariant();
        var tsPositions = FindExceptionTsPositions(lower);
        var builder = new StringBuilder(text.Length);
        var i = 0;
        while (i < text.Length)
        {
            var c = lower[i];
            var next = i + 1 < text.Length ? lower[i + 1] : '\0';
            var afterNext = i + 2 < text.Length ? lower[i + 2] : '\0';
            var isUpper = text[i] != c;

            string? cyrillic = null;
            var length = 1;
            if (c == 's' && next == 'h') (cyrillic, length) = ("ш", 2);
            else if (c == 's' && IsApostrophe(next) && afterNext == 'h') (cyrillic,
length) = ("ш", 2);
            else if (c == 'c' && next == 'h') (cyrillic, length) = ("ч", 2);
            else if (c == 'o' && IsApostrophe(next)) (cyrillic, length) = ("ѣ", 2);
            else if (c == 'g' && IsApostrophe(next)) (cyrillic, length) = ("ѥ", 2);
            else if (c == 'y' && next == 'o' && !IsApostrophe(afterNext)) (cyrillic,
length) = ("ѧ", 2);
            else if (c == 'y' && next == 'u') (cyrillic, length) = ("Ѩ", 2);
        }
    }

```

```

        else if (c == 'y' && next == 'a') (cyrillic, length) = ("я", 2);
        else if (c == 'y' && next == 'e') (cyrillic, length) = ("е", 2);
        else if (c == 't' && next == 's' && tsPositions.Contains(i)) (cyrillic,
length) = ("ц", 2);
        else if (c == 'e')
        {
            // Cyrillic "e" after a vowel or at a word start is written "ye", so a
bare "e" there was "э".
            var previous = i > 0 ? lower[i - 1] : '\0';
            cyrillic = !IsWordChar(text, i - 1) || LatinVowels.Contains(previous)
? "э" : "е";
        }
        else if (IsApostrophe(c) && IsWordChar(text, i - 1) && IsLetter(text, i +
1))
        {
            cyrillic = IsUpperCaseWord(text, i) ? "Ъ" : "ь";
        }
        else if (LatinToCyrillicMap.TryGetValue(c, out var single))
        {
            cyrillic = single.ToString();
        }

        if (cyrillic is null)
        {
            builder.Append(text[i]);
        }
        else
        {
            builder.Append(isUpper ? cyrillic.ToUpperInvariant() : cyrillic);
        }

        i += length;
    }

    return builder.ToString();
}

private static HashSet<int> FindExceptionTsPositions(string lowerText)
{
    var positions = new HashSet<int>();
    foreach (var stem in TsExceptionStems)
    {
        for (var start = lowerText.IndexOf(stem, StringComparison.Ordinal); start
>= 0; start = lowerText.IndexOf(stem, start + 1, StringComparison.Ordinal))
        {
            for (var ts = stem.IndexOf("ts", StringComparison.Ordinal); ts >= 0;
ts = stem.IndexOf("ts", ts + 2, StringComparison.Ordinal))
            {
                positions.Add(start + ts);
            }
        }
    }

    return positions;
}

private static string ApplyUpperCase(string latin, bool wholeWordUpper)
{
    if (latin.Length == 0) return latin;
    return wholeWordUpper ? latin.ToUpperInvariant() :
char.ToUpperInvariant(latin[0]) + latin[1..];
}

private static bool IsLetter(string text, int index) =>
    index >= 0 && index < text.Length && char.IsLetter(text[index]) &&
!IsApostrophe(text[index]);

```

```

    private static bool IsWordChar(string text, int index) =>
        index >= 0 && index < text.Length && (char.IsLetter(text[index]) ||
        IsApostrophe(text[index]));

    /// <summary>True when the word around <paramref name="index"/> has at least two
    cased letters, all upper case.</summary>
    private static bool IsUpperCaseWord(string text, int index)
    {
        var start = index;
        while (IsWordChar(text, start - 1)) start--;
        var end = index;
        while (IsWordChar(text, end + 1)) end++;

        var casedLetters = 0;
        for (var i = start; i <= end; i++)
        {
            var ch = text[i];
            if (char.ToUpperInvariant(ch) == char.ToLowerInvariant(ch)) continue;
            if (!char.IsUpper(ch)) return false;
            casedLetters++;
        }

        return casedLetters >= 2;
    }
}

```

7-ilova. Maxfiy chatlar kriptografiyasi: kalitlarni chiqarish, shifrlash va ratchet (secret-crypto.ts, parcha)

Fayl: NationalChatClient/src/app/features/chat/secret/secret-crypto.ts

```

export async function deriveChains(
    role: SecretRole,
    secretChatId: number,
    myKeyPair: CryptoKeyPair,
    initiatorPublicKey: string,
    participantPublicKey: string,
): Promise<{ send: ChainState; receive: ReceiveState }> {
    const peerPublicKey = role === 'initiator' ? participantPublicKey :
    initiatorPublicKey;
    const peerKey = await subtle().importKey('raw', publicKeyBytes(peerPublicKey), {
    name: 'X25519' }, false, []);
    const shared = new Uint8Array(
        await subtle().deriveBits({ name: 'X25519', public: peerKey },
    myKeyPair.privateKey, KEY_BYTES * 8),
    );
    try {
        if (shared.every((byte) => byte === 0)) throw new SecretCryptoError('Ochiq kalit
    yaroqsiz.');
```

```

        const hkdfKey = await subtle().importKey('raw', shared, 'HKDF', false,
    ['deriveBits']);
        const salt = new Uint8Array(await subtle().digest('SHA-256',
    encoder.encode(HKDF_SALT_LABEL)));
        const info = encoder.encode(`nc-secret-
    v${PROTOCOL_VERSION}|${secretChatId}|${initiatorPublicKey}|${participantPublicKey}`);
        const okm = new Uint8Array(await subtle().deriveBits({ name: 'HKDF', hash: 'SHA-
    256', salt, info }, hkdfKey, KEY_BYTES * 2 * 8));
        try {
            const initiatorChain = await importChainKey(okm.subarray(0, KEY_BYTES));
            const participantChain = await importChainKey(okm.subarray(KEY_BYTES));
            const [send, receive] = role === 'initiator'
                ? [initiatorChain, participantChain]

```

```

        : [participantChain, initiatorChain];
        return { send: { chainKey: send, seq: 0 }, receive: { chainKey: receive, seq: 0,
skipped: new Map() } };
    } finally {
        okm.fill(0);
    }
    } finally {
        shared.fill(0);
    }
}

/**
 * Encrypts the next message. Returns the new chain state, which the caller must
persist before sending
 * so that a key is never used twice.
 */
export async function encryptMessage(
    chain: ChainState,
    secretChatId: number,
    direction: SecretDirection,
    payload: SecretPayload,
): Promise<{ chain: ChainState; seq: number; ciphertext: string }> {
    if (!isValidPayload(payload)) {
        throw new SecretCryptoError(
            payload.kind === 'text' ? `Xabar 1 dan ${MAX_TEXT_LENGTH} belgigacha bo'lishi
kerak.` : 'Xabar formati noto'g'ri.',
        );
    }

    const seq = chain.seq + 1;
    const { messageKey, nextChainKey } = await step(chain.chainKey);
    const iv = globalThis.crypto.getRandomValues(new Uint8Array(IV_BYTES));
    const plaintext = encoder.encode(JSON.stringify({ v: PROTOCOL_VERSION, ...payload
}));
    const sealed = new Uint8Array(
        await subtle().encrypt({ name: 'AES-GCM', iv, additionalData:
associatedData(secretChatId, direction, seq) }, messageKey, plaintext),
    );

    const envelope = new Uint8Array(1 + IV_BYTES + sealed.length);
    envelope[0] = PROTOCOL_VERSION;
    envelope.set(iv, 1);
    envelope.set(sealed, 1 + IV_BYTES);
    return { chain: { chainKey: nextChainKey, seq }, seq, ciphertext: toBase64(envelope)
};
}

/**
 * Decrypts a message from the peer. The returned state is only valid if decryption
succeeded; on any
 * failure this throws and the caller keeps its old state, so a forged message cannot
move the chain.
 */
export async function decryptMessage(
    state: ReceiveState,
    secretChatId: number,
    direction: SecretDirection,
    seq: number,
    ciphertext: string,
): Promise<{ state: ReceiveState; payload: SecretPayload }> {
    if (!Number.isSafeInteger(seq) || seq < 1) throw new SecretCryptoError('Xabar tartib
raqami noto'g'ri.');
```

```

    const envelope = fromBase64(ciphertext);
    if (envelope.length < 1 + IV_BYTES + 16 || envelope[0] !== PROTOCOL_VERSION) {
        throw new SecretCryptoError('Xabar formati noto'g'ri.');
```

```

const skipped = new Map(state.skipped);
let chainKey = state.chainKey;
let chainSeq = state.seq;
let messageKey: CryptoKey;

if (seq <= chainSeq) {
  const key = skipped.get(seq);
  if (!key) throw new SecretCryptoError('Bu xabar allaqachon o'qilgan yoki juda eski.');
```

eski.');

```

  skipped.delete(seq);
  messageKey = key;
} else {
  if (seq - chainSeq > MAX_SKIP) throw new SecretCryptoError('Juda ko'p xabar o'tkazib yuborilgan.');
```

o'tkazib yuborilgan.');

```

  while (chainSeq < seq - 1) {
    const next = await step(chainKey);
    chainSeq++;
    skipped.set(chainSeq, next.messageKey);
    chainKey = next.nextChainKey;
  }
  const next = await step(chainKey);
  messageKey = next.messageKey;
  chainKey = next.nextChainKey;
  chainSeq = seq;
  // Oldest skipped keys go first once there are too many.
  while (skipped.size > MAX_SKIP) skipped.delete(skipped.keys().next().value!);
}

let plaintext: ArrayBuffer;
try {
  plaintext = await subtle().decrypt(
    { name: 'AES-GCM', iv: envelope.subarray(1, 1 + IV_BYTES), additionalData:
associatedData(secretChatId, direction, seq) },
    messageKey,
    envelope.subarray(1 + IV_BYTES),
  );
} catch {
  throw new SecretCryptoError('Xabarni ochib bo'lmadi: u buzilgan yoki soxta.');
```

throw new SecretCryptoError('Xabarni ochib bo'lmadi: u buzilgan yoki soxta.');

```

}

return { state: { chainKey, seq: chainSeq, skipped }, payload:
parsePayload(plaintext) };
}

export interface SecretFingerprint {
  /** Eight emoji; both devices must see the same row. */
  emoji: string[];
  /** Six groups of five digits, for reading out loud. */
  digits: string[];
}

/**
 * Key fingerprint shown on both devices. If a man in the middle swapped the public
keys, the two devices
 * would see different values.
 */
export async function computeFingerprint(initiatorPublicKey: string,
participantPublicKey: string): Promise<SecretFingerprint> {
  const initiator = publicKeyBytes(initiatorPublicKey);
  const participant = publicKeyBytes(participantPublicKey);
  const label = encoder.encode('nc-secret-fingerprint-v1');
  const input = new Uint8Array(label.length + initiator.length + participant.length);
  input.set(label, 0);
  input.set(initiator, label.length);
  input.set(participant, label.length + initiator.length);
  const hash = new Uint8Array(await subtle().digest('SHA-256', input));
}

```

```

const emoji: string[] = [];
let bits = 0;
let bitCount = 0;
for (let index = 0; emoji.length < 8; index++) {
  bits = (bits << 8) | hash[index];
  bitCount += 8;
  while (bitCount >= 6 && emoji.length < 8) {
    bitCount -= 6;
    emoji.push(FINGERPRINT_EMOJI[(bits >> bitCount) & 0x3f]);
  }
}

const digits: string[] = [];
for (let group = 0; group < 6; group++) {
  let value = 0;
  for (let offset = 0; offset < 5; offset++) value = value * 256 + hash[group * 5 + offset];
  digits.push(String(value % 100_000).padStart(5, '0'));
}
return { emoji, digits };
}

/** Decodes and checks a 32-byte X25519 public key in standard Base64. */
export function publicKeyBytes(value: string): Uint8Array<ArrayBuffer> {
  const bytes = fromBase64(value);
  if (bytes.length !== KEY_BYTES) throw new SecretCryptoError('Ochiq kalit noto`g`ri.');
  return bytes;
}

async function step(chainKey: CryptoKey): Promise<{ messageKey: CryptoKey; nextChainKey: CryptoKey }> {
  const messageBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, MESSAGE_KEY_INPUT));
  const chainBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, CHAIN_KEY_INPUT));
  try {
    const messageKey = await subtle().importKey('raw', messageBytes, { name: 'AES-GCM' }, false, ['encrypt', 'decrypt']);
    return { messageKey, nextChainKey: await importChainKey(chainBytes) };
  } finally {
    messageBytes.fill(0);
    chainBytes.fill(0);
  }
}

function importChainKey(bytes: Uint8Array<ArrayBuffer>): Promise<CryptoKey> {
  return subtle().importKey('raw', bytes, { name: 'HMAC', hash: 'SHA-256' }, false, ['sign']);
}

function associatedData(secretChatId: number, direction: SecretDirection, seq: number): Uint8Array<ArrayBuffer> {

```